Продуктовый дизайн

Учебник с примерами из практики

Версия 3 · 31.10.2026

Введение

Учебник собирает основы продуктового дизайна: от постановки задачи и исследований до интерфейса, дизайн-систем, передачи в разработку и метрик. Каждая глава сочетает общие знания профессии с примерами из реальных проектов. У каждого правила и решения объяснено, почему оно устроено так.

Откуда примеры

Блоки «Из практики» взяты из трёх продуктов медицинского научного центра:

- **Кабинет участника клинических исследований.** Человек подаёт заявку на исследование за себя или ребёнка, проходит согласие, анкету, запись на визит и получает результаты. Около 4000 участников, половина из них дети, несколько исследований с повторными визитами.
- **Кабинет врача-модератора.** Врач и координатор проверяют заявки, оставляют замечания к полям, ведут запись на визит. Второй продукт на той же дизайн-системе.
- **Станция биобанка.** Рабочее место лаборанта, который раскладывает аликвоты по штативам и связывает штрихкоды пробирок с ячейками.

Примеры взяты из баз для кейсов и журналов проектов. Цифры и цитаты приведены по этим материалам. Где решение принимала автор, где заказчик, где идею предложили коллеги и где работал ИИ-ассистент, сказано отдельно. Кадры сняты с прототипов этих продуктов.

Как читать объяснения

У каждого правила есть объяснение, почему оно такое. В примерах из практики объяснения бывают трёх видов, и их стоит различать:

Как подписано	Что значит
«Довод», «Почему так», цитата в кавычках	Причина записана в материалах проекта: её назвали автор, заказчик или эксперты

Как подписано	Что значит
«Что это даёт»	Причина в материалах не записана, объяснение даёт учебник по следствию решения
«Общий принцип»	Решение объяснено общим правилом из главы

Почему это важно: настоящую причину решения и объяснение задним числом легко спутать, а для обучения это разные вещи.

Как читать

Главы можно читать по порядку или выборочно. Порядок повторяет ход работы над продуктом:

1. Что такое продуктовый дизайн.
2. Исследования.
3. Постановка задачи.
4. Информационная архитектура и сценарии.
5. Формы и анкеты.
6. Интерфейс.
7. Движение и микровзаимодействия.
8. Рабочие места и интерфейсы с данными.
9. Дизайн-системы.
10. Работа в Figma.
11. Доступность.
12. Тексты в интерфейсе.
13. Возвращение и удержание.
14. Этика и тёмные паттерны.
15. Интерфейсы с ИИ внутри продукта.
16. Прототипирование и проверка.
17. Передача в разработку.
18. Метрики и итерации.
19. Процессы в команде.
20. Сложные домены.

21. Профессия.

Приложения:

- А. Сквозной разбор кабинета участника: все решения продукта подряд, от задачи до метрик.
- Б. Глоссарий.
- В. Чек-листы.
- Г. Ответы к вопросам для самопроверки.
- Д. Литература и источники.

Как устроена глава

- **Основной текст** объясняет понятие и правила, у каждого правила есть «Почему».
- **Таблицы** собирают методы, критерии и варианты для сравнения.
- **Схемы и кадры** показывают устройство и примеры из прототипов.
- **«Из практики»** показывает, как правило сработало или не сработало на настоящем проекте.
- **«Главное в главе»** даёт итог списком.
- **Вопросы и упражнение** помогают проверить себя и применить знания. Ответы в приложении Г.

Как пользоваться сайтом

- **Поиск** находит слово во всех главах.
- **Отметка «Прочитано»** в конце главы ставит галочку в оглавлении. Отметки и пункты чек-листов хранятся в вашем браузере.
- **PDF** со всем учебником можно скачать по ссылке в оглавлении и читать без интернета.

1. Что такое продуктовый дизайн

Продуктовый дизайнер отвечает за то, чтобы продукт решал задачу человека и задачу бизнеса одновременно. Экраны здесь только одна из форм работы. Не меньше времени уходит на исследования, постановку задачи, логику, состояния, проверку и разговоры с командой.

Чем продуктовый дизайн отличается от соседних профессий

Названия должностей на рынке пересекаются, поэтому полезно различать сами работы.

Работа	Главный вопрос	Результат
Графический дизайн	Как сообщение выглядит и запоминается	Айдентика, плакаты, иллюстрации
UI-дизайн	Как устроен экран: сетка, типографика, цвет, компоненты	Макеты, библиотека компонентов
UX-дизайн	Как человек проходит путь и где спотыкается	Сценарии, прототипы, результаты тестов
Продуктовый дизайн	Какую задачу решаем, для кого, что считать успехом и как это проверить	Решение целиком: от постановки до приёмки и метрик
UX-исследования	Что люди делают на самом деле и почему	Находки, инсайты, рекомендации
Дизайн-аналитика	Что показывают данные о поведении	Метрики, воронки, модели эффекта

Продуктовый дизайнер умеет всё из этой таблицы на рабочем уровне. Глубже всего он погружён в постановку задачи и в решение, которое потом реально строят и измеряют.

Что входит в работу

Типичный цикл выглядит так:

1. Понять проблему: кто страдает, где, сколько это стоит.
2. Собрать ограничения: законы, протоколы, технические возможности, сроки, деньги.
3. Найти решение: сценарии, логика, состояния, интерфейс.
4. Проверить решение: прототип, тест с людьми, прогон сценариев, аудит.
5. Передать в разработку и принять результат.
6. Измерить эффект и решить, что делать дальше.

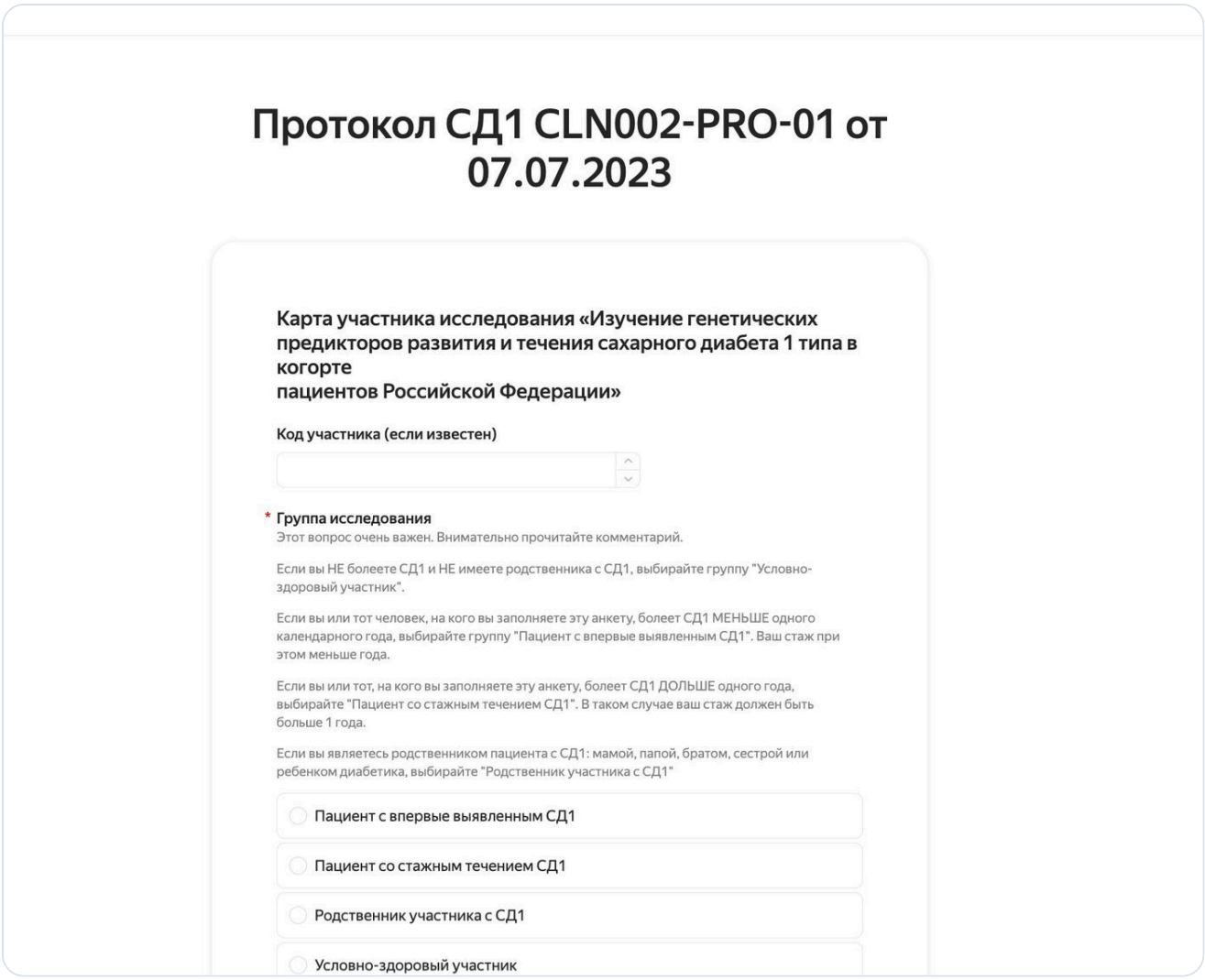
Шаги идут по кругу. Находка на шаге 4 часто возвращает к шагу 1. Почему так: проверка показывает, правильно ли поняли проблему, и иногда выясняется, что нет. Возврат к началу дешевле, чем доводить до запуска решение не той задачи.



Цикл продуктовой работы. Находка на проверке часто возвращает к пониманию проблемы

ИЗ ПРАКТИКИ

В кабинете участника клинических исследований до кабинета данные собирали через Яндекс Формы: одна анкета на сто с лишним полей на одной странице. После отправки человек не получал ни подтверждения, ни статуса, ни ответа врача. Задача здесь сразу продуктовая: данные участника, который ушёл на середине, почти бесполезны для исследования. Поэтому кабинет отвечает за то, чтобы исследование можно было провести до конца, и у каждой части задачи своя метрика.



Старая анкета в Яндекс Формах: перед первым вопросом несколько абзацев о том, к какой группе исследования себя отнести

Три стороны решения

Хорошее продуктовое решение одновременно:

- **ценно для человека**: он решает свою задачу быстрее, спокойнее, с меньшим числом ошибок;
- **выгодно для бизнеса**: растёт удержание, падают затраты, появляются новые возможности;
- **выполнимо**: его можно построить в имеющихся сроках, на имеющейся технике и в рамках закона.

Если одна сторона проседает, решение не живёт. Почему: каждая сторона держит решение по-своему. Без ценности им не пользуются, без выгоды его не развивают, без выполнимости его не построят. Красивый экран, который нельзя построить,

остаётся картинкой. Выполнимое решение без ценности для человека никто не использует.

Роль дизайнера в команде

Продуктовый дизайнер работает на стыке нескольких ролей.

Роль	Что от неё получает дизайнер	Что отдаёт
Продакт-менеджер	Цели, приоритеты, рамки	Варианты решения с ценой каждого
Разработка	Ограничения, оценку сложности	Спецификацию, состояния, крайние случаи
Аналитика	Данные о поведении	Гипотезы, план измерений
Заказчик или эксперт предметной области	Правила процесса	Вопросы, на которые нужно решение
Поддержка	Жалобы и частые вопросы	Тексты, сценарии ошибок

В маленьких командах один дизайнер часто закрывает роли продакта и аналитика. Тогда особенно важно раздельно записывать, что решил сам дизайнер, что решил заказчик и что предложили коллеги.

> **ИЗ ПРАКТИКИ**
> Замечание врача прямо в строке ответа предложили врачи-модераторы. Кейс кабинета участника так и пишет: «Сама я до неё не дошла». Своё, общее и чужое записаны раздельно, и это делает текст о работе честным и проверяемым.

Жизненный цикл продукта

Задачи дизайнера меняются от стадии продукта.

Стадия	Главная задача	Что делает дизайнер
Поиск	Понять, есть ли проблема и стоит ли её решать	Интервью, анализ рынка, концепции
Первая версия	Собрать минимальный путь, который решает задачу	Сценарии, прототип, приоритизация
Рост	Убрать трение, расширить аудиторию	Тесты, метрики, оптимизация пути
Зрелость	Держать качество и согласованность	Дизайн-система, аудиты, доступность
Замена	Перевести людей на новое решение без потерь	Миграция, переходные состояния

Многие ошибки идут от того, что инструменты одной стадии применяют на другой. Например, на стадии поиска полировать визуальный язык рано, а на стадии зрелости поздно выяснять, кто пользователь.

Работа без полных данных

Данных почти никогда не хватает. Профессиональный ответ на это:

- **Явно записать допущения и пометить их как допущения.** Почему: иначе через месяц догадка начнёт выглядеть как факт, и на ней построят новые решения.
- **Давать диапазон вместо точной цифры, если точной нет.** Почему: точная цифра без основания создаёт ложную уверенность, а диапазон честно показывает неопределённость.
- **Назвать, как и когда допущение проверить.** Почему: без плана проверки допущение так и останется непроверенным.
- **Не утверждать неизмеренное.** Почему: одно опровергнутое утверждение подрывает доверие ко всем остальным.

> **ИЗ ПРАКТИКИ**
>
> Цель удержания 81 % в кабинете участника сначала поставили без ориентира. Модель эффекта показала, что эта цель выше верхнего сценария. Решение: новую цель ставить от уровня, измеренного после запуска. Ошибка оценки названа ошибкой и исправлена открыто.

Ограничения как материал

Ограничения не мешают дизайну. Из них растут решения.

> **ИЗ ПРАКТИКИ**
>
> В станции биобанка лаборант работает в перчатках, сканер ведёт себя как клавиатура, ноутбук 13 дюймов, а в требованиях запрещены модальные окна и системные `alert` . Окно, перехватившее фокус, проглатывает коды сканера. Отсюда станция без всплывающих окон. Вывод автора: «Список запретов экономит итерации».

Полезная привычка: в начале проекта вести отдельный список ограничений с источником каждого. Почему: когда решение оспаривают, ссылка на ограничение и его источник заканчивает спор быстрее любых доводов о вкусе.

Чем измеряется работа дизайнера

Работу продуктового дизайнера оценивают по результату для продукта:

- **Решена ли задача человека** (успешность сценария, время, ошибки). Почему: ради этого продукт и существует.
- **Изменилась ли бизнес-метрика** (удержание, конверсия, нагрузка на сотрудников). Почему: решение, которое ничего не меняет для бизнеса, не получит развития.
- **Сколько стоило решение команде** (сроки, число переделок). Почему: дорогое решение отнимает ресурс у других задач.
- **Насколько решение устойчиво** (держится ли при росте, новых ролях, новых исследованиях). Почему: продукт меняется, и решение, которое ломается от

первого изменения, придётся переделывать.

Количество экранов и красота макета в этот список не входят. Почему: это средства, а оценивают результат.

Главное в главе

- Продуктовый дизайн отвечает за решение целиком, от проблемы до измеренного эффекта.
- Решение должно быть ценным, выгодным и выполнимым одновременно.
- Ограничения записывают с источником: из них растут решения.
- Допущения называют допущениями, неизмеренное не утверждают.
- Своё, общее и чужое в работе записывают раздельно.

Вопросы для самопроверки

1. Чем результат продуктового дизайнера отличается от результата UI-дизайнера?
2. Приведите пример решения, которое ценно для человека, но не выполнимо. Что с ним делать?
3. Какие ограничения вашего текущего проекта вы можете назвать с источником?
4. Как бы вы описали цель, для которой у вас нет базового уровня?

Упражнение

Возьмите любой сервис, которым пользуетесь. Запишите в три колонки: что он даёт человеку, что он даёт бизнесу, какие ограничения он явно учитывает. Найдите одно место, где одна из сторон проседает, и предложите, как это проверить.

2. Исследования

Исследование отвечает на вопрос, который команда не может решить спором. Хорошее исследование начинается с решения, которое от него зависит, и заканчивается изменением в продукте.

Зачем исследовать

Исследования снижают риск. Каждое отвечает на один из четырёх вопросов:

- **Есть ли проблема?** Что люди делают сейчас и что им мешает.
- **Правильно ли мы её поняли?** Как устроен процесс, кто в нём участвует, где разрывы.
- **Работает ли решение?** Понимают ли люди интерфейс, проходят ли путь.
- **Дало ли решение эффект?** Изменились ли поведение и метрики.

Под каждый вопрос свои методы. Ошибка новичка в том, чтобы отвечать юзабилити-тестом на вопрос «есть ли проблема». Почему это ошибка: тест проверяет готовое решение и ничего не говорит о том, нужно ли оно вообще.

Карта методов

Методы удобно раскладывать по двум осям: что люди говорят или что делают, и сколько людей в выборке.

Метод	Что даёт	Когда	Объём
Глубинное интервью	Мотивы, контекст, язык пользователя	Поиск, постановка	5–12 человек на сегмент
Контекстное наблюдение	Как работа идёт на самом деле, обходные пути	Сложные рабочие процессы	3–8 сессий
Дневниковое исследование	Поведение во времени	Длинные процессы, редкие события	10–20 человек, недели

Метод	Что даёт	Когда	Объём
Юзабилити-тест	Где человек спотыкается в конкретном интерфейсе	Проверка решения	5–8 человек на раунд
Опрос	Распространённость того, что уже нашли	После качественных методов	Сотни ответов
Анализ данных и логов	Что делают все пользователи	Рост, оптимизация	Вся аудитория
A/B-тест	Причинный эффект изменения	Когда трафика хватает	Тысячи событий
Анализ обращений в поддержку	Частые проблемы с формулировками людей	Всегда	Все обращения
Анализ ошибок и выгрузок	Где ломается процесс	Операционные продукты	Все записи

Качественные методы отвечают на «почему», количественные на «сколько». Обычно сначала идут качественные: они дают гипотезы, которые потом проверяют числами. Почему в таком порядке: опрос может измерить только то, о чём догадались спросить, а догадки дают интервью и наблюдение.

ИЗ ПРАКТИКИ

Станция биобанка началась с выгрузки ошибок импорта: 39 290 отклонённых строк по 747 штативам. Три четверти отказов пришлись на одну причину: «Аликвота с таким номером уже существует». Такой объём опечатками не объяснить, ошибка была в самом процессе. Что дала выгрузка: она показывает работу всех лаборантов сразу и отвечает на вопрос «сколько». Из выгрузки выросли требования: дубликат должен находиться в момент сканирования, а статус в системе не должен обгонять то, что физически лежит на столе.

Интервью

Интервью хорошо для того, чтобы понять опыт человека. Плохо для того, чтобы узнать, чего он захочет в будущем.

Правила:

1. **Спрашивать о прошлом.** «Расскажите, как вы в последний раз записывались к врачу» вместо «Как бы вы хотели записываться». Почему: о прошлом человек рассказывает факты, о будущем фантазирует, и фантазии редко совпадают с поведением.
2. **Не подсказывать ответ.** «Что было дальше?» вместо «Наверное, это было неудобно?». Почему: люди склонны соглашаться с собеседником, и наводящий вопрос получает тот ответ, который в нём заложен.
3. **Копать на конкретике.** Если человек говорит «обычно», просить случай: «Когда это было последний раз?». Почему: «обычно» это обобщение, в котором теряются детали и исключения.
4. **Молчать дольше, чем удобно.** Почему: человек заполняет паузу и часто говорит то, что не собирался.
5. **Записывать дословно.** Почему: цитата точнее пересказа, а пересказ незаметно подменяет слова человека словами исследователя.

Структура сессии на 45–60 минут: знакомство и разрешение на запись, контекст человека, история последнего случая, проблемные места, завершение с вопросом «что ещё важно, о чём я не спросил».

Юзабилити-тест

Тест проверяет, может ли человек выполнить задачу в интерфейсе. Для поиска основных проблем обычно хватает пяти участников на раунд. Это эмпирическое правило из работ Якоба Нильсена: пять человек находят большую часть серьёзных проблем одного интерфейса, дальше выгоднее исправить найденное и провести новый раунд.

Как устроить тест:

- **Задачи языком цели:** «Запишите ребёнка на исследование» вместо «Нажмите кнопку „Подать заявку“». Почему: название кнопки в задании подсказывает путь, и тест перестаёт проверять, найдёт ли его человек сам.

- **Просить думать вслух.** Почему: так видно, что человек ищет и как понимает экран. Одни клики этого не покажут.
- **Не помогать, пока человек совсем не застрял.** Почему: в жизни рядом никого не будет.
- **Фиксировать:** справился или нет, сколько времени, где ошибся, что сказал. Почему: без записи впечатления от пяти сессий смешиваются.
- **Оценивать серьёзность каждой проблемы.** Почему: исправить всё сразу нельзя, и серьёзность задаёт порядок.

Серьёзность удобно считать по четырём уровням:

Уровень	Что значит
4, критичная	Человек не может выполнить задачу
3, серьёзная	Выполняет с большим трудом или с ошибкой
2, заметная	Тормозит, но справляется
1, косметическая	Не мешает, но выглядит неаккуратно

Прогон сценариев и тест с людьми

Это разные проверки, и одна не заменяет другую. Прогон сценариев проходит все пути и состояния прототипа руками или скриптом. Он ловит сбои на стыке состояний. Тест показывает, понимает ли человек, что от него ждут.

> **ИЗ ПРАКТИКИ**
>
> Прогон 22 сценариев кабинета участника на двух размерах экрана нашёл 29 расхождений: 26 исправлено, 3 оказались ошибкой самой проверки. Среди критичных: удаление ребёнка стирало все его заявки и результаты с одного клика; выход из аккаунта стирал черновик анкеты, хотя экран обещал автосохранение; возрастной отсев срабатывал после согласия и проверки врачом. Тест с людьми нашёл бы такое очень нескоро. Зато прогон не скажет, понятен ли текст.

Анализ рынка и конкурентов

Конкурентный анализ нужен, чтобы понять ожидания людей и найти незанятое место. Копировать чужие решения он не помогает.

Полезная форма таблицы:

Сервис	Что умеет	Чего нет	Что берём себе

Колонка «Что берём себе» заставляет делать вывод, а колонка «Чего нет» показывает нишу. У каждого факта должна быть ссылка на источник. Почему: сервисы меняются, и без ссылки через полгода нельзя проверить, верен ли факт.

> **ИЗ ПРАКТИКИ**
>
> Для кабинета участника разобраны три группы сервисов: кабинеты лабораторий, государственные сервисы и платформы клинических исследований. Вывод: семью и процесс исследования в одном аккаунте не совмещает никто. Поэтому семейная модель оказалась самой тяжёлой частью проекта: её не у кого было списать.

Ещё одна практика: разбирать не только прямых конкурентов, но и системы, с которыми продукт будет интегрироваться.

> **ИЗ ПРАКТИКИ**
>
> Разбор 11 лабораторных систем дал выводы для интерфейса. У части систем нет программного интерфейса, значит, стадии «в обработке» неоткуда взяться: участник увидит «сдал» и «готово» и несколько дней пустоты между ними. Половина систем работает под Windows, поэтому экран проверки анкеты у врача должен быть самодостаточным.

Анализ результатов

Сырые заметки превращаются в находки за несколько шагов.

1. **Разложить наблюдения.** Одна мысль на карточку, с источником. Почему: так карточки можно переставлять, а источник позволит вернуться к записи.

2. **Сгруппировать.** Аффинити-карта: похожие карточки рядом, у группы название. Почему: повторяющееся у разных людей видно только рядом.
3. **Сформулировать находку.** Что происходит, у кого, почему, с примерами. Почему: группа карточек сама по себе ничего не объясняет.
4. **Связать с решением.** Что из этого следует для продукта. Почему: находка без следствия остаётся в отчёте и ничего не меняет.

Находка отличается от наблюдения тем, что объясняет. «Три участника не нашли кнопку» это наблюдение. «Люди ищут следующее действие внизу формы, а кнопка стоит в шапке» уже находка.

Сценарии работ (JTBD)

Подход Jobs to be Done описывает задачу, ради которой человек «нанимает» продукт. Сам человек здесь на втором плане. Формула: «Когда [ситуация], я хочу [мотив], чтобы [результат]».

Пример: «Когда я раз в полгода захожу в кабинет, я хочу сразу увидеть, что у меня открыто, чтобы не пропустить запись на визит».

Такая формулировка полезнее персоны, потому что прямо подсказывает, что должно быть на первом экране.

Персоны

Персона описывает типичного пользователя. Она полезна, только если основана на данных и влияет на решения. Признаки бесполезной персоны: придуманное имя и хобби, нет ни одного факта из исследования, никто не открывал её после презентации.

Рабочая персона короткая: роль, цель, контекст использования, главные трудности, что для неё значит успех. Если персона смоделирована без людей, это нужно писать прямо.

> **ИЗ ПРАКТИКИ**
>
> Персоны кабинета врача-модератора смоделированы без людей: основная роль врач-модератор, вторичные координатор и руководитель. В симуляции не подтвердились две модели проекта: «сорок заявок подряд за смену» и «координатору нужны медицинские разделы». Обе записаны как гипотезы для первых интервью.

Честность исследования

- **Указывать метод, объём и кто проводил.** Почему: читатель должен оценить, насколько выводу можно доверять.
- **Отделять то, что проверено на людях, от того, что проверено прогоном или экспертно.** Почему: эти проверки ловят разное, и смешение создаёт ложную уверенность.
- **Не выдавать симуляцию за интервью.** Почему: симуляция повторяет допущения того, кто её строил, и ничего нового о людях не говорит.
- **Проверять саму проверку:** если сверка годами показывает ноль, сломайте что-нибудь нарочно и убедитесь, что она это поймает. Почему: ноль от сломанной проверки выглядит так же, как ноль от исправного продукта.

> **ИЗ ПРАКТИКИ**
>
> Сверка дизайн-системы три недели сравнивала её со старым файлом и показывала ноль расхождений. После этого сверку стали проверять нарочной поломкой: нулю верят только тогда, когда пойманная поломка показала, что проверка работает.

Главное в главе

- Исследование начинается с решения, которое от него зависит.
- Качественные методы дают «почему», количественные «сколько».
- В интервью спрашивают о прошлом, в тесте дают задачу языком цели.
- Прогон сценариев и тест с людьми ловят разное.
- В анализе рынка нужны колонки «Чего нет» и «Что берём себе».

- Метод, объём и кто проводил пишут всегда.

Вопросы для самопроверки

1. Каким методом вы ответите на вопрос «почему люди бросают анкету на третьем разделе»? А на вопрос «сколько людей бросают»?
2. Перепишите вопрос «Вам было бы удобно получать напоминания?» так, чтобы он не подсказывал ответ.
3. Чем находка отличается от наблюдения?
4. Почему симулированные интервью нельзя подавать как исследование с людьми?

Упражнение

Составьте план юзабилити-теста для любого сценария записи: три задачи языком цели, критерии успеха для каждой, шкала серьёзности, список того, что будете фиксировать.

3. Постановка задачи

Плохо поставленная задача стоит дороже плохого решения: её можно идеально решить и не получить пользы. Постановка отвечает на вопросы «что за проблема», «для кого», «как поймём, что решили» и «что не делаем».

Проблема и решение

Заказчик почти всегда приходит с решением: «нужен личный кабинет», «добавьте фильтр». Первая работа дизайнера в том, чтобы найти за решением проблему.

Вопросы, которые помогают:

- **Что происходит сейчас и чем это плохо?** Почему: ответ показывает проблему, которую решение должно убрать.
- **Кто страдает и сколько это стоит?** Почему: без цены проблемы нельзя решить, сколько вкладывать в решение.
- **Что изменится, если ничего не делать?** Почему: иногда ответ «ничего страшного», и задачу можно отложить.
- **Как вы поймёте, что задача решена?** Почему: этот ответ становится критерием успеха и защищает от бесконечных доработок.

> **ИЗ ПРАКТИКИ**
>
> Задачу кабинета участника можно было свести к переносу анкеты из Яндекс Форм в кабинет. Её сформулировали через последствие для центра: данные участника, который ушёл на середине, почти бесполезны. У задачи три части, и у каждой своя метрика.

Бриф

Бриф фиксирует договорённость о задаче до начала работы. Короткий бриф полезнее длинного: его читают.

Состав брифа:

Раздел	Что внутри
Контекст	Продукт, что было до, почему сейчас
Проблема	Что не так, у кого, с фактами
Пользователи	Роли и их задачи
Цель и метрики	Что изменится и как измерим
Ограничения	Законы, протоколы, техника, сроки
Что не делаем	Границы задачи
Открытые вопросы	Что решаем по ходу и кто решает

Хорошая практика: перед составлением брифа собрать документы проекта и спрашивать только то, чего в них нет. Почему: время заказчика ограничено, и на короткий список вопросов отвечают быстрее и точнее. Так бриф не превращается в анкету на сто вопросов.

PRD

PRD (product requirements document) описывает, что должно быть построено. Он подробнее брифа и пишется после исследования.

Обычные разделы PRD:

1. Проблема и цель.
2. Пользователи и сценарии.
3. Требования: функциональные и нефункциональные.
4. Что не входит в версию.
5. Метрики успеха.
6. Риски и зависимости.
7. Открытые вопросы.

Требование пишут так, чтобы его можно было проверить. Почему: непроверяемое требование каждый понимает по-своему, и при приёмке спорить не о чем. «Удобная

запись» не проверить. «Человек видит дату следующего визита на первом экране без прокрутки» проверить можно.

Гипотезы

Гипотеза связывает изменение с результатом и способом проверки:

> Если [изменение], то [метрика] изменится [на сколько], потому что [причина]. Проверим [как].

Пример: «Если показывать нагрузку исследования (длительность и число визитов) на карточке каталога, то доля брошенных заявок снизится, потому что человек оценит нагрузку до входа в поток. Проверим сравнением доли брошенных заявок до и после».

Гипотеза без способа проверки остаётся мнением. Почему: если заранее не решить, как проверять, после запуска любой результат можно объяснить в свою пользу.

Критерии успеха и метрики

Метрики подробно разобраны в главе 18. На этапе постановки важно:

- **Назвать главную метрику,** ради которой делается работа. Почему: так команда понимает, что важнее при конфликте решений.
- **Назвать контрольные метрики,** которые не должны ухудшиться. Почему: улучшение одного часто ломает другое, и это нужно заметить.
- **Договориться о базовом уровне** или о том, как его получить. Почему: без точки отсчёта изменение не измерить.

> **ИЗ ПРАКТИКИ**
>
> Для кабинета участника главная находка закреплена контрольной метрикой: «потеря данных при переключении равна нулю». Заявки хранятся по паре «участник и исследование», и метрика проверяет, что переключение между участниками семьи ничего не теряет.

Объём работы (scope)

Scope отвечает на вопрос «что входит в эту версию». Раздел «Что не делаем» здесь так же важен, как список того, что делаем: он защищает команду от расползания задачи.

Признаки того, что объём пора резать:

- **В первую версию входит всё, что придумали.** Почему это плохо: обратная связь от людей приходит только после запуска, и чем позже запуск, тем позже она придёт.
- **Нет функции, без которой продукт можно запустить.** Почему это плохо: значит, приоритеты не расставлены.
- **Сроки растут, а обязательных функций не становится меньше.** Почему это плохо: объём растёт быстрее, чем команда успевает делать.

Приоритизация

Методы приоритизации помогают команде договориться. Истину они не вычисляют. Выбирайте метод, который понятен команде.

MoSCoW

Функции делят на четыре группы:

- **Must**: без этого запуск невозможен;
- **Should**: важно, но можно отложить ненадолго;
- **Could**: хорошо бы, если останется время;
- **Won't**: в эту версию точно не входит.

> **ИЗ ПРАКТИКИ**
>
> Первая версия кабинета участника разобрана по MoSCoW: 57 функций, из них 28 обязательных, 14 желательных, 10 возможных и 5 отложенных. Серьёзные проблемы UX-аудита тоже отнесены к обязательным: запуск с ними потерял бы людей первого цикла. 20 из 28 обязательных уже были в прототипе.

RICE

Оценка по четырём параметрам:

- **Reach**: сколько людей затронет за период;
- **Impact**: насколько сильно (обычно шкала 0,25 / 0,5 / 1 / 2 / 3);
- **Confidence**: насколько уверены в оценках (в процентах);
- **Effort**: сколько работы (человеко-недели).

Итог: Reach × Impact × Confidence / Effort. Метод хорош тем, что заставляет назвать уверенность: без неё смелая идея и проверенная идея весят одинаково. Плох тем, что числа создают видимость точности.

Модель Кано

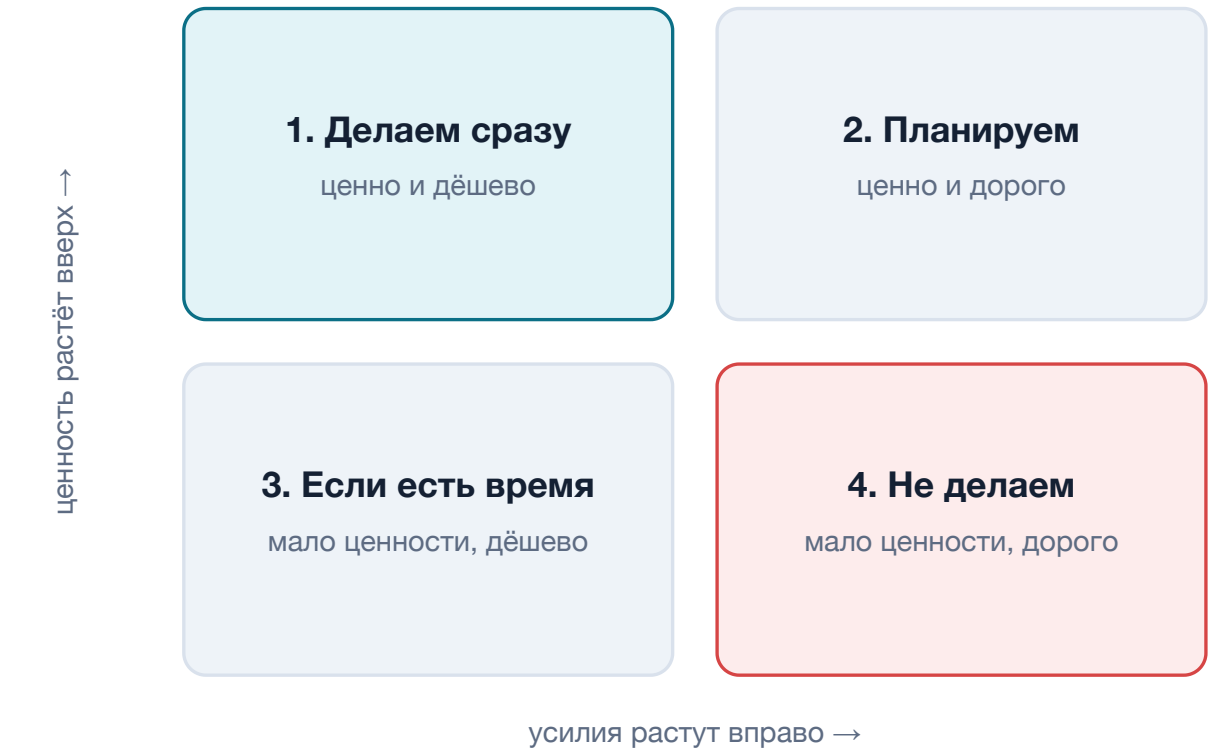
Делит функции по тому, как они влияют на удовлетворённость:

Тип	Пример	Что будет без него
Базовые	Сохранение черновика анкеты	Сильное недовольство
Линейные	Скорость проверки заявки	Чем лучше, тем довольнее
Восхищающие	Динамика результатов за два года	Никто не заметит отсутствия
Безразличные	Выбор цвета интерфейса	Никак
Обратные	Навязчивые уведомления	Раздражают, если есть

Модель напоминает: базовые функции не приносят похвалы, но их отсутствие губит продукт. Почему это важно для приоритетов: базовые функции легко отложить в пользу ярких, а без них яркие не спасут.

Матрица ценности и усилий

Самый простой способ: две оси, ценность и усилия. Сначала делаем ценное и дешёвое, потом ценное и дорогое. Почему в таком порядке: дешёвое ценное быстро приносит пользу и даёт время на дорогое. Дешёвое и малоценное делаем, если есть время. Дорогое и малоценное не делаем.



Матрица ценности и усилий

Ограничения

Ограничения записывают в постановку с источником. Некоторые ограничения сразу дают решения.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника единый номер пациента закрыл идею перевода ребёнка в собственный кабинет в 18 лет: второй номер ломает автоматическую рассылку отчётов. Лабораторная система ещё не выбрана, поэтому ручные статусы спроектированы как штатный режим. Протокол задаёт число визитов и интервал, поэтому в интерфейсе это параметры протокола.

Открытые вопросы

Некоторые вопросы дизайнер не может и не должен решать один. Их стоит собрать списком, назвать, кто решает, и обсудить до разработки. Почему: решение, принятое после разработки, обычно означает переделку.

ИЗ ПРАКТИКИ

Для кабинета участника вынесены вопросы, на которые автор не отвечала одна: совершеннолетие ребёнка, доступ родителя, история при удалении профиля, общий научный отчёт семьи. Формулировка из кейса: «Команде дешевле обсудить их до разработки, чем переделывать после».

Купить или сделать

Иногда правильная постановка заканчивается выводом, что делать ничего не надо: готовое решение уже есть. Сравнение «купить или сделать» стоит проводить явно.

ИЗ ПРАКТИКИ

Для станции биобанка рассматривали покупку системы с модулем биобанка: готовая модель хранения, поддержка вендора. Не подошло. Момент, когда лаборант стоит у стола и связывает два штрихкода с ячейкой, не поддерживает ни одна система, внедрение стоит миллионы, а доработки почти везде делает только вендор за отдельные деньги. Выбрана станция как отдельный инструмент для одной операции, с расчётом, что данные потом уйдут в основную систему. Вывод автора: «Без этого сравнения проект пришлось бы защищать ощущениями».

Главное в главе

- За решением заказчика ищут проблему.
- Бриф фиксирует договорённость, PRD описывает, что строим.
- Требование пишут так, чтобы его можно было проверить. Почему: непроверяемое требование каждый понимает по-своему, и при приёмке спорить не о чем.
- У гипотезы есть изменение, метрика, причина и способ проверки.
- Приоритизация помогает договориться: MoSCoW, RICE, Кано, ценность и усилия.
- Раздел «Что не делаем» защищает объём работы.
- Открытые вопросы собирают списком и решают до разработки.

Вопросы для самопроверки

1. Перепишите требование «удобный поиск» так, чтобы его можно было проверить.
2. Чем гипотеза отличается от мнения?
3. Какую функцию вашего продукта вы бы отнесли к базовым по Кано? Как вы это проверите?
4. Когда стоит рассмотреть вариант «купить»?

Упражнение

Напишите бриф на одну страницу для вымышленной задачи «нужен экран истории заказов». Обязательно заполните разделы «Что не делаем» и «Открытые вопросы».

4. Информационная архитектура и сценарии

До экранов нужно понять, из каких частей состоит продукт, как человек между ними движется и в каких состояниях может оказаться. Ошибки этого уровня потом не исправить вёрсткой.

Информационная архитектура

Информационная архитектура отвечает на вопросы: какие сущности есть в продукте, как они связаны, как называются и где лежат.

Порядок работы:

1. **Выписать сущности.** Пользователь, участник, заявка, визит, образец, результат, уведомление. Почему: сущности это то, о чём продукт хранит данные, и экран без понятной сущности не на что опереть.
2. **Описать связи.** У одного пользователя несколько участников, у участника несколько заявок, у заявки несколько визитов. Почему: ошибки в связях («один или несколько?») проявляются позже всего и стоят дороже всего.
3. **Назвать.** Одно слово на одну сущность во всём продукте. Почему: разные слова для одной вещи заставляют человека думать, что это разные вещи.
4. **Разложить по разделам.** Что видно на первом экране, что внутри, что в настройках. Почему: частое и важное должно быть ближе, редкое и опасное дальше.

Модель данных раньше экранов

Самая дорогая ошибка архитектуры это неправильная связь между сущностями. Её не видно на одном экране, она проявляется на стыке сценариев.

> **ИЗ ПРАКТИКИ**
>
> Главная находка кабинета участника касалась модели данных. Все системные ошибки проекта оказались одной ошибкой в разных формах: логику писали для одного участника, а применяли к нескольким. Возрастная блокировка прилипала к следующему участнику, своя заявка называлась «другой», номер был у всех один. После находки первой в плане стала модель: заявки хранятся по паре «участник и исследование». Вывод, который автор записала как то, что сделала бы иначе: «Проверять модель данных в первую неделю».

Одно слово на одну вещь

Если одна сущность в разных местах называется по-разному, человек решает, что это разные вещи. Перед проектированием полезно завести словарь продукта: термин, определение, где используется, какие слова не используем.

Пользовательские сценарии

Сценарий (user flow) описывает путь человека к цели: с чего начинает, какие шаги проходит, где принимает решения, чем заканчивает.

Хороший сценарий:

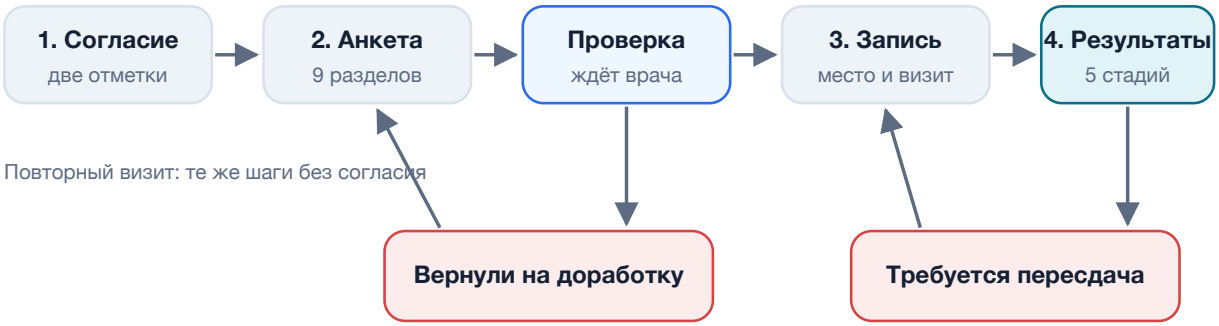
- **Начинается с цели человека.** Почему: экраны служат цели, и путь, который начинается с экрана, легко теряет её из виду.
- **Показывает развилки и их условия.** Почему: именно на развилках чаще всего забывают ветку.
- **Включает возвраты: ошибки, отказ, повтор.** Почему: счастливый путь проектируют все, а люди ломаются на возвратах.
- **Заканчивается результатом, понятным человеку.** Почему: если конец пути неясен, человек не знает, сделал ли он всё.

Карта пути (CJM)

Карта пути (customer journey map) шире сценария. Она показывает весь опыт по этапам: что человек делает, что думает и чувствует, через какие каналы проходит, где болит. Её используют, чтобы найти места, где продукт теряет людей.

ИЗ ПРАКТИКИ

Путь заявки в кабинете участника состоит из четырёх шагов: согласие, анкета, запись на визит, результаты. У повторного визита три шага, без согласия. Слева всегда виден список шагов, и у шага цвет состояния: пройден, ждёт врача, вернули на доработку, готово. Человек в любой момент видит, где он и что дальше.



Путь заявки в кабинете участника с ожиданием врача и двумя возвратами

Состояния

Экран почти никогда не существует в одном состоянии. Для каждого экрана и компонента проверяют:

Состояние	Вопрос
Пустое	Что видит человек, когда данных ещё нет?
Загрузка	Что видно, пока данные идут?
Частичное	Что, если заполнено не всё?
Полное	Как выглядит экран с большим объёмом данных?
Ошибка	Что случилось и как исправить?
Успех	Что произошло и что дальше?

Состояние	Вопрос
Нет доступа	Почему недоступно и что сделать?
Ожидание чужого действия	Чего ждём, от кого, сколько?

Последнее состояние часто забывают, хотя в сервисах с проверкой человеком оно самое длинное.

> **ИЗ ПРАКТИКИ**
>
> Экран ожидания врача в кабинете участника пишет, что заявка у врача, что проверка обычно занимает до одного рабочего дня и что придёт уведомление. В кейсе это названо самой дешёвой функцией в проекте и, кажется, самой полезной.

Статусы

Статус отвечает на вопрос «что сейчас происходит». Правила:

- **Статус называет состояние.** «На проверке у врача», «Ожидаем визита». Почему: человек хочет знать, что происходит, а слово «обрабатывается» этого не говорит.
- **Статус не обгоняет реальность.** Нельзя показывать «готово», пока результат не пришёл. Почему: обещание, которое не сбылось, разрушает доверие ко всем статусам.
- **У статуса одна точка правды.** Уведомление и карточка меняются одним действием. Почему: если уведомление и карточка говорят разное, человек не знает, чему верить.
- **Разные типы статусов не подменяют друг друга.** Административный статус заявки и клинический статус результата живут отдельно. Почему: «запись подтверждена» и «образец хорош» отвечают на разные вопросы, и смешение даёт ложную информацию.

> **ИЗ ПРАКТИКИ**
>
> При прогоне сценариев визит получал статус «образец хорош» в момент, когда врач подтверждал запись. Тост сообщал «требуется пересдача», а карточка ниже показывала прежний статус. Отсюда два правила: административный статус не подменяет клинический, и у статуса одна точка правды.

Даты, которых никто не назначал

Прогнозная дата читается как обещание. Если дату никто не назначал, её лучше не показывать.

> **ИЗ ПРАКТИКИ**
>
> Сначала в плане визитов были прогнозные даты всех будущих визитов. Выбрано иначе: будущие визиты без дат с пометкой «Ещё впереди», ориентировочная дата только у следующего визита с подписью «ориентировочно». Побочно ушла хрупкость: арифметика дат на неверных данных роняла приложение.

Навигация

Навигация отвечает на три вопроса: где я, куда могу пойти, как вернуться.

Основные типы:

- **Глобальная**: шапка, боковое меню, нижний таб-бар на телефоне. Нужна, чтобы попасть в любой раздел откуда угодно.
- **Локальная**: вкладки, сегменты, оглавление внутри раздела. Нужна, чтобы переключаться внутри одной темы.
- **Контекстная**: ссылки и кнопки внутри содержания. Нужна, чтобы перейти к связанному там, где о нём зашла речь.
- **Пошаговая**: мастер с шагами. Нужна, когда задачу выполняют по порядку.

Правило: человек, который заходит редко, должен на первом экране увидеть, что у него открыто и что требует действия. Почему: он не помнит устройство продукта и пришёл с одним вопросом: что мне делать.

> **ИЗ ПРАКТИКИ**
>
> Для семьи сначала был переключатель «Все / Себе / Ребёнок». Выбран другой путь: все заявки семьи на одном экране, по людям. Первый вопрос человека, который заходит раз в полгода, «что у меня сейчас открыто». Кроме того, переключатель заявок поверх переключателя участников давал два уровня скрытого состояния.

Очередь и реестр

В рабочих инструментах полезно разделять очередь и реестр. Очередь содержит только то, что ждёт действия этого человека. Реестр хранит всё.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора очередь отвечает на вопрос «я всё разобрал?». Пока в ней лежит то, что ждёт участника, ответить нечем. Поэтому очередь собрана только из дел, всё остальное ушло в реестр. Пустая очередь значит «на сегодня всё».

Пошаговые формы и мастера

Длинную задачу делят на шаги, когда:

- шаги идут в естественном порядке, и порядок помогает думать;
- следующий шаг зависит от предыдущего, и его нельзя показать раньше;
- человеку нужно видеть прогресс, иначе длинная задача кажется бесконечной.

Правила мастера:

1. **Видно, сколько шагов и где ты.** Почему: человек решает, хватит ли у него сил сейчас, по тому, сколько осталось.
2. **Прогресс считает факт заполнения.** Раздел засчитан, когда в нём есть ответы. Позиция в мастере не важна.
3. **Черновик сохраняется сам.** С меткой времени. Почему: без метки человек не верит, что сохранилось, и боится закрыть страницу.
4. **Можно вернуться** без потери данных. Почему: человек часто вспоминает ошибку на следующем шаге.

5. **Перед отправкой есть сводка** с возможностью изменить ответ. Почему: проверить всё один раз в конце легче, чем на каждом шаге.
6. **Вопросы раскрываются по ответу.** Почему: вопрос не по адресу тратит время и заставляет думать, как на него ответить.

> **ИЗ ПРАКТИКИ**
>
> Анкета исследования сахарного диабета первого типа в кабинете участника состоит из девяти разделов по смыслу, от 65 до 100 полей по возрасту. Вложенные вопросы раскрываются по ответу: «была госпитализация» открывает дату, причину, стационар. У взрослых есть вопросы про алкоголь и табак, у детей перинатальная история. Прогресс считает разделы, заполненные по факту: ответ «нет» засчитывается, пропуск не засчитывается.

Когда проверять поля

Проверять каждое поле сразу или всё при отправке, решают по сценарию. Если люди листают форму, чтобы посмотреть, что их ждёт, красные поля при каждом «Далее» наказывают за просмотр.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника «Далее» ничего не проверяет, все обязательные поля проверяются один раз при отправке. Кнопка «Далее» всегда нажимается и меняет вес: второстепенная, пока есть пустые поля, главная, когда всё заполнено. Слово автора: «Красный — это ошибка в анкете, а не незаполнено». Цена решения: об ошибках человек узнаёт позже.

Развилки и крайние случаи

Крайние случаи находят вопросом «а что, если». Почему так: счастливый путь в голове у всех, а ломается продукт на исключениях, и их нужно искать специально.

- а что, если участников двое?
- а что, если человек вышел посреди анкеты?
- а что, если врач вернул анкету, а человек уже исправил другое поле?
- а что, если результат не пришёл?

- а что, если удалить ребёнка, у которого есть заявка?

> **ИЗ ПРАКТИКИ**
>
> Сверка прототипа v44 показала: ребёнок в настройках удаляется одним кликом, без подтверждения, а его заявки переходят к следующему ребёнку в списке. После удаления сына его заявка показывалась с именем дочери. Это та же ошибка модели данных: логику писали для одного участника. Крайний случай с удалением потом исправили отдельной версией прототипа.

Главное в главе

- Модель данных проверяют в первую неделю.
- Одно слово на одну сущность во всём продукте.
- У каждого экрана есть пустое, частичное, ошибочное состояние и ожидание чужого действия.
- Статус не обгоняет реальность и имеет одну точку правды.
- Даты, которых никто не назначал, не показывают.
- Очередь хранит только дела, реестр хранит всё.
- Прогресс мастера считает факт заполнения.

Вопросы для самопроверки

1. Какие сущности есть в сервисе записи к врачу? Как они связаны?
2. Назовите три состояния экрана, которые чаще всего забывают.
3. Почему прогнозная дата может навредить?
4. Чем очередь отличается от реестра?

Упражнение

Нарисуйте сценарий «родитель записывает двух детей на одно исследование». Отметьте все развилки и все состояния ожидания. Задайте пять вопросов «а что, если» и проверьте, что сценарий на них отвечает.

5. Формы и анкеты

Форма это место, где продукт получает данные от человека. Почти каждая потеря пользователя в сервисе с заявками происходит в форме: человек устал, не понял вопрос, испугался ошибки или потерял введённое. Поэтому форму проектируют так же тщательно, как главный экран.

Сначала решить, нужен ли вопрос

Каждое поле стоит человеку времени и внимания. Перед тем как добавить вопрос, его проверяют тремя вопросами:

1. **Кто использует ответ и для чего?** Если ответ никто не читает, поле лишнее. Почему: лишние поля удлиняют форму и снижают долю тех, кто её закончит.
2. **Можно ли узнать ответ без человека?** Возраст из даты рождения, индекс массы тела из роста и веса, город из адреса. Почему: человек не должен считать то, что посчитает система, и не ошибётся в расчёте.
3. **Нужен ли ответ сейчас?** Часть данных можно спросить позже, когда они понадобятся. Почему: короткий первый шаг чаще доводят до конца, а доверие к сервису растёт по ходу работы.

> **ИЗ ПРАКТИКИ**
>
> В анкете кабинета участника индекс массы тела считается сам. Документы для первого шага тоже пересмотрели: вместо загрузки паспорта, полиса и направления участник прикладывает выписку из истории болезни. Причина замены в материалах не записана. Общий принцип за ней такой: просить документ, по которому принимают решение, и не просить остальные.

Порядок и группировка

- **Вопросы группируют по смыслу.** «О здоровье», «О семье», «Образ жизни». Почему: человек отвечает на похожие вопросы быстрее, когда держит в голове одну тему.
- **От простого к сложному.** Сначала то, что человек знает наизусть, потом то, что нужно вспомнить или найти. Почему: начатая форма удерживает лучше, чем

пустая, и к сложным вопросам человек подходит уже вовлечённым.

- **Вопросы, которые относятся не ко всем, раскрываются по ответу.** Почему: человек не читает то, что к нему не относится, и форма выглядит короче, чем есть на самом деле.
- **Ветки по признаку человека.** Разные вопросы детям и взрослым. Почему: вопрос не по адресу раздражает и заставляет думать, как на него ответить.

> **ИЗ ПРАКТИКИ**
> Анкета исследования сахарного диабета первого типа в кабинете участника разбита на девять разделов по смыслу, полей от 65 до 100 в зависимости от возраста. «Была госпитализация» открывает дату, причину и стационар. У взрослых есть вопросы про алкоголь и табак, у детей перинатальная история. До этого в старой форме сто с лишним полей шли на одной странице, а перед первым вопросом стояли четыре абзаца о том, к какой группе себя отнести.

Один шаг или мастер

Форма	Когда выбирать	Почему
Одна страница	До 10–15 полей, одна тема	Видно всё сразу, легко проверить перед отправкой
Мастер с шагами	Длинная форма, шаги зависят друг от друга	Прогресс виден, на каждом шаге меньше нагрузки
Разделы с навигацией	Длинная анкета, порядок не важен	Человек заполняет, что знает, и возвращается к остальному

Анкета в кабинете участника устроена разделами с навигацией внутри шага мастера. Что это даёт: человек может прерваться, вернуться через день и сразу увидеть, какие разделы ещё пусты.

Подписи, подсказки и примеры

- **Подпись над полем.** Почему: так глаз движется вниз по одной линии, а подпись видна и после ввода.

- **Пример под полем,** а не внутри. Почему: серый пример в пустом поле можно принять за уже введённое значение.
- **Обязательность одинаково везде.** Почему: если в одном месте звёздочка значит «обязательно», а в другом «важно», человек перестаёт доверять пометкам.
- **Вопрос как в разговоре.** «Сколько раз за последний год вы лежали в больнице?» вместо «Количество госпитализаций (за 12 мес.)». Почему: живую формулировку понимают с первого раза и реже отвечают неточно.

> **ИЗ ПРАКТИКИ**
>
> UX-аудит кабинета участника отнёс к серьёзным проблемам то, что пустые обязательные поля показывали серые примеры «170» и «65». Почему это серьёзно: человек видит число и решает, что поле заполнено, а при отправке получает ошибку, которую не понимает.

Тип поля под ответ

Ответ	Поле	Почему
Один вариант из 2–5	Переключатели (радиокнопки)	Все варианты видны сразу
Один из многих	Список с поиском	Длинный список без поиска приходится листать
Несколько вариантов	Чекбоксы	Видно, что можно выбрать больше одного
Да или нет	Два переключателя «Да» и «Нет»	Чекбокс не различает «нет» и «не ответил»
Дата	Поле с маской или три поля	Календарь неудобен для даты рождения 40 лет назад
Число	Поле с цифровой клавиатурой	На телефоне сразу открываются цифры

> **ИЗ ПРАКТИКИ**
>
> Анализ путей кабинета участника нашёл в анкете список из 47 родственных связей. Длинный список без поиска заставляет человека читать всё подряд, чтобы найти «тётя».

Проверка полей

Когда показывать ошибку, решают по поведению людей в этой форме.

Подход	Когда подходит	Цена
Сразу при уходе из поля	Короткие формы, формат (телефон, почта)	Ошибки появляются, пока человек ещё вводит соседние поля
При переходе к следующему шагу	Мастер с зависимыми шагами	Нельзя посмотреть следующие шаги, не заполнив текущий
Один раз при отправке	Длинные анкеты, которые люди листают	Об ошибках узнают позже

Общие правила:

- **Все ошибки шага показывают одним проходом.** Почему: если ошибки выдаются по одной, человек исправляет, жмёт снова и получает следующую. Это раздражает больше, чем список сразу.
- **Красный только для ошибки.** Незаполненное поле ошибкой не является, пока человек не попытался отправить. Почему: красный при просмотре наказывает человека за то, что он ещё не начал.
- **Сводка ошибок со ссылками на поля.** Почему: в длинной форме ошибка может быть за пределами экрана, и человек не найдёт её сам.

ИЗ ПРАКТИКИ

В кабинете участника «Далее» в анкете ничего не проверяет, обязательные поля проверяются один раз при отправке. Почему так: многие листают анкету, чтобы посмотреть, что их ждёт. Слово автора: «Красный — это ошибка в анкете, а не незаполнено». Кнопка «Далее» при этом меняет вес: второстепенная, пока есть пустые поля, главная, когда раздел заполнен. Почему: человек видит, что раздел готов, без отдельного сообщения.

ИЗ ПРАКТИКИ

Прогон сценариев нашёл, что на шаге записи ошибки выдавались порциями, а невыбранная клиника не подсвечивалась. Отсюда правило системы: все ошибки шага одним проходом.

Сохранение

- **Черновик сохраняется сам при каждом изменении.** Почему: человек может уйти в любой момент, и потеря введённого почти всегда означает, что он не вернётся.
- **Метка времени сохранения видна.** «Черновик сохранён в 14:32». Почему: без метки человек не доверяет автосохранению и боится закрыть страницу.
- **Сохранённое переживает выход из аккаунта.** Почему: обещание «сохранено» должно держаться, иначе следующему обещанию не поверят.

ИЗ ПРАКТИКИ

Прогон сценариев кабинета участника нашёл, что выход из аккаунта стирал черновик анкеты, хотя экран обещал автосохранение. Ошибку исправили. Почему это было критично: человек, который заполнял анкету частями, терял всё и узнавал об этом только при следующем входе.

ИЗ ПРАКТИКИ

На станции биобанка нет кнопки «Сохранить», данные сохраняются при уходе из поля. Довод из кейса: «На серии из восьмидесяти позиций это восемьдесят лишних нажатий».

Прогресс

- **Прогресс считает факт заполнения.** Раздел засчитан, когда в нём есть ответы. Почему: если прогресс считает позицию, человек видит «70 %», хотя пропустил половину.
- **Ответ «нет» засчитывается.** Почему: «нет» это полноценный ответ, и человек не должен чувствовать, что анкета считает его ленивым.
- **Один расчёт прогресса на весь продукт.** Почему: если в двух местах прогресс считается по-разному, человек видит противоречие и перестаёт верить обоим.

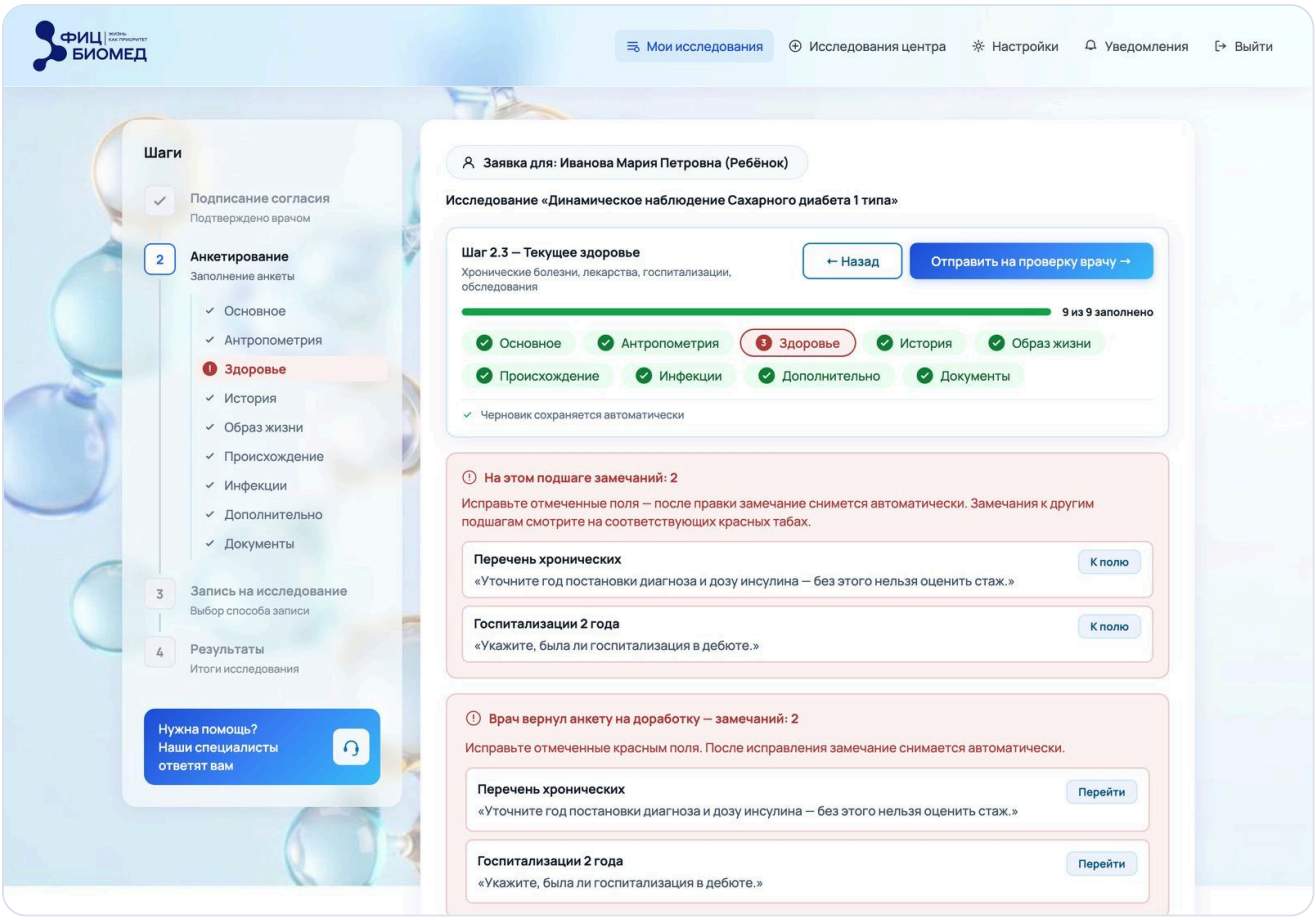
> **ИЗ ПРАКТИКИ**
>
> В кабинете участника на «Моих исследованиях» однажды стояло «шаг 0 из 3» рядом с «Текущий шаг 1 из 3» у повторного визита. Теперь подпись «шаг N из M» считается одной функцией в трёх местах.

Возврат на доработку

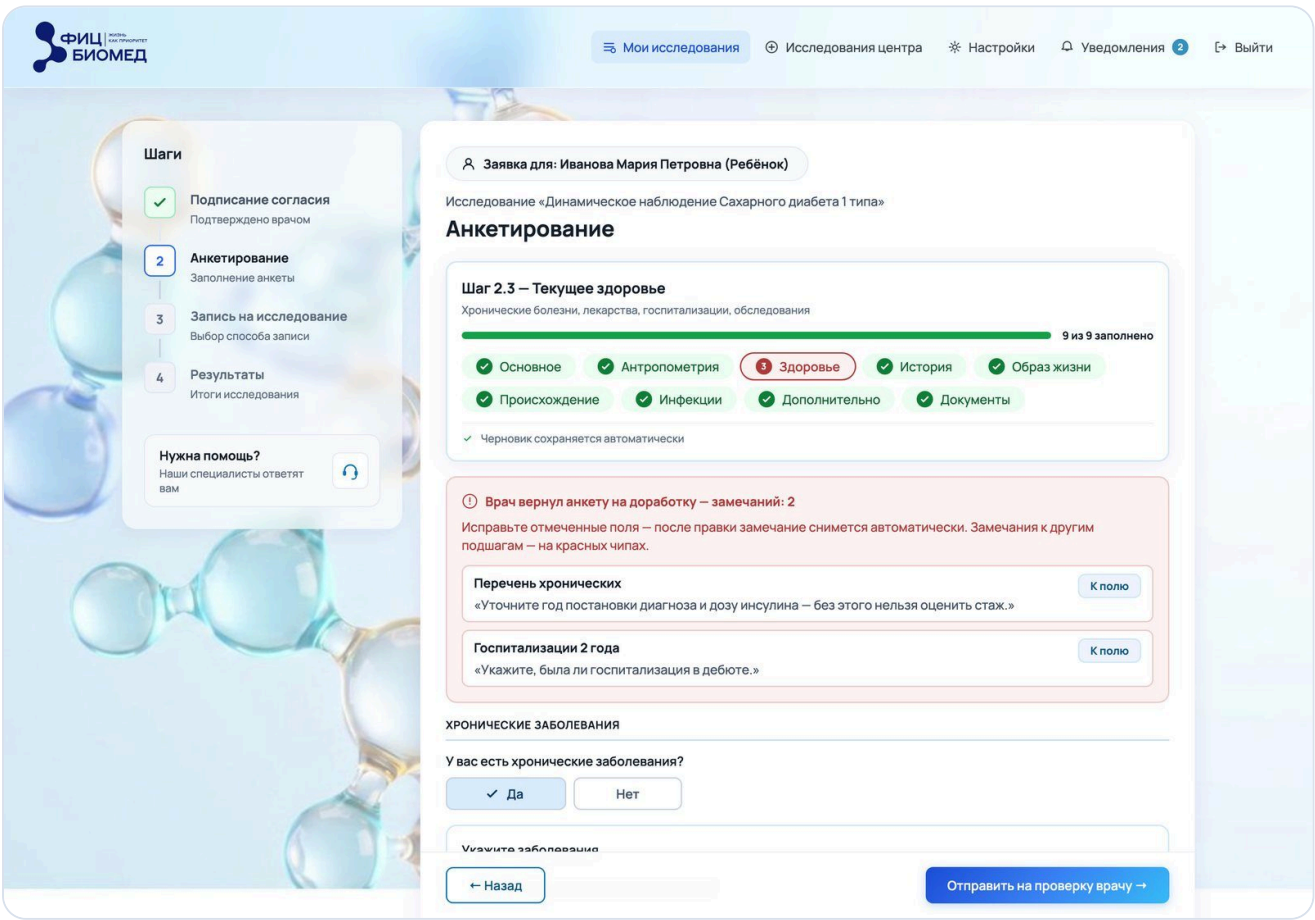
Если форму проверяет человек и возвращает с замечаниями, возврат проектируют отдельно.

- **Замечание привязано к полю.** Почему: «исправьте анкету» не говорит, что чинить.
- **Раздел с замечанием выделен и показывает их число.** Почему: человек сразу видит объём работы и не ищет замечания по всей форме.
- **Есть переход прямо к полю.** Почему: в анкете на сто полей поиск поля сам по себе отнимает силы.
- **Исправленное замечание снимается само.** Почему: человеку не нужно отдельно отмечать, что он исправил, и врач видит актуальное состояние.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника раздел с замечаниями врача краснеет и показывает их число. Кнопка «К полю» открывает раздел, прокручивает к полю, ставит фокус и подсвечивает его. Исправил, и замечание снялось само. Если в заполненном разделе есть замечание, чип раздела красный. Почему: то, что надо доделать, важнее того, что сделано.



До, версия v02. Возврат на доработку: два красных блока с одними и теми же замечаниями



После, версия v03. Один блок с замечаниями текущего подшага и счётчиком по всей анкете

> **ИЗ ПРАКТИКИ**
>
> На стороне врача-модератора сначала было одно поле для замечания на всю заявку, как у Wildberries. Выбрали замечание к полю. Довод: «направление понятно, что чинить — нет».

Сводка перед отправкой

Перед отправкой длинной формы показывают сводку ответов с возможностью изменить каждый. Почему: человек проверяет себя один раз в конце и уверен в том, что отправляет. Проверять каждый шаг отдельно утомительно.

Согласия

- **Отдельная отметка на каждое согласие.** Почему: согласие должно быть осознанным действием. Побочный эффект другой кнопки согласием не считается.
- **Ссылка на документ рядом.** Почему: человек вправе прочитать, на что соглашается, не уходя из формы.
- **Без формулы «продолжая, вы соглашаетесь».** Почему: такая формула делает согласие незаметным, а в медицине и при работе с данными это недопустимо.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника первый шаг не пускает дальше, пока не отмечены оба согласия. Формулы «продолжая, вы соглашаетесь» нет.

Форма за другого человека

Если форму заполняют за ребёнка или родственника:

- **сразу спросить, за кого.** Почему: от ответа зависят вопросы, подписи и согласия;
- **дать добавить человека прямо в форме.** Почему: уход в настройки рвёт поток, и человек теряет место, где остановился;
- **писать «ребёнок», «участник» вместо «вы» там, где речь о другом.** Почему: «Ваш рост» в анкете за ребёнка заставляет гадать, чей рост вписать.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника окно «Кого вы записываете?» позволяет добавить участника прямо при подаче заявки, без ухода в настройки. Для исследования СД1 есть отдельный выбор роли из двух крупных карточек: я или мой ребёнок. Он пропускается, если участников ещё нет.

Главное в главе

- Каждое поле должно иметь потребителя ответа.
- Вопросы группируют по смыслу, лишние раскрываются по ответу.
- Пример стоит под полем, подпись над ним.
- Время проверки полей выбирают по поведению людей.
- Все ошибки шага показывают одним проходом.
- Черновик сохраняется сам и переживает выход.
- Прогресс считает факт заполнения одним расчётом на весь продукт.
- Замечание привязано к полю и снимается само.

Вопросы для самопроверки

1. Почему для вопроса «да или нет» чекбокс хуже двух переключателей?
2. Когда проверять поле сразу, а когда при отправке?
3. Зачем показывать время сохранения черновика?
4. Почему исправленное замечание должно сниматься само?

Упражнение

Возьмите любую длинную форму (регистрация, заявка, анкета). Для каждого поля ответьте: кто использует ответ, можно ли его не спрашивать, нужен ли он сейчас. Сократите форму и опишите, что изменится для человека.

6. Интерфейс

Визуальная часть интерфейса отвечает за то, чтобы человек быстро понял, что на экране главное, что можно сделать и что сейчас происходит. Красота здесь следствие порядка.

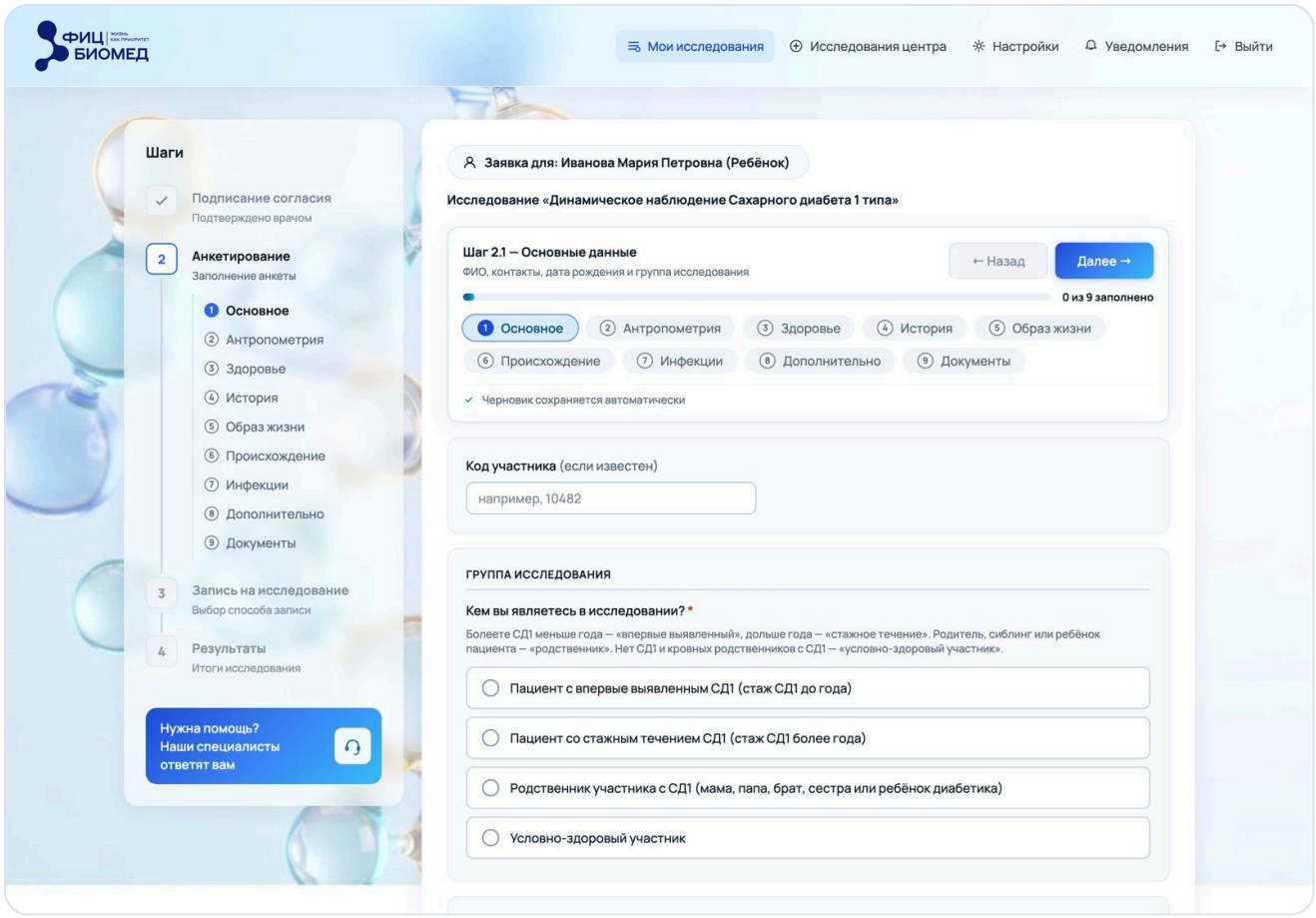
Иерархия и фокус

На каждом экране есть одна главная задача. Всё остальное по весу ей уступает. Вес элемента складывается из размера, цвета, контраста, места на экране и числа повторов.

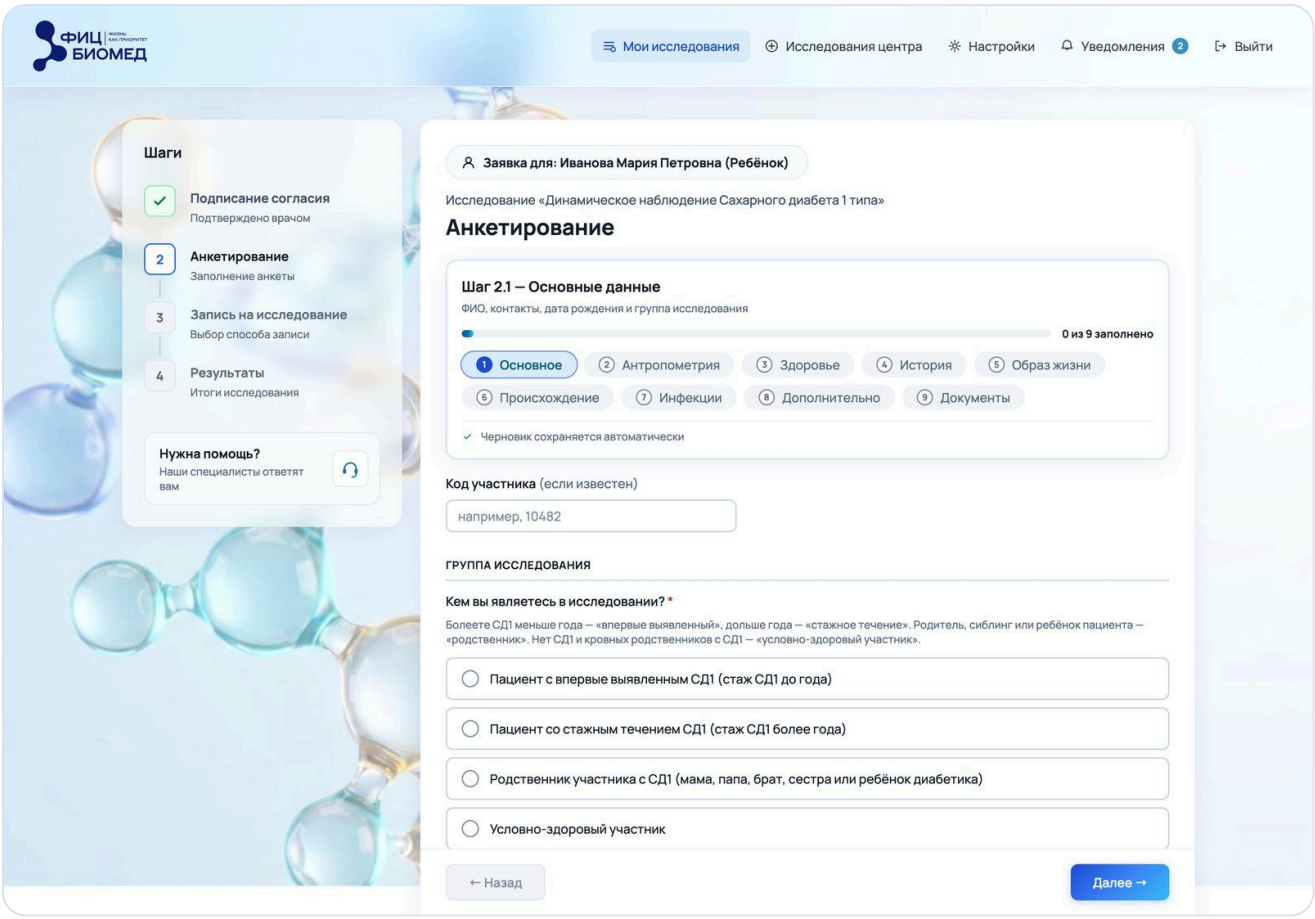
Проверка фокуса:

1. Прищуриться или размыть снимок экрана. Что видно первым? Почему: при размытии исчезают слова, и остаётся только вес элементов, как его видит периферийное зрение.
2. Совпадает ли это с главным действием экрана?
3. Сколько элементов спорят за внимание?

> **ИЗ ПРАКТИКИ**
>
> Первый аудит вида кабинета участника нашёл, что списки собраны спокойно, а в мастере фокус теряется. Плашка помощи была в цвете главной кнопки и спорила с «Далее» на каждом экране. Заголовка не было, весь текст был одного размера 14 px. Прогресс показывался четырежды. Замечания при возврате шли двумя красными блоками. Оранжевый использовался в пяти смыслах. Все 29 пунктов аудита внесли следующей версией, два из них автор потом вернула: на живой странице они читались хуже.



До, версия v02. Синий блок «Нужна помощь?» внизу колонки шагов, заголовка шага нет



После, версия v03. Тихая плашка помощи, заголовок «Анкетирование», подшаги одной навигацией

«Заметно, но не главное»

Самая трудная задача иерархии: элемент должен быть заметен и при этом не перетягивать фокус.

> **ИЗ ПРАКТИКИ**
>
> Плашка «Нужна помощь?» прошла шесть заходов. Сначала она была в градиенте главной кнопки и спорила с «Далее». Потом стала серой и потерялась. Потом почти белой фиолетовой и осталась слишком тихой. Варианты подбирали на белом фоне, а в кабинете фон с молекулами и матовое стекло. Решилось за один заход, когда четыре кандидата сняли прямо в живой странице рядом. Выбрана белая плашка с голубой рамкой. Урок: цвет подбирают на том фоне, где он будет жить.

Сетка и отступы

Сетка делает экран предсказуемым. Основные решения:

- **Шаг сетки.** Обычно 4 или 8 px: все отступы и размеры кратны шагу. Почему: общий шаг делает расстояния предсказуемыми, и разные дизайнеры получают одинаковый ритм.
- **Колонки.** На десктопе часто 12, на планшете 8, на телефоне 4. Почему: деление на 12 даёт половины, трети и четверти, на узком экране колонок меньше, иначе они слишком узкие.
- **Гаттер.** Расстояние между колонками, обычно 16–24 px. Почему: одинаковый гаттер держит блоки на одной сетке, и экран не выглядит случайным.
- **Поля страницы.** На телефоне не меньше 16 px. Почему: текст у самого края экрана трудно читать, а палец задевает края корпуса.

Отступы показывают связь: близкие элементы воспринимаются как группа (закон близости из гештальт-психологии). Внутри группы отступ меньше, между группами больше.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника введена сетка 4: отступы и размеры кратны 4 с округлением вверх, исключения 2 px и 18. Межстрочный интервал задан в пикселях семью парами. Мелкие несоответствия выровнены: плитка иконки 38 → 40, аватар 30 → 32, переключатель 42 → 44.

Типографика

Типографика интерфейса решает две задачи: читаемость и иерархию.

Правила:

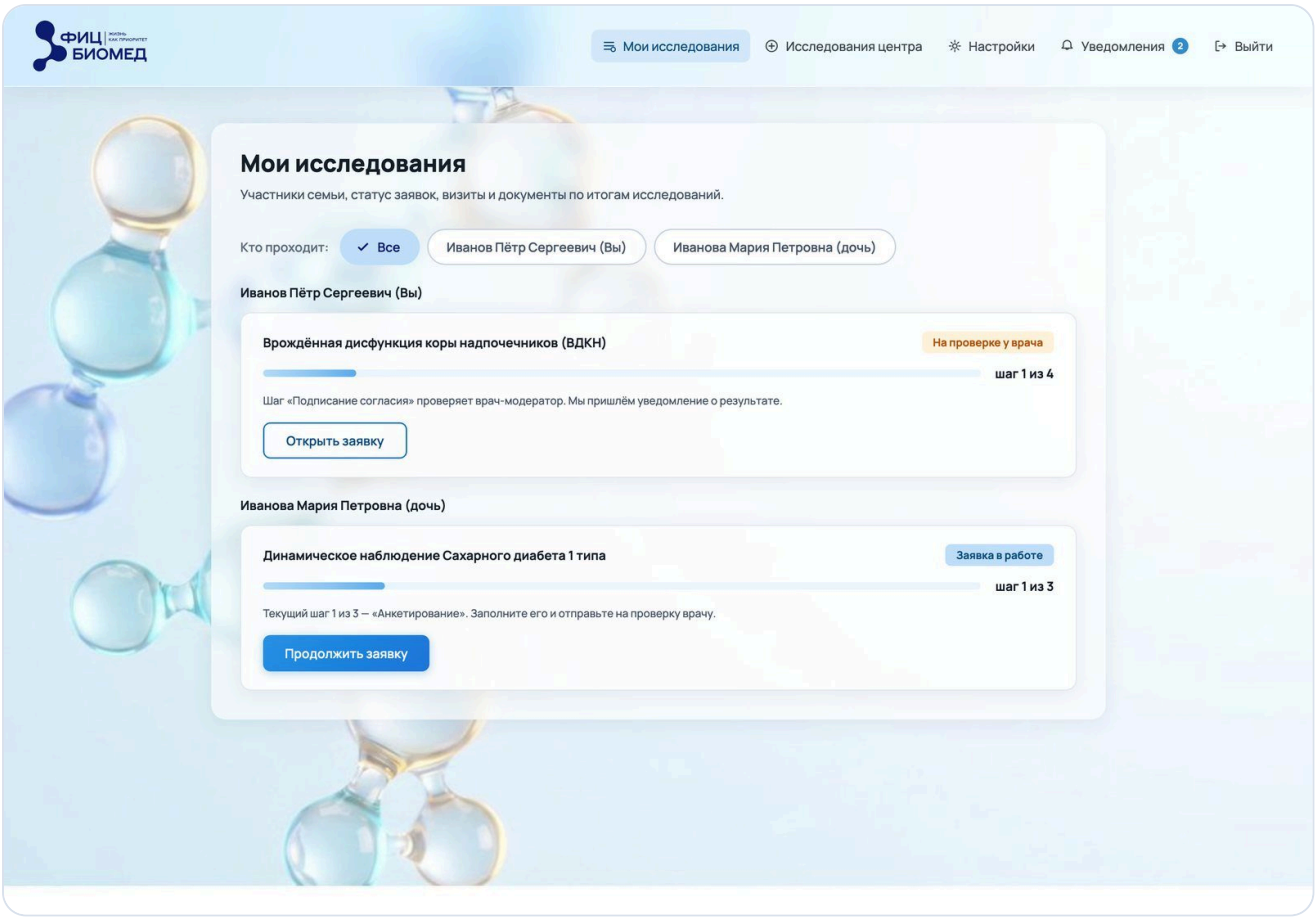
- **Основной текст** 14–16 px на десктопе, на телефоне в полях ввода не меньше 16 px: иначе iOS увеличивает форму при фокусе.
- **Мелкие подписи** не меньше 12 px. Почему: мельче читается с трудом, особенно людьми старшего возраста.
- **Межстрочный интервал** для основного текста около 1,4–1,5 от кегля. Чем мельче заголовок, тем больше нужен запас по интерлиньяжу, чтобы длинные названия не

слипались при переносе.

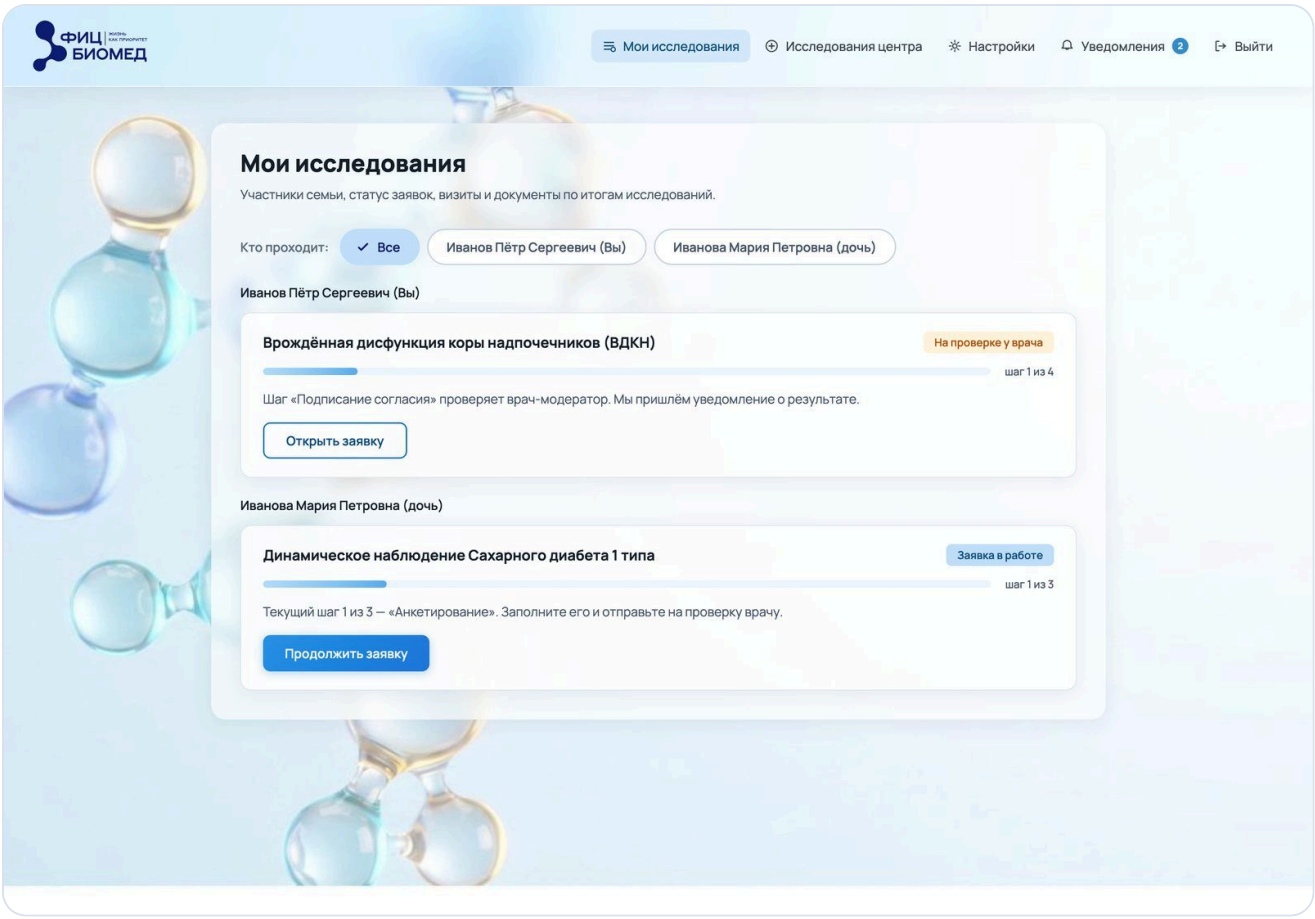
- **Шкала ролей.** Каждый уровень текста описан ролью (заголовок страницы, заголовок карточки, основной текст, подпись) с кеглем, строкой и начертанием. Почему: дизайнер выбирает роль, размер к ней уже привязан, и одинаковые вещи выглядят одинаково на всех экранах.
- **Начертаний немного.** Соседние начертания на одном кегле (500, 600, 700) плохо различаются глазом. Почему: если глаз не видит разницы, разница бесполезна и только создаёт шум.

> **ИЗ ПРАКТИКИ**
>
> Второй аудит шрифта в кабинете участника разобрал 435 объявлений шрифта. Обычного начертания 400 при встроенном Manrope не было вовсе. На кегле 14 жили три соседних начертания. Заголовки групп, названия и числа прогресса весили одинаково. Итоговая шкала:

Роль	Кегль / строка	Начертание
Заголовок страницы и мастера	26 / 32	800
Заголовок главной карточки, окна	18 / 24	700
Заголовок карточки в списке, секции	16 / 24	700 и 600
Подзаголовок, группа, вопрос	14 / 20	600
Основной текст	14 / 20	400
Подпись поля, пункт меню	14 / 20	500
Подпись, счётчик	12 / 16	500
Бейдж, шапка таблицы	12 / 16	600



До, версия v07. «Мои исследования»: имя группы, название исследования и «шаг 1 из 4» одного веса



После, версия v08. Название исследования главнее, счётчик шага тише, описание обычным начертанием

Цвет

Цвет в интерфейсе несёт смысл. Поэтому начинают с ролей, а оттенки подбирают потом.

Типичные роли:

Роль	Где
Основной (бренд, действие)	Главная кнопка, ссылки, выбранное
Нейтральные	Текст, фон, рамки, подложки
Успех	Готово, принято
Предупреждение	Особенность процесса, внимание
Ошибка	Ошибка и опасное действие

Роль	Где
Информация	Справка, подсказка

Правила:

- **Одна роль, один вид.** Если «выбрано» показано шестью способами, человек не узнаёт его.
- **Один цвет, один смысл.** Оранжевый в пяти смыслах перестаёт что-либо значить.
- **Красный только для ошибки и опасного действия.** Почему: если красный значит и ошибку, и пустое поле, человек перестаёт отличать опасное от незаконченного.
- **Цвет не единственный носитель смысла.** Рядом всегда текст или иконка: часть людей цвет не различает.
- **Основной цвет не совпадает со статусом.** Если главная кнопка зелёная, её путают с «успехом».

> **ИЗ ПРАКТИКИ**
>
> Изумрудный основной цвет кабинета участника отклонили: он сливался со статусом «успех». Выбрана шкала «морская волна» вокруг бирюзы. Раньше её откладывали из-за контраста 4,32, притемнение дало 4,68. Минимум контраста главной кнопки в той версии вырос с 2,14 до 4,68.

Шкалы цвета

Цвет задают шкалой ступеней (например, 50–900) и ролями, которые ссылаются на ступени. Так легко сделать тёмную тему и проверить контраст: проверяется пара ролей, и перебирать экраны не нужно.

Контраст

Минимум по WCAG 2.1 уровня AA:

Что	Минимум
Обычный текст	4,5 : 1

Что	Минимум
Крупный текст (от 18 pt или 14 pt жирным, это примерно 24 px и 18,7 px)	3 : 1
Границы полей, иконки, элементы управления	3 : 1

Подробнее в главе 11.

> **ИЗ ПРАКТИКИ**
>
> UI-аудит кабинета участника по WCAG поднял контраст серых подписей с 2,56 до 4,76: цвет заменили в 34 местах. Контраст активного шага вырос с 3,96 до 5,17.

Кнопки

У кнопок несколько уровней веса: главная, второстепенная, текстовая, опасная. Правила:

- **Одна главная кнопка на экран или блок.** Почему: две равные главные кнопки заставляют человека думать, какая из них правильная.
- **Вид кнопки задаёт действие, а экран на него не влияет.** Одно и то же действие везде выглядит одинаково. Почему: человек узнаёт действие по виду, и разный вид одного действия сбивает.
- **«Отмена» нейтральная.** Никогда не яркая и не красная. Почему: яркая «Отмена» спорит с главным действием и выглядит опасной.
- **Опасное действие тише главного.** Удаление аккаунта не должно весить столько же, сколько «Продолжить». Почему: случайное нажатие опасной кнопки стоит дороже, чем лишняя секунда на поиск.
- **Неактивная кнопка объясняет, почему.** Иначе лучше оставить кнопку активной и показать ошибку при нажатии.

ИЗ ПРАКТИКИ

В кабинете участника «Продолжить заявку» была контурной в одном разделе и градиентной в карточке участника. После аудита правило стало общим: вид кнопки зависит от действия. Удаление аккаунта было крупной сплошной красной кнопкой одного веса с главным действием. Стало неброской контурной кнопкой с подтверждением в той же карточке.

Обратная связь и уведомления

Человек должен видеть результат каждого действия. Выбор формы зависит от важности:

Форма	Когда
Изменение на месте	Мелкое действие: отметка, переключение
Тост на несколько секунд	Обратимое действие, которое не требует внимания
Плашка до следующего события	Важное обязательство: запись, оплата
Окно	Нужно решение человека
Уведомление	Событие, которое произошло без человека

ИЗ ПРАКТИКИ

Подтверждение записи на визит в кабинете участника сделано плашкой «Запись оформлена», которая держится до следующей стадии. Причина: запись на визит слишком конкретное обязательство, чтобы подтверждать его тостом на три секунды.

Паттерны

Повторяющиеся задачи решают известными паттернами: пользователь их уже знает.

- **Формы**: подпись над полем, пример в подсказке, ошибка под полем, обязательные поля помечены одинаково. Подробно и с причинами в главе 5.
- **Таблицы**: выравнивание чисел по правому краю, текста по левому; закреплённая шапка; на телефоне таблица превращается в список карточек.
- **Окна**: заголовок называет действие, кнопки продолжают заголовок, закрываются по Esc и клику вне окна. Почему: человек должен понять окно по заголовку и кнопкам, не читая текст, и легко выйти.
- **Лист снизу**: на телефоне вместо окна, большая зона касания. Почему: нижняя часть экрана достаётся большим пальцем, а окно в центре закрывает контекст.
- **Пустые состояния**: что здесь будет и как это получить. Почему: пустой экран без объяснения выглядит как ошибка.
- **Подтверждение опасного действия**: последствия прямым текстом, подтверждение рядом с объектом. Почему: так видно, что именно удаляется и что пропадёт вместе с ним.

> **ИЗ ПРАКТИКИ**
>
> Подтверждение отзыва согласия в кабинете участника сделано в строке участника, без модального окна. Оверлей потерял бы связь с тем, чьё согласие отзывают.

Материал и глубина

Тени, размытие и стекло показывают, что над чем лежит. Правила современных систем похожи: глубину строят тоном, тень оставляют поднятым элементам, стекло отводят слою управления.

> **ИЗ ПРАКТИКИ**
>
> Аудит стекла в кабинете участника сверил решения с Apple HIG, Fluent 2, Material 3 и IBM Carbon. Apple отводит Liquid Glass слою управления и не советует класть стекло на стекло. Fluent держит Acrylic для меню и всплывающих окон. Material 3 и Carbon строят глубину тоном. Аудит нашёл «стекло на стекле» на карточке содержания, одну тень на всех уровнях в 26 местах и отсутствие запасного вида для режима «Уменьшить прозрачность».

Мобильная версия

Мобильная версия может отличаться от десктопа самим устройством экрана, кроме ширины. На телефоне чаще работают:

- **Поток вместо панели:** один шаг на экран, главная кнопка у нижнего края. Почему: на узком экране панель и форма вместе не помещаются. Нижний край экрана при этом достаётся пальцем.
- **Нижний таб-бар вместо бокового меню.** Почему: разделы всегда на виду и под пальцем.
- **Лист снизу вместо окна.** Почему: его удобно закрыть жестом вниз, и он не прячет весь экран.
- **Список вместо таблицы.** Почему: колонки таблицы на узком экране становятся нечитаемыми.
- **Зоны касания не меньше 44 × 44 pt (Apple) или 48 × 48 dp (Material).** Почему: палец намного толще курсора.

Главное правило проверки: обходить все состояния. Экранов мало, состояний много.

> **ИЗ ПРАКТИКИ**
>
> Разбор мобильной версии кабинета участника нашёл 16 поломок, все в состояниях мастера, куда прежние проверки не заходили: кнопки шапки мастера уезжали за экран, колонка шагов обрезалась посреди строки, третья вкладка настроек была недостижима. Урок: «Проверка, которая не заходит в состояния, ничего не проверяет».

> **ИЗ ПРАКТИКИ**
>
> Позже по разбору практик Ozon и Wildberries мастер на телефоне стал потоком: панель со стрелкой и шагом, главная кнопка у нижнего края, без таб-бара. Окна на телефоне стали листом снизу.

Главное в главе

- У экрана одна главная задача, всё остальное уступает ей по весу.
- Сетка и отступы показывают связь элементов.

- Шрифт и цвет описывают ролями: одна роль, один вид.
- Красный только для ошибки и опасного действия.
- Цвет подбирают на том фоне, где он будет жить.
- Важное обязательство подтверждают плашкой, мелочь тостом.
- Мобильную версию проверяют по состояниям.

Вопросы для самопроверки

1. Как быстро проверить фокус экрана?
2. Почему основной цвет не должен совпадать с цветом успеха?
3. Когда тост неуместен?
4. Чем мобильная версия может отличаться от десктопа, кроме ширины?

Упражнение

Возьмите снимок любого экрана с формой. Выпишите все цвета и начертания шрифта, которые на нём есть, и разложите их по ролям. Найдите роль, у которой больше одного вида, и предложите один.

7. Движение и микровзаимодействия

Движение в интерфейсе объясняет, что изменилось и откуда это взялось. Микровзаимодействие отвечает на одно действие человека: нажал, навёл, перетащил, отправил. Хорошее движение незаметно. Плохое мешает работать или укачивает.

Зачем нужно движение

Движение оправдано, когда решает одну из задач:

Задача	Пример	Почему движение помогает
Показать связь	Лист выезжает от кнопки, которая его открыла	Человек понимает, откуда взялся новый слой и куда он денется
Показать изменение состояния	Кнопка меняет вид при нажатии	Человек видит, что система приняла действие
Направить внимание	Новое уведомление появляется в углу	Глаз замечает движение раньше, чем статичный элемент
Показать ожидание	Индикатор загрузки в кнопке	Человек знает, что система работает, и не жмёт повторно
Сохранить ориентацию	Плавная прокрутка к полю с ошибкой	Человек видит, куда его перенесли, и не теряется в длинной форме

Если движение не решает ни одной из этих задач, его убирают. Почему: каждое лишнее движение тратит внимание и время человека.

Состояния элемента

У каждого интерактивного элемента есть состояния. Их показывают всегда, даже если нет анимации.

Состояние	Что показывает	Почему нужно
Обычное	Элемент можно использовать	Базовый вид
Наведение	Элемент кликабелен	На десктопе это главный способ понять, что можно нажать
Фокус	Элемент выбран с клавиатуры	Без него человек с клавиатурой не знает, где он
Нажатие	Действие принято	Подтверждает, что нажатие засчитано
Ожидание	Действие выполняется	Защищает от повторного нажатия
Неактивное	Действие сейчас недоступно	Объясняет, что нажимать бесполезно

ИЗ ПРАКТИКИ

В кабинете участника наведение добавили всему кликабельному. До этого на разных экранах было от 2 до 100 кликабельных элементов без наведения, после правки ноль. Кнопке в дизайн-системе добавили состояния нажатия и ожидания. Ожидание добавили по правилам системы, хотя в статичном прототипе его не воспроизвести. Что это даёт: разработка знает, как выглядит кнопка, пока запрос идёт на сервер.

Длительность и плавность

Ориентиры, которые используют крупные системы (Material, Apple, Carbon):

Что	Длительность	Почему
Наведение, нажатие	100–150 мс	Ответ должен ощущаться мгновенным

Что	Длительность	Почему
Появление мелкого элемента	150–250 мс	Глаз успевает заметить, работа не тормозит
Окно, лист, панель	250–400 мс	Крупный объект движется дольше, иначе движение кажется рывком
Переход между экранами	300–500 мс	Дольше уже воспринимается как ожидание

Плавность (кривая ускорения):

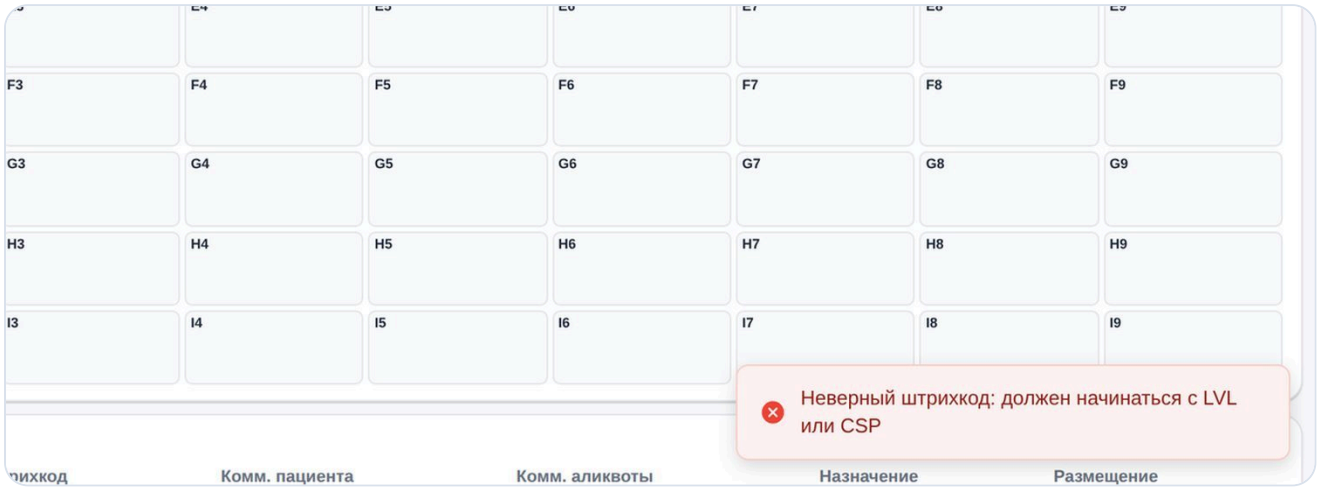
- **Появление: замедление в конце** (ease-out). Почему: объект быстро входит и мягко останавливается, как настоящий.
- **Исчезновение: ускорение в конце** (ease-in). Почему: уходящее не держит внимание.
- **Перемещение: ускорение и замедление** (ease-in-out). Почему: так двигаются предметы в жизни.
- **Линейная кривая только для бесконечных индикаторов.** Почему: в остальных случаях она выглядит механически.

Обратная связь на действие

- **Каждое действие получает ответ.** Почему: без ответа человек повторяет действие, и система получает два запроса.
- **Ответ стоит там, где человек смотрит.** Почему: ответ в другом углу экрана могут не заметить.
- **Форма ответа зависит от важности.** Мелкое действие меняет элемент на месте, важное получает плашку, событие без участия человека приходит уведомлением. Почему: если всё подтверждать одинаково громко, важное тонет в мелочах.

ИЗ ПРАКТИКИ

На станции биобанка уведомления появляются в углу на 1,8 секунды и ничего не блокируют. Почему так: сканер работает как клавиатура, и окно, которое перехватывает фокус, проглатывает коды. Вторая причина: окно пришлось бы закрывать курсором в перчатках.



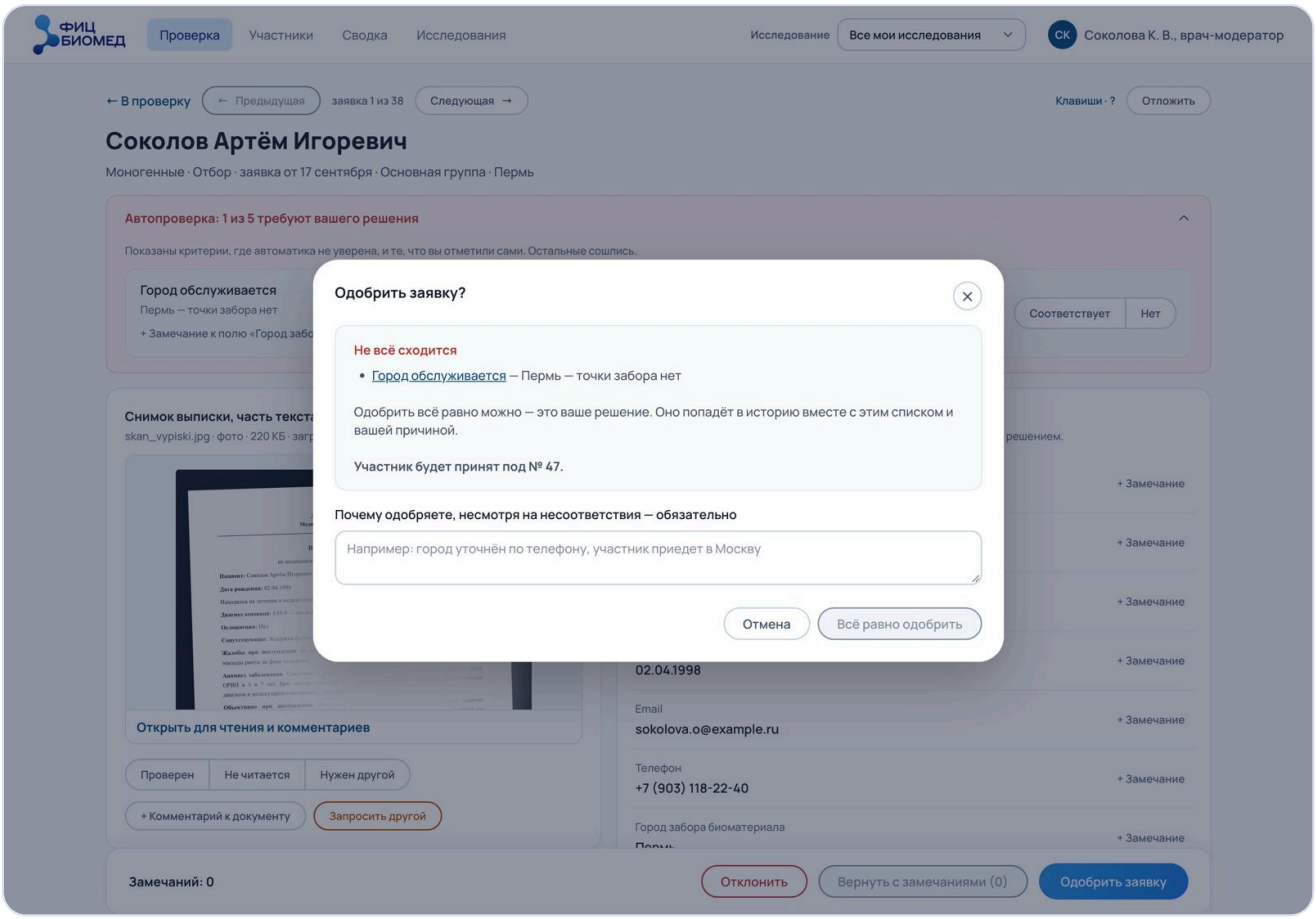
Станция биобанка: уведомление об ошибке формата в углу экрана, работу оно не блокирует

Отмена вместо подтверждения

Для частых действий отмена удобнее окна подтверждения. Человек действует сразу, а в уведомлении несколько секунд есть кнопка «Отменить».

ИЗ ПРАКТИКИ

В кабинете врача-модератора решение по заявке без несоответствий проходит без окна, с отменой 10 секунд в уведомлении. Уведомление участнику уходит только после окна отмены. Почему: окно на каждой заявке перестают читать, а из-за необратимости решение перепроверяют по три раза. Окно осталось только при несоответствиях, возврате и запросе документа.



Кабинет врача: окно «Одобрить заявку?» появляется только при несоответствии, причина обязательна

Скелеты и индикаторы загрузки

Ожидание	Что показывать	Почему
До 1 секунды	Ничего или смену состояния кнопки	Индикатор мелькнёт и отвлечёт
1–3 секунды	Индикатор в месте действия	Человек видит, что система работает
Загрузка экрана	Скелет: серые блоки в форме будущего содержимого	Человек заранее видит раскладку, и ожидание кажется короче
Дольше 10 секунд	Прогресс с объяснением	Без объяснения долгое ожидание выглядит как зависание

Уважение к настройкам

- **«Уменьшить движение»** (`prefers-reduced-motion`): крупные перемещения заменяют на появление без сдвига. Почему: у части людей движение вызывает головокружение и тошноту.
- **«Уменьшить прозрачность»** (`prefers-reduced-transparency`): размытие заменяют плотной поверхностью. Почему: текст на размытом фоне хуже читается.
- **Ничего не мигает чаще трёх раз в секунду.** Почему: это может вызвать приступ у людей со светочувствительной эпилепсией.

Как проверять движение

Движение плохо проверяется по снимкам: снимок показывает один кадр, и хода по нему не видно. Автоматические проверки в скрытых вкладках браузера часто вообще не видят анимации.

> **ИЗ ПРАКТИКИ**
> Одна из трёх ошибочных находок прогона сценариев кабинета участника: анкета якобы «не доводила до первой ошибки». На деле плавная прокрутка не анимируется в скрытой вкладке, где шла проверка. Почему важно: если не знать особенностей среды проверки, можно «починить» то, что работает. Свой обработчик прокрутки добавили и сразу удалили: встроенный в браузер делал строго больше.

Правила проверки:

- смотреть движение глазами в обычном окне;
- проверять на медленном устройстве или с замедлением в инструментах разработчика;
- проверять с включённым «Уменьшить движение».

Спецификация движения

Для разработки движение описывают так же точно, как цвет:

- что движется;

- откуда и куда;
- длительность в миллисекундах;
- кривая ускорения;
- что запускает движение;
- что происходит при «Уменьшить движение».

Почему: «плавно выезжает» каждый разработчик поймёт по-своему, а числа читаются одинаково.

Главное в главе

- Движение оправдано, когда показывает связь, изменение, ожидание или направляет внимание.
- Состояния элемента показывают всегда: наведение, фокус, нажатие, ожидание.
- Мелкое быстрее, крупное медленнее, появление с замедлением в конце.
- Для частых действий отмена удобнее окна подтверждения.
- Движение уважает «Уменьшить движение».
- Движение проверяют глазами в настоящей среде.

Вопросы для самопроверки

1. Почему у кнопки должно быть состояние ожидания?
2. Почему окно появляется дольше, чем подсветка при наведении?
3. Когда отмена лучше окна подтверждения, а когда нет?
4. Что делать с анимацией, если включено «Уменьшить движение»?

Упражнение

Выберите три действия в любом приложении: нажатие кнопки, открытие окна, удаление элемента. Для каждого запишите спецификацию движения: что движется, откуда и куда, длительность, кривая, вариант для «Уменьшить движение».

8. Рабочие места и интерфейсы с данными

Рабочее место это интерфейс, в котором человек проводит смену: врач проверяет заявки, лаборант раскладывает пробирки, оператор разбирает обращения. Здесь главное это сотое повторение одного и того же действия. Первое впечатление важно меньше. Секунда, сэкономленная на операции, превращается в часы за месяц.

Чем рабочее место отличается от сервиса для клиентов

Сервис для клиентов	Рабочее место	Что из этого следует
Человек приходит редко	Человек работает каждый день	Можно учить сочетаниям клавиш и плотной раскладке
Важно понять с первого раза	Важно делать быстро и без ошибок	Плотность важнее воздуха
Один сценарий за визит	Сотни повторов за смену	Каждое лишнее нажатие умножается на сотни
Мобильный в приоритете	Компьютер в приоритете	Мобильная версия вспомогательная
Ошибка стоит клиенту	Ошибка стоит организации и данным	Защита от ошибок важнее скорости

Начинать с условий работы

Перед экранами записывают, где и как человек работает: что у него в руках, на что он смотрит, что его прерывает, какой у него экран.

ИЗ ПРАКТИКИ

Лаборант на станции биобанка держит в одной руке пробирку, в другой сканер, на руках перчатки. Снимать перчатки нужно как можно реже: «Перчатки обязательны при работе с биоматериалом, и каждая замена стоит времени, расходников и риска». Сканер вводит символы туда, где фокус. Лаборант постоянно сверяет экран со штативом на столе. Ноутбук 13 дюймов. Отсюда, по словам автора, «почти вся механика станции»: ввод сканером, следующую позицию выбирает система, данные сохраняются сами, всё, что можно сделать сразу для нескольких аликвот, делается сразу, клавиатура только для исключений.

ИЗ ПРАКТИКИ

Врач-модератор работает за компьютером сменой, мобильная версия вспомогательная. Самый частый путь врача «что ждёт меня сейчас». Поэтому исследование выбирается сквозным переключателем в шапке, и работа с выбора не начинается.

Один экран или несколько

- **Если человек постоянно сверяет данные, всё нужное держат на одном экране.** Почему: каждое переключение удлиняет сверку, а значит, её будут делать реже.
- **Если работы разные по смыслу, их разводят по разделам.** Почему: смешанный экран заставляет каждый раз искать своё.

ИЗ ПРАКТИКИ

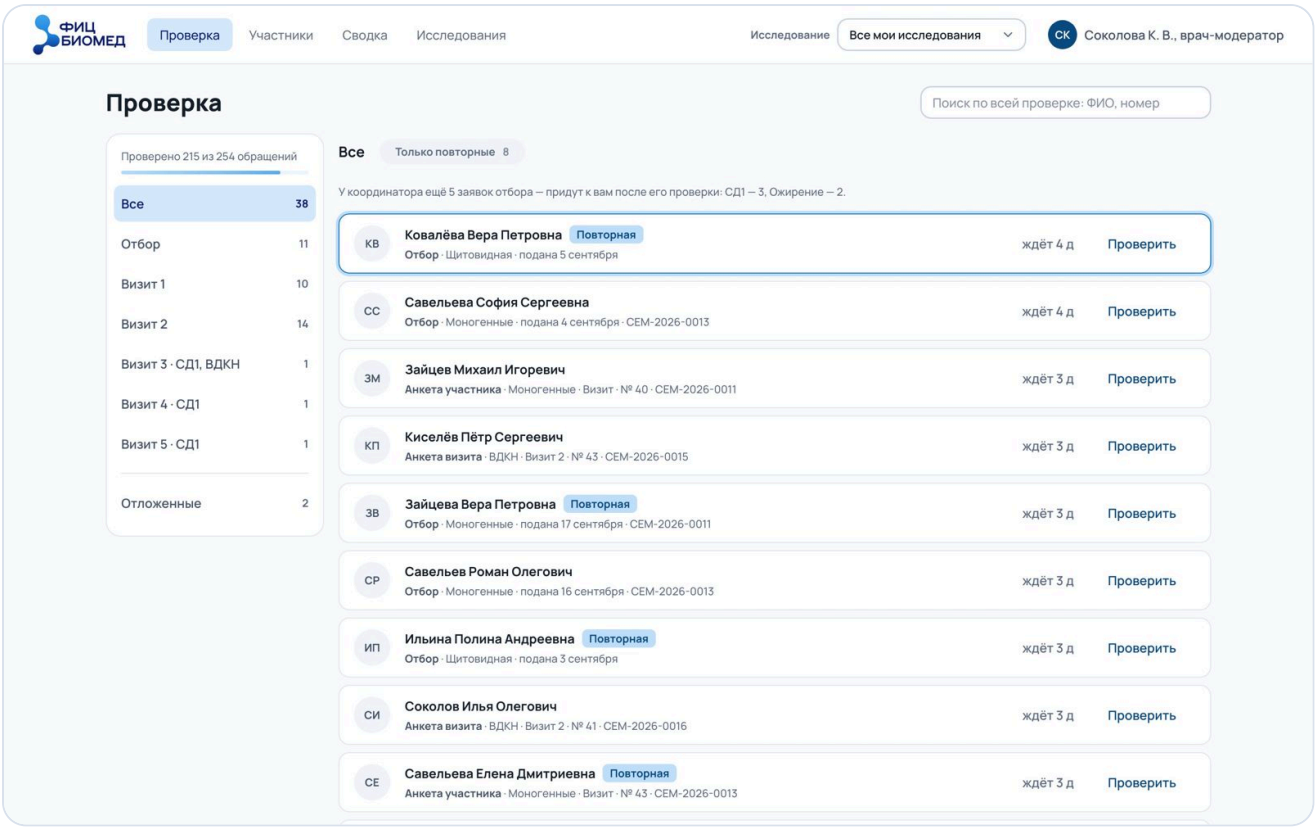
Станция биобанка устроена одним экраном без вкладок и переходов: настройки серии, сводка, карточки аликвот, штатив, редактор, реестр. Довод из кейса: «Каждое переключение делает сверку дольше, а значит, реже». Раскладка подстраивается под ширину и обходится без вертикальной прокрутки. Почему: лаборант должен видеть штатив, карточки серии и реестр одновременно.

Очередь

Очередь отвечает на вопрос «что мне делать сейчас». Правила:

- **В очереди только то, что ждёт действия этого человека.** Почему: если там же лежит то, что ждёт других, на вопрос «я всё разобрал?» ответить нечем.
- **Пустая очередь это хорошая новость.** Почему: человек должен видеть, что смена закончена.
- **Дольше ждущие выше.** Почему: так никто не застрянет в конце списка.
- **Строка говорит, что ждёт и сколько ждёт.** Почему: решение «браться ли сейчас» человек принимает по строке, не открывая карточку.
- **Кнопка в строке называет задачу.** «Проверить анкету», «Связаться». Почему: человек сразу видит, какая работа его ждёт.
- **Нажимается вся строка.** Почему: маленькую кнопку в длинной строке трудно попасть.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора сначала был один список со статусными фильтрами. Выбрали раздел «Проверка» только из того, что ждёт действия, остальное ушло в «Участников». Довод: пока заявка «возвращена» лежит рядом с «ждёт проверки», на вопрос «я всё разобрал?» ответить нечем. Очередь делится по визитам протокола: «Отбор», «Визит 1…N». Почему: врач мыслит визитами протокола. Сначала предлагали деление по типу работы, но автор поправила по устройству исследования.



Кабинет врача: раздел «Проверка», колонка очередей слева, в строке что ждёт и сколько ждёт

> **ИЗ ПРАКТИКИ**
>
> Разметку очередей выбирали из трёх вариантов. Вкладки в ряд не подошли: у одного исследования семь вкладок плюс отложенные, и ряд переносился. По умолчанию стоит колонка очередей слева с числом у каждой, а «Отложенные» стоят в той же колонке за чертой. Два других варианта оставлены в прототипе для сравнения. Довод за колонку: при многих очередях колонка читается лучше ряда, и десять разобранных продуктов держат очереди колонкой.

Таблицы и реестры

Реестр хранит всё. Правила таблиц для работы:

- **Строка это объект, с которым работают.** Почему: если строка не совпадает с единицей работы, фильтры и действия путаются.
- **Числа по правому краю, текст по левому.** Почему: так числа сравниваются по разрядам.
- **Сортировка отвечает «кто первый», фильтр отвечает «с кем я сейчас работаю».** Почему: это разные вопросы, и одна колонка на оба не отвечает.

- **Шапка закреплена.** Почему: в длинной таблице без шапки непонятно, какая колонка какая.
- **Плотность выше, чем в клиентском интерфейсе.** Почему: человеку нужно видеть больше строк сразу.

> **ИЗ ПРАКТИКИ**
>
> В разделе «Участники» кабинета врача строка это участие в исследовании. Что это даёт: один человек может участвовать в нескольких исследованиях, и у каждого участия свой этап и свои решения. Колонку «Этап» разделили на «Визит» и «Шаг» и добавили фильтр по шагу. Довод: «сортировка отвечает „кто первый“, фильтр — „с кем я сейчас работаю“».

> **ИЗ ПРАКТИКИ**
>
> Скругление панелей в кабинете врача сделали 16 вместо 28, как в кабинете участника, и записали исключением из дизайн-системы. Почему: рабочее место плотное, и у таблиц большое скругление срезает углы строк.

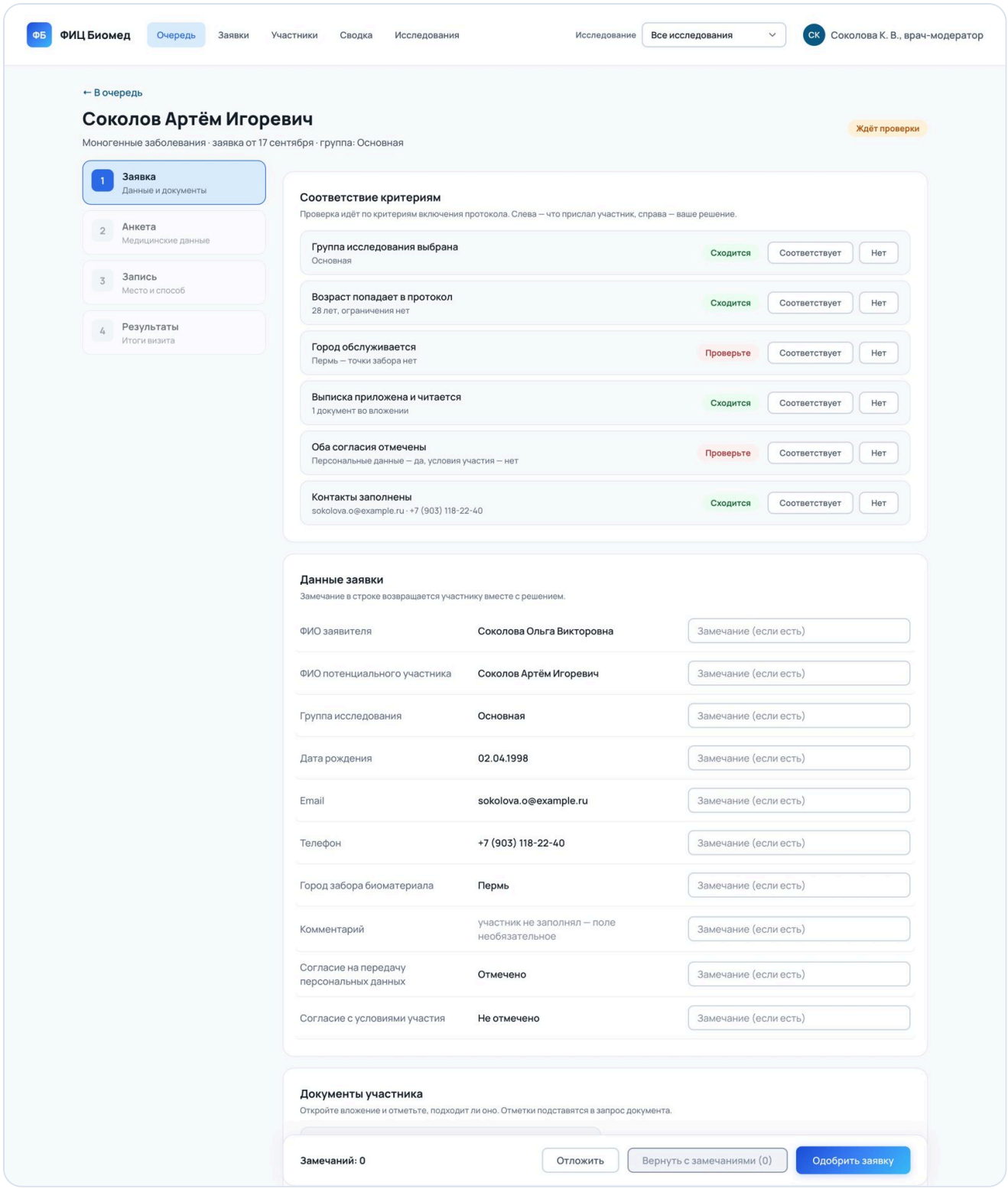
Карточка объекта

Карточка показывает один объект целиком для решения.

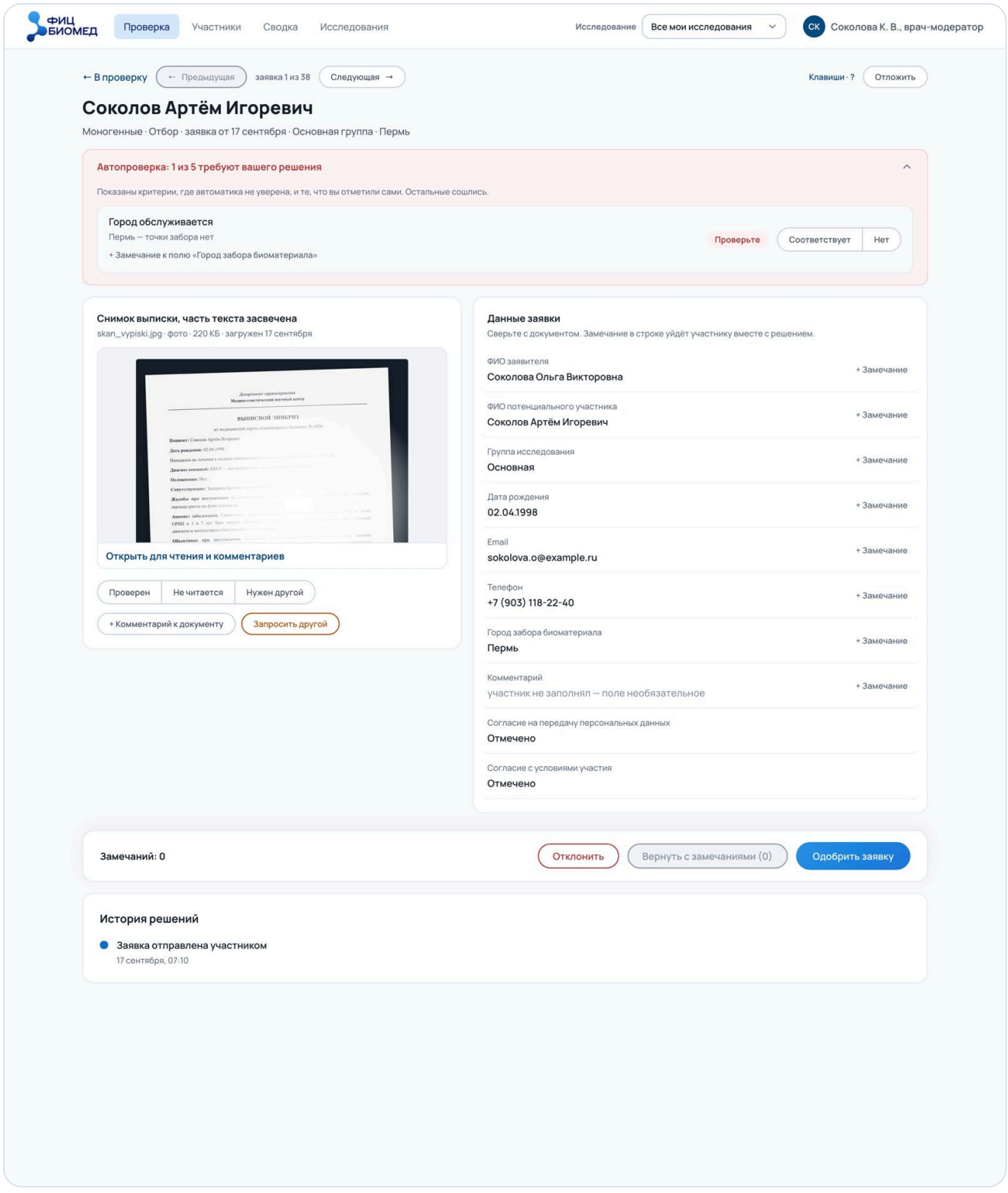
- **Сначала данные, потом выводы.** Почему: решение принимает человек, и он должен видеть то, на чём оно основано.
- **Автоматическая проверка свёрнута, пока всё сходится, и раскрывает спорное.** Почему: внимание уходит туда, где нужно решение.
- **Документ рядом с данными для сверки.** Почему: сверка идёт глазами слева направо, а переключение окон её замедляет.
- **Навигация к следующему объекту из карточки.** Почему: в смене человек идёт по очереди, и возвращаться в список каждый раз долго.

ИЗ ПРАКТИКИ

Карточка отбора в кабинете врача сначала открывалась выводами: шесть критериев «Сходится», данные ниже экрана и двенадцать кнопок «Соответствует / Нет». После слов «карточка проверки тоже непонятна» её пересобрали: автопроверка свёрнута, документ и данные в две колонки, ручной переключатель только у спорных критериев.



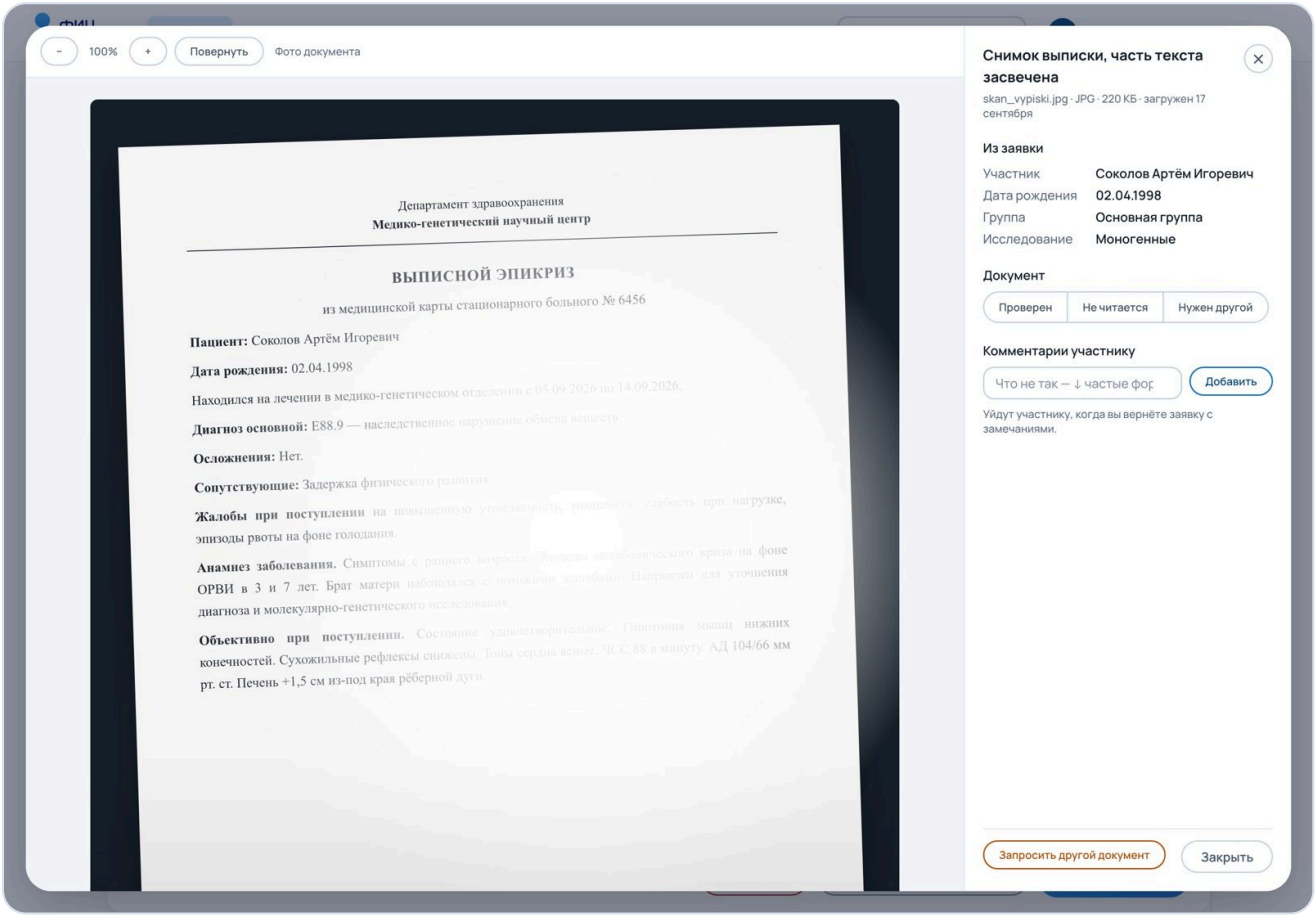
До, версия v14. Карточка отбора открывалась критериями и двенадцатью кнопками «Соответствует / Нет»



После, версия v83. Та же заявка: автопроверка раскрывает только спорный критерий, документ и данные в две колонки

ИЗ ПРАКТИКИ

Документ участника это выписка на лист А4, часто фото листа, и «в колонке его не прочесть». Поэтому документ открывается в окне на 90 % экрана с масштабом и поворотом, а справа стоят данные для сверки и комментарии участнику. Слово автора: «врачу должно быть удобно читать выписку и писать комменты».



Кабинет врача: окно чтения документа, справа данные для сверки и комментарии участнику

Решения и их защита

- **Решения называют действием.** «Одобрить», «Вернуть с замечаниями». Почему: человек видит, что произойдёт.
- **Окно подтверждения только там, где есть риск.** Почему: окно на каждом решении перестают читать.
- **Решение вопреки автоматике возможно, но с причиной.** Почему: решение за специалистом, а журнал должен отвечать и «что», и «почему».

- **Опасные действия не делают пакетом.** Почему: пакетное одобрение пропускает то, что система сама считает несоответствием.
- **Журнал решений.** Почему: коллега, который подменяет, должен понять, что и почему уже решено.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача при несоответствии критериям есть кнопка «Всё равно одобрить» с обязательной причиной, а несоответствия попадают в историю. Пакетного одобрения нет, массовыми бывают только безопасные действия: отложить, взять себе.

Статусы в физическом мире

Если интерфейс отражает физические объекты, статус не должен обгонять реальность.

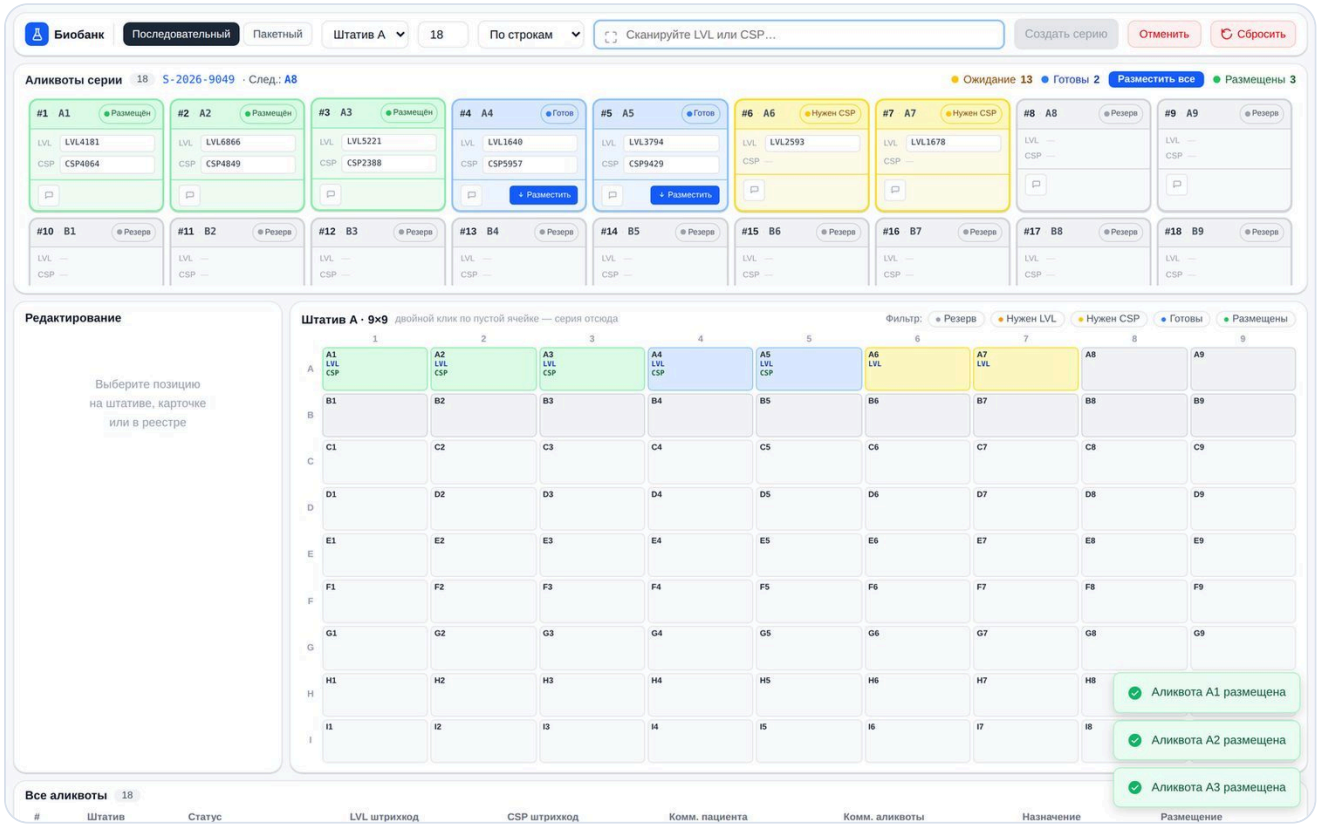
> **ИЗ ПРАКТИКИ**
>
> На станции биобанка можно было считать аликвоту размещённой сразу после второго скана. Выбрали два статуса: «Готов», когда оба кода привязаны, и «Размещён», когда лаборант подтверждает, что пробирка в ячейке. Довод: «Отсканированная пробирка может остаться лежать на столе. Если система засчитает её раньше, база разойдётся с тем, что стоит в штативе». Чтобы не нажимать подтверждение на каждую пробирку, есть «Разместить все».

Визуализация плотных данных

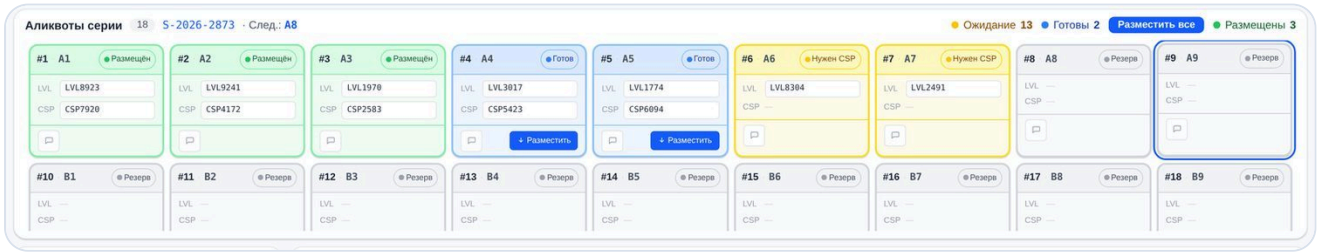
- **Объект читается целиком, детали по запросу.** Почему: человек сначала охватывает картину, потом смотрит частности.
- **Статус кодируется двумя способами: цветом и формой или текстом.** Почему: часть людей плохо различает оттенки, а освещение на рабочем месте бывает разным.
- **Фильтр подсвечивает прямо на схеме.** Почему: видно и сколько работы, и где она.

ИЗ ПРАКТИКИ

В ячейке штатива сначала помещалось всё: статус текстом, оба полных кода, комментарий. Влезала только сетка 4×4. Выбрали номер позиции, последние четыре цифры каждого кода и значок комментария, а полный код в редакторе. Почему: штатив должен читаться целиком. Статус кодируется заливкой и цветной левой границей в 2 px. Довод: «часть людей плохо различает оттенки, а освещение в лабораториях разное», а границу видно боковым зрением. Фильтр по статусу подсвечивает ячейки прямо в штативе.



Станция биобанка: серия, штатив 9 × 9, редактор и реестр на одном экране



Станция биобанка: карточки аликвот серии, статус заливкой и цветной левой границей

Что убрать

На рабочем месте лишний элемент стоит дороже, чем на витрине.

> **ИЗ ПРАКТИКИ**
>
> Со станции биобанка убрали панель очереди: ячейки и так окрашены по статусу, очередь повторяла штатив и занимала место. Убрали переключатель горизонтального и вертикального вида: «Лаборант всегда смотрит на штатив с одной стороны». Оформление сделали плотнее Material Design 3: скругления 8–14 px, тонкие тени, моноширинный шрифт для штрихкодов. Довод: «Большие отступы Material Design проиграли требованию видеть всю рабочую зону сразу».

Слова специалистов

Рабочее место говорит словами тех, кто в нём работает.

> **ИЗ ПРАКТИКИ**
>
> На станции «батч» заменили на «серию» везде, включая тексты уведомлений: «Лаборанты так не говорят». Сетка сначала называлась стеллажом, «хотя стеллаж это мебель, на которой стоят штативы». Довод автора: «Обе замены заняли пару часов, но от них зависит, поймёт ли лаборант экран без обучения».

Клавиатура

- **Сочетания клавиш для частых действий.** Почему: опытный человек работает быстрее с клавиатуры, чем мышью.
- **Подсказка по сочетаниям по клавише `?`.** Почему: так сочетания можно найти, не читая инструкцию.
- **Однобуквенные сочетания можно выключить.** Почему: по WCAG 2.1.4 они мешают людям, которые пользуются голосовым вводом.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача стрелки двигают по очереди, Enter открывает, A выполняет главное действие экрана, Esc закрывает, `?` показывает подсказку. Однобуквенные сочетания можно выключить.

Главное в главе

- Рабочее место проектируют от условий работы.
- Всё, что нужно для сверки, держат на одном экране.
- В очереди только то, что ждёт действия этого человека.
- Сортировка и фильтр отвечают на разные вопросы.
- Окно подтверждения только там, где есть риск.
- Статус не обгоняет физическую реальность.
- Рабочее место говорит словами специалистов.

Вопросы для самопроверки

1. Почему плотность на рабочем месте выше, чем в клиентском сервисе?
2. Что должно быть в строке очереди, чтобы не открывать карточку?
3. Почему пакетное одобрение опасно?
4. Зачем статус «Размещён» отдельно от «Готов»?

Упражнение

Понаблюдайте 30 минут, как кто-то работает в рабочей программе: кассир, администратор, диспетчер. Запишите, какие действия повторяются, что человек держит в руках, на что смотрит, что его прерывает. Предложите три изменения и объясните каждое.

9. Дизайн-системы

Дизайн-система хранит решения, которые команда уже приняла: цвета, отступы, шрифты, компоненты и правила их использования. Она нужна, чтобы новый экран не начинался с нуля и чтобы одна вещь везде выглядела одинаково.

Из чего состоит система

Слой	Что внутри	Пример
Основы (токены)	Цвета, отступы, радиусы, размеры, шрифты, тени	`spacing/16`, `color/primary/600`
Атомы	Простые компоненты	Кнопка, поле, чип, чекбокс
Композиты	Собранные из атомов блоки	Карточка заявки, шапка, строка таблицы
Паттерны	Правила для повторяющихся задач	Подтверждение удаления, пустое состояние
Документация	Как и когда использовать	Варианты, состояния, примеры «так и не так»
Связь с кодом	Как токен называется в коде	`var(--spacing-16)`

Система без документации и связи с кодом быстро превращается в красивую библиотеку, которой никто не пользуется.

Токены

Токен это значение с именем. Имена бывают двух уровней:

- **Примитивы** называют само значение: `blue/600`, `spacing/16`. Почему: примитивы задают палитру и шкалу, из которых всё собирается.
- **Семантические токены** называют роль: `color/action/primary`, `color/text/muted`, `color/border/field`. Почему: роль переживает смену значения, и компонент не нужно

переделывать при смене бренда или темы.

Компоненты ссылаются на семантические токены, а те на примитивы. Тогда смена бренда или тёмная тема меняют только связь роли с примитивом.



Компоненты ссылаются на роли, роли ссылаются на значения

Правила имён:

- имя говорит, где применять; Почему: дизайнер выбирает токен по задаче, и имя должно на задачу отвечать.
- имя не кодирует значение, которое может измениться; Почему: когда значение поменяется, имя начнёт врать.
- одно имя на одну роль. Почему: два имени на одну роль со временем разойдутся.

> **ИЗ ПРАКТИКИ**
> В кабинете участника пять служебных классов кодировали в имени старое значение: класс «про 14 пикселей» на деле давал 16. Их переименовали. В прототипе было 63 уникальных цвета, и один синий работал фоном кнопки, цветом активного шага и рамкой. Цвета разобрали по ролям (фон, текст, граница) и получили переменные с именами, которые говорят, где их применять.

Откуда берутся значения

Есть два пути:

1. **Сначала система, потом продукт.** Подходит, когда продукт только начинается и есть время договориться об основах. Почему: договориться об основах проще до

того, как появились сотни экранов.

2. **Система из продукта.** Значения снимают с работающего прототипа или кода, сводят и чистят. Почему: значения уже проверены на настоящих экранах.

Второй путь часто честнее: значения уже проверены на реальных экранах, и вёрстка по системе даёт тот же результат, что и продукт.

> **ИЗ ПРАКТИКИ**
>
> Дизайн-систему кабинета участника автор собрала из прототипа. Значения сняты с фактической разметки, поэтому вёрстка по Figma и по прототипу даёт одинаковый результат. Причина, по которой система понадобилась: при добавлении нового элемента было непонятно, какое значение взять за образец.

Сведение значений

Когда систему собирают из живого продукта, первым делом находят разброс.

> **ИЗ ПРАКТИКИ**
>
> Первый сбор системы кабинета участника нашёл 31 уникальную тень. Их свели к восьми токенам по назначению: карточка, окно, тост, кнопка, поле и другие. У текста одного размера было до девяти значений интерлиньяжа, стало три: 1,2 для заголовков, 1,4 для компактного текста, 1,5 для текста в две строки и больше. Отступы 2, 3, 5, 6, 7, 9, 10, 11 и ещё десяток значений привели к сетке, кратной 4. Высоты контролов свели к 36 для кнопок, полей и списков и к 32 для компактных.

Не всё сводится. Если разброс нельзя убрать сейчас, его записывают как долг с числом случаев.

Компоненты

Хороший компонент:

- описан вариантами (тип, размер) и состояниями (обычное, наведение, нажатие, фокус, неактивное, ожидание, ошибка); Почему: разработке нужны все состояния, иначе недостающие придумают на ходу.
- использует только токены, без чисел внутри; Почему: число внутри компонента не обновится при смене системы.
- имеет понятные свойства, привязанные к переменным; Почему: свойство меняет компонент переключателем, без ручной правки.
- документирован: когда использовать, когда нет. Почему: без документации компонент используют не по назначению.

Вариант или отдельный компонент

Частый вопрос: добавить кнопке ещё один вариант или завести новый компонент. Простое правило:

- если отличается вид при той же роли, это **вариант**; Почему: роль та же, и человек узнаёт элемент.
- если отличается роль или поведение, это **отдельный компонент**. Почему: другая роль требует другого вида, иначе человек спутает действия.

Например, «Отмена» выглядит похоже на второстепенную кнопку, но у неё своя роль: она никогда не бывает яркой. Поэтому её удобно держать отдельным типом.

Не удалять неиспользуемые состояния

Если состояние сейчас нигде не встречается, это не повод его удалять. Состояние неактивной кнопки или ожидания понадобится при следующей задаче, и восстановить его дороже, чем сохранить.

Связь с Figma и кодом

Система живёт в двух местах: в файле дизайна и в коде. Они расходятся, если их не сверять.

Что помогает:

- **У каждой переменной синтаксис для кода.** В Figma это code syntax: `spacing/16 → var(--spacing-16)`. Почему: разработчик берёт готовое имя и не подбирает значение на глаз.

- **Реестр сопоставления.** Какой фрагмент кода какому компоненту соответствует. Почему: без него непонятно, что менять в коде при правке компонента.
- **Автоматическая сверка.** Скрипт сравнивает значения в коде и в системе и выдаёт отчёт. Почему: глазами расхождения в десятках значений не найти.
- **Проверка проверки.** Сверку время от времени проверяют нарочной поломкой. Почему: сломанная сверка тоже показывает ноль.

> **ИЗ ПРАКТИКИ**
>
> Для кабинета участника завели реестр сопоставления фрагментов прототипа и компонентов Figma. Из кейса: «Без такого реестра система расходится с продуктом за пару недель и превращается в красивый архив». Сверку прототипа с системой гоняет отдельный скрипт, а отчёт показывает расхождения. Когда выяснилось, что сверка три недели шла по старому файлу и показывала ноль, источник переключили, и с тех пор нулю верят только после пойманной поломки.

Что система не ловит

Система следит за значениями. Иерархию она не проверяет.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора автор заметила: первая версия соответствует дизайн-системе, а десятая нет. Слепок стилей обеих версий совпал побайтно. Разошлась иерархия: кнопка «Одобрить заявку» уехала под счётчик. Вывод: проверка контраста ловит цвет, проверка сетки ловит отступ, а «главное действие уехало вниз» не ловит ни одна автоматическая проверка.

Отсюда правило: после автоматической проверки экран всё равно смотрят глазами целиком.

Второй продукт на системе

Польза системы видна, когда на ней строят второй продукт.

> **ИЗ ПРАКТИКИ**
>
> Кабинет врача-модератора собран на той же системе, что и кабинет участника. Ни одно значение в его экранах не придумывали заново: кнопки 40, 36 и 32, поле 36 с рамкой 1,5, чип 28, та же типографика и сетка, кратная четырём. Контраст 21 пары проверен, исключения те же, что в кабинете участника: подсказка в поле и неактивная кнопка.

Общий источник или копии

Второй продукт может читать значения из первого или держать свою копию. У каждого пути цена.

> **ИЗ ПРАКТИКИ**
>
> Сначала кабинет врача читал значения из кабинета участника. От этого отказались в тот же день: правка в одном продукте меняла бы вид другого, когда об этом никто не просил и никто этого не увидит. Выбраны копии значений с переносом отдельной командой. Цена: копии расходятся. С правилами поступили наоборот: правила пользования системой лежат одной копией в общей папке. Формула: состояние у каждого продукта своё, знание общее и в одном экземпляре.

Ещё одно наблюдение из той же работы: система переносит значения и не переносит выученные уроки. Три из девяти дефектов первого разбора кабинета врача повторяли ошибки кабинета участника. Для уроков нужен другой носитель, например общий файл находок.

Документация компонента

Минимальная страница компонента:

1. Назначение: для чего и когда. Почему: первым делом человек решает, тот ли это компонент.
2. Анатомия: из чего состоит.
3. Варианты и состояния с примерами.
4. Поведение: что происходит при нажатии, фокусе, ошибке.

5. Доступность: роль, подпись, фокус, контраст. Почему: доступность закладывают в компонент один раз, и она есть на всех экранах.
6. Тексты: как формулировать подписи.
7. Так и не так: примеры правильного и неправильного использования. Почему: пример ошибки запоминается лучше правила.

Главное в главе

- Компоненты ссылаются на семантические токены, токены на примитивы.
- Имя токена говорит, где применять, и не кодирует значение.
- Систему можно и полезно собирать из работающего продукта.
- Разброс сводят, а то, что не свелось, записывают как долг.
- Систему сверяют с кодом скриптом, а сверку проверяют поломкой.
- Иерархию система не ловит, её проверяют глазами.
- Польза системы видна на втором продукте.

Вопросы для самопроверки

1. Чем семантический токен отличается от примитива?
2. Когда нужен новый компонент, а когда вариант?
3. Почему нельзя удалять неиспользуемые состояния?
4. Что дизайн-система не может проверить автоматически?

Упражнение

Откройте любой сайт и выпишите все тени, радиусы и цвета кнопок, которые найдёте на трёх экранах. Сведите их к минимальному набору токенов и придумайте им имена по роли.

10. Работа в Figma

Figma стала общим местом работы дизайнеров, разработчиков и продактов. Глава про то, как устроить файл, чтобы им можно было пользоваться через полгода, и про ловушки, которые стоят дороже всего.

Устройство файла

Страница	Что на ней	Почему отдельно
Обложка	Название, статус, ссылки, ответственный	Человек сразу понимает, туда ли попал
Основы	Цвета, шрифты, отступы, радиусы, тени	Значения меняются редко и нужны всем
Атомы	Простые компоненты, по странице на атом или группу	Компонент легко найти и проверить все его варианты
Композиты	Блоки из атомов	Отделены от атомов, чтобы было видно, из чего что собрано
Флоу	Экраны сценариев со стрелками	Показывают путь и развилки
Мобильная версия	Экраны телефона	Отличается устройством, а не только шириной
Архив	Старые варианты	Их не удаляют, но и не путают с текущими

ИЗ ПРАКТИКИ

В дизайн-системе кабинета участника каждый атом лежит на своей странице, композиты собраны в шести секциях. Флоу исследования СД1 собраны в два: первый и повторный визит. Что это даёт: повторный визит идёт без согласия, и в отдельном флоу сразу видно, какие экраны в каком пути.

Имена

- **Префиксы по уровню.** «Атом: Кнопка», «Композит: Шапка». Почему: в поиске и на панели слоёв сразу видно, что перед тобой.
- **Иконки по группам.** «Иконка/навигация/назад». Почему: так их легко найти и подменять.
- **Токены по пикселям.** `spacing/16`, `radius/8`. Почему: имя сразу говорит значение, и не нужно открывать переменную, чтобы понять отступ.
- **Слои называют по смыслу.** «Заголовок», «Кнопка-главная» вместо «Frame 237». Почему: разработчик и следующий дизайнер читают структуру по именам.

Переменные и стили

- **Цвета, отступы, радиусы и размеры задают переменными.** Почему: правка в одном месте меняет все экраны.
- **У переменной ограничена область применения (scopes).** Цвет текста не предлагается для заливки фона. Почему: так в списке остаются только подходящие значения, и ошибиться труднее.
- **У переменной есть синтаксис для кода** вида `var(--color-text-muted)`. Почему: разработчик видит в режиме разработки готовое имя из кода.
- **Режимы переменных для тем.** Светлая и тёмная тема, разные акценты. Почему: один компонент работает во всех темах, и не нужно рисовать копии.
- **Свойства компонентов привязаны к переменным.** Почему: число, вбитое руками, не обновится при смене системы.

> **ИЗ ПРАКТИКИ**
>
> У цветов кабинета участника два режима: «Основной» и «Анкета». Компонент один, режим меняет акцент. Что это даёт: анкете не нужна вторая библиотека. Значит, нет и расхождений между двумя копиями.

Компоненты

- **Варианты по свойствам:** тип, размер, состояние. Почему: так компонент меняется переключателями, без подмены.

- **Все состояния в компоненте,** включая нажатие, фокус, ожидание и неактивное. Почему: разработке нужны все состояния, даже те, что редко встречаются на макетах.
- **Неиспользуемые состояния не удаляют.** Почему: они понадобятся при следующей задаче, и восстановить их дороже.
- **Иконки подменяются свойством** (instance swap). Почему: иначе для каждой иконки нужен отдельный вариант компонента.
- **Авто-лейаут везде, где есть содержимое переменной длины.** Почему: длинное название не наедет на соседний элемент.

> **ИЗ ПРАКТИКИ**
>
> Кнопка в дизайн-системе кабинета участника выросла до 90 вариантов: девять типов и пять состояний, включая нажатие и ожидание. Что это даёт: у каждого типа кнопки есть все состояния, и разработка берёт их из одного места.

Ловушки

Композит не сверили с продуктом

Если композит собран давно и не сверен с продуктом, ошибка уходит во все экраны, где он стоит.

> **ИЗ ПРАКТИКИ**
>
> Шапку кабинета участника в Figma собрали раньше экранов и не сверили с прототипом. В ней стоял кружок с подписью вместо настоящего знака, не было пункта «О центре», отступ 8 вместо 4, не было нижней границы и тени. Расхождение уехало сразу в три экрана. Нашла его автор, когда посмотрела экраны. Правило: перед тем как ставить композит на экран, сверить его с продуктом и чинить в компоненте, чтобы экземпляры подтянулись сами.

Числовые проверки не видят состава

Автоматическая проверка может показать, что все тексты на месте, хотя они лежат не в своих элементах.

> **ИЗ ПРАКТИКИ**
>
> После автоматической подмены слоёв на атомы три проверки кабинета врача были чистыми: высота кадра, сдвиг элементов, набор текстов. При этом в очереди выбранный чип фильтра подменился чипом подшага: подпись «Ждут проверки» легла в кружок, а счётчик в подпись. Набор текстов совпал, просто тексты были не в своих узлах. Поймал только взгляд на кадр. Правило: после автоматической сборки смотреть хотя бы по одному кадру на набор.

Смена варианта сохраняет размер

Если у экземпляра задана фиксированная ширина, при смене варианта она остаётся старой.

> **ИЗ ПРАКТИКИ**
>
> После смены варианта чипа на выбранный подпись «Антропометрия» полужирным шрифтом обрезалась: ширина осталась от обычного варианта. Правило: после смены варианта ставить ширину «по содержимому» и проверять наложение на соседей.

Удаление рвёт связи

Удалённая переменная или компонент не меняет вид сразу, но рвёт привязки там, где они использовались.

Почему опасно: ошибка проявится позже, когда кто-то поменяет значение и половина экранов не обновится. Перед удалением проверяют, где элемент используется, на всех страницах.

Флоу и развилки

- **Экраны в порядке пути,** слева направо или сверху вниз. Почему: путь читается как текст.
- **Стрелки только на развилках.** Почему: на прямом пути порядок и так понятен, а лишние стрелки превращают схему в паутину.
- **Подпись условия у развилки.** «Анкета вернулась», «Возраст выше порога». Почему: без условия неясно, когда человек идёт по ветке.

- **Флоу собирают из компонентов.** Почему: правка компонента обновит все экраны флоу.

> **ИЗ ПРАКТИКИ**
>
> Флоу кабинета участника собраны на атомах дизайн-системы: в моногенном флоу 26 экранов и 245 экземпляров компонентов, во флоу СД1 34 экрана и 511 экземпляров. Стрелки стоят на развилках.

Связь с кодом

- **Режим разработки (Dev Mode)** показывает значения, переменные и синтаксис для кода. Почему: разработчик не перемеряет макет линейкой.
- **Code Connect** связывает компонент Figma с компонентом в коде. Почему: разработчик видит готовый фрагмент кода вместо картинки.
- **Реестр соответствия** между фрагментами кода и компонентами. Почему: без него система и продукт расходятся за пару недель.

Дизайн из продукта и продукт из дизайна

Есть два пути, и оба рабочие:

- **Сначала Figma, потом код.** Подходит, когда продукта ещё нет. Почему: обсуждать картинку дешевле, чем переписывать код.
- **Сначала рабочий прототип, потом Figma.** Подходит, когда важна логика и состояния. Почему: Figma плохо показывает поведение, а прототип даёт проверенные значения.

> **ИЗ ПРАКТИКИ**
>
> Дизайн-система кабинета участника собрана в Figma из рабочего прототипа: значения сняты с разметки, экраны сняты из прототипа и собраны на атомах. Поэтому вёрстка по Figma и по прототипу даёт одинаковый результат.

Порядок и дисциплина

- **Одна правка, одна версия файла с описанием** (история версий Figma). Почему: к описанной версии легко вернуться.
- **Ничего не удалять без решения владельца.** Почему: то, что кажется лишним, может использоваться на другой странице.
- **Не рисовать в библиотеке временное.** Почему: всё, что попало в библиотеку, рано или поздно кто-то использует.

> **ИЗ ПРАКТИКИ**
>
> Демо-панель, которая имитирует действия врача в прототипе кабинета участника, в дизайн-систему не переносили. Довод из кейса: «всё, что попадает в библиотеку, рано или поздно кто-то использует». То, что всё же пробралось (кнопки демо-типа), удалили.

Главное в главе

- Файл разложен по страницам: основы, атомы, композиты, флоу, мобильная версия, архив.
- Значения задают переменными с областью применения и синтаксисом для кода.
- У компонента все состояния, неиспользуемые не удаляют.
- Композит сверяют с продуктом до того, как ставить на экраны.
- После автоматической сборки кадры смотрят глазами.
- Стрелки во флоу только на развилках, с подписью условия.

Вопросы для самопроверки

1. Зачем ограничивать область применения переменной?
2. Почему ошибка в композите опаснее ошибки на одном экране?
3. Что может пропустить проверка «все тексты на месте»?
4. Когда разумно собирать Figma из рабочего прототипа?

Упражнение

Откройте любой свой файл Figma. Найдите пять слоёв с именами вида «Frame 123» и переименуйте по смыслу. Найдите три числа, вбитых руками, и замените переменными. Запишите, что изменилось для разработчика.

11. Доступность

Доступный интерфейс могут использовать люди с разным зрением, слухом, моторикой и способами ввода: с клавиатуры, с экранного диктора, с увеличением, одной рукой, в перчатках, на ярком солнце. Доступность нужна всем: временные и ситуационные ограничения бывают у каждого.

Стандарт

Основной ориентир это WCAG (Web Content Accessibility Guidelines). Версия 2.1 широко используется в требованиях, версия 2.2 добавила критерии про размер цели, фокус и повторный ввод. Рабочий уровень для большинства продуктов AA.

WCAG держится на четырёх принципах:

Принцип	Что значит	Примеры
Воспринимаемость	Информацию можно воспринять	Контраст, подписи к изображениям, не только цвет
Управляемость	Интерфейсом можно управлять	Клавиатура, достаточный размер цели, без ловушек фокуса
Понятность	Интерфейс понятен	Подписи полей, понятные ошибки, предсказуемость
Надёжность	Работает с вспомогательными технологиями	Правильная разметка, роли, состояния

Контраст

Что	Минимум AA
Обычный текст	4,5 : 1

Что	Минимум AA
Крупный текст	3 : 1
Границы полей, иконки, состояния элементов	3 : 1

Частые ошибки:

- светло-серые подписи и плейсхолдеры; Почему: их делают светлыми, чтобы не спорили с текстом, и перебарщивают.
- текст на градиенте: контраст проверяют на самом светлом краю; Почему: на тёмном краю контраст хороший, а на светлом текст пропадает.
- текст на стекле и фотографиях: контраст зависит от того, что под ним; Почему: фон под текстом меняется, и в худшем месте текст не читается.
- рамки полей, которых почти не видно. Почему: человек не видит, куда вводить.

> **ИЗ ПРАКТИКИ**
>
> Анализ путей кабинета участника нашёл контраст 1,81 : 1 на краю градиента главной кнопки. Позже на новой палитре минимум контраста главной кнопки подняли с 2,14 до 4,68. Границы полей усилили с 1,48 до 3,10, серые подписи с 2,56 до 4,76. Из кейса: «Для продукта, аудитория которого статистически чаще имеет проблемы со зрением, это не бюрократия».

Исключения бывают: например, неактивная кнопка по WCAG не обязана проходить контраст. Но каждое исключение записывают явно.

Клавиатура и фокус

Всё, что можно сделать мышью, должно быть доступно с клавиатуры.

Проверка:

1. Пройти экран клавишей Tab от начала до конца.
2. Видно ли, где сейчас фокус? Почему: без видимого фокуса работать с клавиатуры невозможно.
3. Совпадает ли порядок фокуса с порядком чтения? Почему: если фокус прыгает, человек теряется.

4. Открывается ли всё по Enter и пробелу?
5. Закрываются ли окна по Esc, и возвращается ли фокус туда, откуда окно открыли? Почему: если фокус остаётся на закрытом окне, человек оказывается в начале страницы.
6. Не застревает ли фокус внутри блока? Почему: из такой ловушки нельзя выйти без мыши.

Кнопка должна быть кнопкой

Если кликабельный элемент сделан блоком с обработчиком, он не получает фокус, не называется диктором и не нажимается с клавиатуры.

> **ИЗ ПРАКТИКИ**
>
> Анализ путей кабинета участника в августе нашёл, что 84 из 142 кликабельных элементов (59 %) были блоками с обработчиком вместо кнопок. ARIA-разметки не было, окна не держали фокус и не закрывались по Esc.

Подписи и формы

Формы чаще всего ломают доступность. Правила:

- **У каждого поля видимая подпись.** Плейсхолдер подписью не считается: он исчезает при вводе.
- **Подпись связана с полем** в разметке, чтобы диктор её читал. Почему: без связи диктор прочитает «поле ввода», и человек не узнает, что вводить.
- **Обязательность** обозначена текстом или знаком с пояснением. Почему: только цвет или звёздочка без пояснения понятны не всем.
- **Ошибка** стоит у поля, говорит, что не так, и связана с полем. Почему: ошибка в другом месте страницы может остаться незамеченной.
- **Сводка ошибок** в начале формы со ссылками на поля, если ошибок несколько. Почему: в длинной форме ошибки бывают за пределами экрана.
- **Автозаполнение** для имени, телефона, почты, адреса и даты рождения. Почему: ввод с телефона долгий, а с моторными трудностями ещё дольше.
- **Повторный ввод** не требуется: если данные уже есть, их подставляют. Почему: повторный ввод это лишняя работа и лишний шанс ошибиться.

ИЗ ПРАКТИКИ

Отдельная версия прототипа кабинета участника была посвящена доступности: подписи полей, состояние выбора для экранного диктора, сводка ошибок со ссылками на поля, автозаполнение и кнопка «Изменить» в сводке перед отправкой.

Серые примеры в пустых полях

Если в пустом обязательном поле стоит серый пример вроде «170», человек может решить, что поле уже заполнено.

ИЗ ПРАКТИКИ

UX-аудит кабинета участника отнёс к серьёзным проблемам то, что пустые обязательные поля показывали серые примеры «170» и «65». Пример лучше давать подсказкой под полем.

Цвет не единственный носитель смысла

Около 8 % мужчин и около 0,5 % женщин различают цвета иначе. Поэтому статус, ошибка и выбранное состояние всегда дублируются текстом, иконкой или формой.

Размер цели

Цель касания должна быть достаточно большой. Ориентиры:

Источник	Размер
WCAG 2.2, критерий 2.5.8 (AA)	не меньше 24 × 24 CSS px или достаточный отступ
WCAG 2.1, критерий 2.5.5 (AAA)	44 × 44 CSS px
Apple HIG	44 × 44 pt
Material Design	48 × 48 dp

В рабочих инструментах для людей в перчатках или в движении цели делают ещё крупнее.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника зоны касания на телефоне не меньше 44, поля ввода с кеглем 16 до ширины 900 px, чтобы iOS не увеличивал форму. Однажды скрипт, который растягивал цели касания, растянул и круглый маркер внутри нажимаемой строки: номерки клиник на телефоне стали овалами 22 × 46. Исправили отдельной версией: маркер снова кружок 22, нажимается вся строка.

Движение и прозрачность

- Анимации уважают системную настройку «Уменьшить движение» (`prefers-reduced-motion`). Почему: у части людей движение вызывает головокружение.
- Стекло и прозрачность уважают настройку «Уменьшить прозрачность» (`prefers-reduced-transparency`): у поверхности есть плотный запасной вид. Почему: текст на размытом фоне хуже читается.
- Ничего не мигает чаще трёх раз в секунду. Почему: мигание может вызвать приступ у людей со светочувствительной эпилепсией.

> **ИЗ ПРАКТИКИ**
>
> Аудит стекла кабинета участника отметил, что у стеклянных поверхностей нет запасного вида для режима «Уменьшить прозрачность». Этот режим потом отдельно не проверялся глазами, и в отчёте это записано как непроверенное.

Язык и понятность

Доступность включает понятный текст:

- простые слова вместо терминов; Почему: термин понятен специалисту, а продуктом пользуются все.
- пометки результатов объяснены: «Повышен» без объяснения пугает;

- одинаковые вещи называются одинаково; Почему: разные слова для одного путают всех, а диктор произносит их буквально.
- инструкции не опираются только на расположение («нажмите справа») или цвет («зелёная кнопка»). Почему: человек с диктором не видит расположения. Человек с нарушением цветового зрения не различает цвет.

Как проводить аудит доступности

1. Автоматическая проверка (axe, Lighthouse, WAVE). Ловит только часть проблем. Почему начинать с неё: она дешёвая и быстро убирает очевидное.
2. Проход с клавиатуры.
3. Проход с экранным диктором (VoiceOver на Mac и iPhone, TalkBack на Android, NVDA на Windows). Почему: только так видно, понятен ли экран без зрения.
4. Увеличение до 200 % и 400 %: ничего не обрезается, нет горизонтальной прокрутки. Почему: люди со слабым зрением увеличивают страницу, и она не должна ломаться.
5. Проверка контраста всех пар текста и фона, включая состояния.
6. Проверка на телефоне одной рукой. Почему: на ходу люди держат телефон одной рукой.

Результат оформляют таблицей: критерий WCAG, где нарушено, серьёзность, как исправить.

Главное в главе

- Рабочий уровень AA: текст 4,5 : 1, крупный текст и элементы 3 : 1.
- Кликабельное делают кнопкой или ссылкой. Блок с обработчиком не годится.
- У каждого поля видимая подпись, у ошибки связь с полем.
- Цвет дублируют текстом или иконкой.
- Цели касания 44–48, для перчаток крупнее.
- Уважают «Уменьшить движение» и «Уменьшить прозрачность».
- Автоматика ловит только часть проблем, остальное проверяют руками.

Вопросы для самопроверки

1. Почему плейсхолдер нельзя считать подписью поля?
2. Где проверять контраст текста на градиенте?
3. Что должно произойти с фокусом после закрытия окна?
4. Какие проблемы не найдёт автоматическая проверка?

Упражнение

Пройдите любую форму регистрации только с клавиатуры, а затем с включённым VoiceOver или TalkBack. Запишите все места, где вы не поняли, что происходит, и оформите их таблицей с критерием WCAG.

12. Тексты в интерфейсе

Текст в интерфейсе работает как часть механики. Он объясняет, что происходит, что сделать и что будет дальше. Плохой текст ломает даже хорошо собранный экран.

Принципы

1. **Важное в начало.** Человек читает по диагонали и бросает после первых слов.
2. **Одна мысль, одно предложение.** Почему: длинное предложение приходится перечитывать, чтобы удержать начало.
3. **У действия есть кто.** «Врач проверит анкету» вместо «Анкета будет проверена». Почему: человек понимает, кто действует и чего ждать, а безличная фраза прячет ответственного.
4. **Без канцелярита.** «Чтобы» вместо «в целях», «этот» вместо «данный». Почему: канцелярит удлиняет текст и звучит как отписка.
5. **Без оценок.** «Удобно», «быстро», «интуитивно» ничего не сообщают. Почему: оценку человек делает сам, а текст должен дать факты.
6. **Нейтральный тон.** Без шуток в ошибках и без восклицательных знаков. Почему: шутка, которая одному мила, другого раздражает, особенно когда что-то сломалось.
7. **Одно слово на одну вещь** во всём продукте. Почему: разные слова для одной вещи заставляют думать, что это разные вещи.

Связка заголовка, подзаголовка и кнопки

Почти любое сообщение состоит из заголовка, подзаголовка и кнопки. Они читаются как одна фраза.

- **Заголовок** понятен сам по себе: «Заявка отправлена врачу». Почему: заголовок часто единственное, что прочитают.
- **Подзаголовок** объясняет и подсказывает, что дальше: «Проверка обычно занимает до одного рабочего дня. Мы пришлём уведомление». Почему: подзаголовок читают реже, поэтому он дополняет и не несёт главного.
- **Кнопка** продолжает мысль: «Перейти к моим исследованиям». Почему: связка из трёх частей читается как одна фраза и ведёт к действию.

Кнопки

Кнопка называет действие глаголом или задачу, которую человек решает.

Хорошо	Плохо
Записаться на визит	Далее
Отправить на проверку врачу	Готово
Пересдать, выбрать новую дату	OK
Удалить аккаунт	Подтвердить

Правила:

- кнопка отвечает на вопрос «я хочу…»; Почему: кнопка совпадает с намерением человека, и он узнаёт её сразу.
- в кнопке нет точки; Почему: кнопка это команда, и точка делает её похожей на фразу из текста.
- кнопка в окне подтверждения называет действие: «Удалить» вместо «Да»; Почему: «Да» требует перечитать вопрос, а «Удалить» понятно само.
- закрыть окно с ошибкой можно кнопкой «Понятно». Почему: ошибка не «хорошо», и «OK» звучит неуместно.

> **ИЗ ПРАКТИКИ**
>
> На шаге анкеты кабинета участника, когда все разделы заполнены, «Далее» становится «Отправить на проверку врачу» с любого раздела. Кнопка прямо говорит, что произойдёт и кто получит анкету.

Ошибки

Анатомия ошибки: что случилось и как исправить.

Хорошо	Плохо
Введите дату в формате ДД.ММ.ГГГГ	Неверный формат

Хорошо	Плохо
Выберите клинику, чтобы записаться	Ошибка: поле не заполнено
Не получилось загрузить файл. Проверьте, что он меньше 10 МБ	Что-то пошло не так

Правила:

- не начинать со слова «Ошибка»; Почему: слово «Ошибка» пугает и ничего не объясняет.
- не упрекать: «Введите 10 цифр» вместо «Вы неправильно ввели номер»; Почему: упрёк вызывает защиту. Инструкция помогает исправить.
- говорить, на чьей стороне проблема; Почему: человек должен понять, может ли он что-то исправить сам.
- «Попробуйте снова» писать, только если это поможет. Почему: бесполезный совет злит больше, чем его отсутствие.

Особенность процесса это не ошибка

Если человек ввёл правильные данные, но они не подходят по правилам процесса, это особенность процесса. Человек не ошибся. Текст объясняет причину и даёт выход.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника для возраста выше порога протокола сначала сделали плашку с объяснением и двумя выходами: указать другую дату или посмотреть другие исследования. Человек указал настоящую дату и не ошибся. Позже по первому аудиту вида запрет стали показывать красным у поля даты рождения, а отправку сделали неактивной. Это пример того, как одно правило системы (жёлтый для особенности процесса) уступило другому решению, и оба варианта стоит взвешивать на своём продукте.

Успех

- Без слова «успешно»: действие либо выполнено, либо нет. Почему: «успешно» ничего не добавляет к факту.
- Что произошло и что будет дальше: «Вы записались на 12 марта в 9:00. Накануне придёт напоминание». Почему: человеку важно и что произошло, и что будет дальше.
- Без восклицательных знаков. Почему: восклицание звучит громче, чем событие того стоит.

Статусы

Статус называет состояние, в котором объект находится сейчас.

Хорошо	Плохо
На проверке у врача	Обрабатывается
Ожидаем визита	В процессе
Требуется пересдача	Проблема
Результаты готовы	Завершено

Статус пишут с точки зрения человека: чего он ждёт и что от него нужно.

> **ИЗ ПРАКТИКИ**
> В кабинете врача-модератора срок ожидания показан фактом: «ждёт 2 ч», без красного и слова «просрочено». Норматива срока проверки ещё нет, и интерфейс не придумывает его за клинику. Место под норматив заложено.

Пустые состояния

Пустое состояние отвечает на три вопроса: что здесь будет, почему пока пусто, как это получить.

Пример: «Здесь появятся результаты анализов. Они придут после визита, обычно за две недели».

В рабочих инструментах пустое состояние может быть хорошей новостью: пустая очередь значит «на сегодня всё».

Поля и формы

- Подпись над полем называет, что вводить: «Дата рождения». Почему: подпись должна быть видна и до, и после ввода.
- Пример под полем: «Например, 20.05.2010». Почему: пример показывает формат и не путается с введённым значением.
- Плейсхолдер не заменяет подпись. Почему: плейсхолдер исчезает при вводе, и человек забывает, что вводил.
- Вопрос в анкете формулируется так, как его задал бы врач в разговоре. Почему: так вопрос понимают с первого раза.
- Чекбокс согласия начинается с глагола в первом лице: «Соглашаюсь с условиями». Так текст не зависит от рода.

> **ИЗ ПРАКТИКИ**
>
> Согласие в кабинете участника устроено двумя отметками со ссылками на документы. Кнопка не пускает дальше, пока не отмечены оба согласия. Формулы «продолжая, вы соглашаетесь» в кабинете нет.

Даты, числа и единицы

- В полях и таблицах дата `20.05.2010`, в тексте «20 мая 2010 г.». Почему: короткий формат удобен для сравнения в таблице, полный легче читать в тексте.
- Время `9:00`, интервал `9:00–11:00` коротким тире.
- Числа с пробелом в разрядах от пяти знаков: `39 290`. Почему: длинное число без пробелов трудно прочитать.
- Единицы через неразрывный пробел: `10 МБ`, `24 месяца`. Почему: число и единица не должны разрываться переносом строки.

Тексты для сложных доменов

В медицине, финансах и праве тексты особенно ответственны:

- **не пугать** пометками без объяснения; Почему: человек сам додумает худшее.
- **не обещать** того, чего система не знает: даты, сроки, результаты; Почему: несбывшееся обещание подрывает доверие.
- **не упрощать до неправды**: если исход зависит от врача, так и писать; Почему: простота, которая искажает смысл, вводит в заблуждение.
- **согласовывать** юридически значимые тексты с юристами, а интерфейсные с врачами. Почему: ошибка в юридическом или медицинском тексте стоит больше, чем ошибка стиля.

> **ИЗ ПРАКТИКИ**
>
> UX-аудит кабинета участника отнёс к серьёзным проблемам пометки результатов «Повышен» и «Выявлены» без объяснения. Человек видит слово, которое пугает, и не понимает, что делать дальше.

Словарь продукта

Словарь фиксирует термины, их значения и запрещённые синонимы. Пример строки:

Термин	Значение	Не используем
Заявка	Участие одного человека в одном исследовании	Анкета, запрос, обращение
Визит	Встреча для забора образца	Приём, посещение

Как проверять текст

1. Прочитать вслух: где спотыкаетесь, там и правка. Почему: на слух слышно то, что глаз пропускает.
2. Пройти по чек-листу: точки, кавычки «», тире, «вы» с маленькой буквы, нет капслока.

3. Проверить связку заголовка, подзаголовка и кнопки. Почему: три части читаются как одна мысль, и разрыв в ней сбивает.
4. Проверить на воду (например, сервисом «Главред»).
5. Отличать ошибку от вкуса: если правило не нарушено, это предпочтение. Почему: иначе ревью превращается в спор о вкусе.

Главное в главе

- Текст работает как часть механики интерфейса.
- Кнопка называет действие, ошибка говорит, что случилось и как исправить.
- Особенность процесса не подаётся как ошибка человека.
- Статус называет состояние с точки зрения человека.
- Интерфейс не обещает того, чего система не знает.
- Одно слово на одну вещь, словарь продукта это закрепляет.

Вопросы для самопроверки

1. Перепишите ошибку «Некорректный ввод данных» для поля телефона.
2. Чем статус «Обрабатывается» хуже статуса «На проверке у врача»?
3. Как написать пустое состояние для списка уведомлений?
4. Почему «Соглашаюсь» лучше «Я согласен»?

Упражнение

Выпишите все тексты одного сценария любого сервиса: кнопки, ошибки, статусы, подсказки. Перепишите их по правилам главы и для каждой правки запишите причину.

13. Возвращение и удержание

Многие продукты живут циклом визитов: человек приходит, уходит и должен вернуться через неделю, месяц или полгода. Удержание проектируют так же, как первый вход. Если человек не вернулся, всё сделанное в первый раз теряет ценность.

Почему люди не возвращаются

Причины обычно такие:

Причина	Что с ней делать	Почему это работает
Не узнал, что пора	Уведомления вне продукта: почта, СМС, пуш	Человек не открывает продукт сам, пока нет повода
Не смог войти	Простой вход, восстановление доступа	Забытый пароль за полгода почти гарантирован
Не понял, что делать	Первый экран показывает открытое и следующее действие	Через полгода человек не помнит, где остановился
Не увидел пользы	Показывать результат и прогресс	Люди возвращаются туда, где получили что-то ценное
Слишком тяжело	Снижать нагрузку каждого визита	Каждый барьер добавляет шанс уйти

Редкие визиты

Если человек приходит раз в полгода, он каждый раз почти новичок.

- **Первый экран отвечает «что у меня сейчас открыто».** Почему: это первый вопрос человека, который давно не заходил.
- **Следующее действие одно и заметное.** Почему: человек не помнит устройство продукта и не должен его вспоминать.

- **История доступна, но не мешает.** Почему: прошлое нужно для контекста, но работа идёт с текущим.
- **Никаких новых терминов без объяснения.** Почему: за полгода человек забыл и старые.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника все заявки семьи показаны на одном экране, по людям. Довод из кейса: первый вопрос человека, который заходит раз в полгода, «что у меня сейчас открыто». Сначала был переключатель «Все / Себе / Ребёнок». От него отказались: переключатель заявок поверх переключателя участников давал два уровня скрытого состояния.

Уведомления

Уведомление возвращает человека, когда есть повод. Правила:

- **Уведомление о событии, которое важно человеку.** «Открылась запись на визит», «Результаты готовы». Почему: уведомления без пользы учат человека их игнорировать.
- **Канал по важности.** Важное и срочное по СМС или пушу, остальное почтой, мелкое внутри продукта. Почему: тревожный канал для мелочи раздражает и снижает доверие к важным сообщениям.
- **Текст уведомления самодостаточен.** Почему: человек часто читает только уведомление и не открывает продукт.
- **Ссылка ведёт прямо к действию.** Почему: каждый лишний шаг после перехода теряет людей.
- **Разные тексты для хорошего и плохого исхода.** Почему: «Статус изменён» в случае пересдачи пугает неопределённостью.
- **Человек управляет каналами.** Почему: если нельзя отключить лишнее, отпишутся от всего.

ИЗ ПРАКТИКИ

В кабинете участника у стадий образца разные уведомления для хорошего и плохого исхода. В разборе продукта отдельно записан риск: без уведомлений вне кабинета (СМС, почта) главную метрику не достичь. Почему: запись на следующий визит открывается за месяц, и без уведомления человек через полгода не узнает, что она открылась.

ИЗ ПРАКТИКИ

После первого аудита вида в кабинете участника уведомления получили счётчик и отметку непрочитанных. Что это даёт: без них человек не отличает новое от уже прочитанного.

Напоминания

Напоминания полезны, но сами по себе удержание не создают.

ИЗ ПРАКТИКИ

В разборе исследований об удержании, сделанном для кабинета участника, есть систематический обзор 143 когорт (Teague и соавторы, 2018). Снижение барьеров, например гибкие визиты и удалённый сбор данных, связано с удержанием примерно на 10 % выше. Стратегии напоминаний связаны с потерей ещё около 10 %, но это корреляция. Возможное объяснение: напоминания чаще ставят там, где люди и так уходят. Вывод для продукта: сначала убирать барьеры, потом напоминать.

Вход

Для редких визитов вход становится главным барьером.

Способ	Плюсы	Минусы
Логин и пароль	Привычно	Пароль забывают за полгода
Код по СМС или почте	Не нужно помнить пароль	Зависит от доступа к телефону или почте

Способ	Плюсы	Минусы
Биометрия в приложении	Быстро	Только на своём устройстве
Вход через госсервис	Подтверждённая личность	Зависимость от внешней системы

ИЗ ПРАКТИКИ

В PRD кабинета участника вход раз в полгода по логину и паролю записан риском. Опора: рандомизированное исследование на 265 участниках, где приложение со входом по биометрии дало лучшее выполнение заданий, чем сайт с паролем. UX-аудит отдельно отметил вход по «имени пользователя» как серьёзную проблему. За этим стоит простая вещь: через полгода человек не помнит, какое имя придумал.

Нагрузка визита

Каждый визит должен быть настолько лёгким, насколько это возможно.

- **Показывать нагрузку заранее.** Почему: человек, который заранее знает объём, реже бросает на середине.
- **Не спрашивать повторно то, что уже известно.** Почему: повторный ввод воспринимается как неуважение ко времени.
- **Короткие повторные анкеты.** Почему: длина анкеты прямо влияет на то, начнут ли её.

ИЗ ПРАКТИКИ

Каталог кабинета участника показывает в карточке статус набора, длительность и число визитов. Почему: человек оценивает нагрузку до входа в поток. Повторный визит идёт тремя шагами, без согласия.

> **ИЗ ПРАКТИКИ**
>
> В том же разборе исследований есть работа Galesic и Bosnjak (2009) о длине опроса: при объявленных 10 минутах начали 75 % людей, при 20 минутах 65 %, при 30 минутах 62 %.

Показывать прогресс и результат

- **Где человек в общем пути.** «Визит 2», история пройденных. Почему: длинный путь без видимого прогресса кажется бесконечным.
- **Результат, ради которого человек участвует.** Почему: если польза не видна, остаётся только нагрузка.
- **Не обещать дат, которых никто не назначал.** Почему: несбывшееся обещание подрывает доверие сильнее, чем отсутствие даты.

> **ИЗ ПРАКТИКИ**
>
> План визитов в кабинете участника показывает визиты вертикальным списком. Будущие визиты идут без дат с пометкой «Ещё впереди», ориентировочная дата есть только у следующего. Подписи визитов без знаменателя: «Визит 2» вместо «Визит 2 из 4». Почему: общее число визитов назначает врач, и оно меняется.

Ожидание

Между визитами человек часто ждёт чужого действия: проверки, результата, звонка. Это ожидание проектируют.

- **Сказать, кто сейчас действует.** Почему: человек понимает, что от него ничего не требуется.
- **Сказать, сколько это обычно занимает.** Почему: ожидание без срока тревожит.
- **Пообещать уведомление.** Почему: человек может закрыть продукт и не проверять его каждый день.

> **ИЗ ПРАКТИКИ**
>
> Экран ожидания врача в кабинете участника пишет, что заявка у врача, что проверка обычно занимает до одного рабочего дня и что придёт уведомление. В кейсе это названо самой дешёвой функцией в проекте и, кажется, самой полезной.

Возвращение после выпадения

Если человек пропустил шаг, ему нужен путь обратно без вины.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора неявка участника на визит не закрывает заявку. Она превращается в задачу «связаться». Слово автора, текст участнику: «для уточнения причины с вами свяжется наш сотрудник». Довод: в кабинете участника после неявки участнику предлагают записаться заново, значит, заявка должна оставаться живой.

Измерять удержание

- **Делить выбывание по причинам,** на которые продукт влияет и на которые нет. Почему: иначе любая цифра удержания будет означать что угодно.
- **Смотреть, где именно уходят.** Почему: разные места ухода лечатся по-разному.
- **Настроить измерения до запуска.** Почему: при длинном цикле запуск без метрик теряет целый цикл.

Подробнее в главе 18.

Главное в главе

- Человек, который приходит редко, каждый раз почти новичок.
- Первый экран отвечает «что у меня сейчас открыто».
- Уведомление вне продукта возвращает человека, когда есть повод.
- Вход для редких визитов проектируют отдельно.

- Сначала убирают барьеры, потом напоминают.
- Ожидание чужого действия объясняют: кто, сколько, как узнаю.

Вопросы для самопроверки

1. Почему вход по паролю особенно плох для продукта с визитом раз в полгода?
2. Почему напоминания связаны с потерей участников в обзоре Teague, и что из этого следует?
3. Что должно быть на экране ожидания?
4. Как вернуть человека, который пропустил шаг?

Упражнение

Выберите продукт, которым пользуются редко: страховка, налоговая, медицинский кабинет. Пройдите путь человека, который вернулся через полгода. Запишите, что он должен вспомнить, где может застрять, и предложите три изменения с объяснением.

14. Этика и тёмные паттерны

Интерфейс влияет на решения людей. Этим можно помочь человеку сделать то, что он хочет, а можно подтолкнуть его к тому, что выгодно продукту и вредно ему. Продуктовый дизайнер отвечает за то, на какой стороне окажется его работа.

Тёмные паттерны

Тёмный паттерн это приём интерфейса, который заставляет человека сделать то, чего он бы не сделал, если бы понимал, что происходит. Термин ввёл Гарри Бригнулл, каталог примеров ведётся на сайте deceptive.design.

Паттерн	Как выглядит	Почему вредно
Пристыживание	«Нет, я не хочу заботиться о здоровье» на кнопке отказа	Давит на чувство вины вместо выбора
Ловушка	Подписаться в один клик, отписаться через звонок	Человек платит за то, что уже не хочет
Скрытые расходы	Сборы появляются на последнем шаге	Человек уже вложил время и соглашается через силу
Отмеченное за человека	Галочка «Подписаться на рассылку» стоит заранее	Согласие перестаёт быть согласием
Вопрос с подвохом	Двойное отрицание в отметке	Человек выбирает противоположное тому, что хотел
Навязчивость	Одна и та же просьба на каждом экране	Человек соглашается, чтобы отстали
Ложная срочность	«Осталось 2 места» без основания	Решение принимается под давлением
Спрятанная настройка	Отключение сбора данных в пятом уровне меню	Право формально есть, а пользоваться им нельзя

Почему тёмные паттерны плохи и для бизнеса: они дают короткий выигрыш в метрике и долгую потерю доверия. В ряде стран они уже прямо запрещены законами о защите потребителей и данных.

Принципы честного интерфейса

- **Отказ так же доступен, как согласие.** Почему: если «нет» спрятано, выбор ненастоящий.
- **Последствия написаны до действия.** Почему: человек должен знать, на что соглашается, до того как согласился.
- **Ничего не отмечено за человека.** Почему: согласие это действие человека. Отсутствие возражения согласием не считается.
- **Цена и условия видны с первого шага.** Почему: решение принимается на полной информации.
- **Выйти так же просто, как войти.** Почему: продукт, из которого трудно уйти, держит людей силой.
- **Срочность только настоящая.** Почему: придуманная срочность обманывает.

Согласие

В продуктах с персональными и медицинскими данными согласие особенно важно.

- **Отдельная отметка на каждое согласие.** Почему: одна галочка на всё не даёт понять, на что именно человек согласился.
- **Документ доступен рядом.** Почему: человек вправе прочитать, что подписывает, не уходя из формы.
- **Без «продолжая, вы соглашаетесь».** Почему: такая формула делает согласие незаметным.
- **Отзыв согласия доступен и понятен.** Почему: право отозвать согласие должно быть реальным.
- **Последствия отзыва написаны прямым текстом.** Почему: человек должен понимать, что потеряет.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника первый шаг не пускает дальше, пока не отмечены оба согласия. Формулы «продолжая, вы соглашаетесь» нет. Отзыв согласия в настройках показывает последствия прямым текстом, подтверждение разворачивается в строке участника. Довод против модального окна: оверлей потерял бы связь с тем, чьё согласие отзывают.

Обходные пути

Иногда заказчик просит второй путь в обход основного: загрузить бумагу, прислать по почте. Это кажется удобным, но создаёт риск.

> **ИЗ ПРАКТИКИ**
>
> Заказчик кабинета участника предлагал загрузку скана бумажного согласия. От неё отказались: это второй, менее контролируемый путь в обход основного. Почему это этический вопрос: если согласие можно получить двумя путями, на одном из них проверки слабее, и человек может оказаться в исследовании без настоящего согласия.

Опасные действия

- **Опасное действие тише главного.** Почему: человек не должен случайно нажать удаление вместо продолжения.
- **Подтверждение рядом с объектом.** Почему: так видно, что именно удаляется.
- **Последствия перечислены.** Почему: «Удалить?» не говорит, что пропадёт вместе с объектом.
- **Опасное убрано глубже, но не спрятано.** Почему: случайно не нажмёшь, но при необходимости найдёшь.

> **ИЗ ПРАКТИКИ**
>
> Удаление аккаунта в кабинете участника было крупной сплошной красной кнопкой одного веса с главным действием. Стало неброской контурной кнопкой с подтверждением в той же карточке: «Это действие необратимо». Отзыв согласия и удаление аккаунта убрали в закрытый аккордеон «Дополнительные настройки».

Не придумывать то, чего нет

Честность интерфейса касается и информации.

- **Не показывать дат, которых никто не назначал.** Почему: человек читает дату как обещание.
- **Не подсвечивать просрочку без нормы.** Почему: интерфейс придумывает правило, которого нет в процессе, и наказывает за его нарушение.
- **Не показывать ложную точность.** Почему: балл или процент там, где решение бинарное, создают видимость измерения.
- **Подписывать смоделированное.** Почему: зритель прототипа может принять придуманное за готовое решение.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора норматива срока проверки нет, поэтому «ждёт 2 ч» показано фактом, без красного и слова «просрочено». Довод из разбора: подсветка просрочки «придумывает норму». Балл полноты анкеты по образцу контент-рейтинга Ozon отклонили: медицинские критерии бинарны, а сумма создаёт ложную точность. Этапы без данных подписаны «примерное решение».

Данные и доверие

- **Собирать только то, что нужно.** Почему: каждое лишнее поле с личными данными это риск утечки без пользы.
- **Говорить, кто использует данные.** Почему: человек вправе знать, кому он их доверяет.

- **Объяснять, что будет при удалении.** Почему: часть данных может храниться по закону, и человек должен знать это заранее.

> **ИЗ ПРАКТИКИ**
>
> Конкурентный анализ кабинета участника записал урок 23andMe: участнику должно быть понятно, кто использует его генетические данные. В PRD кабинета этот урок стоит в рисках рядом с доверием к генетическим данным.

Уязвимые пользователи

Особенно осторожно проектируют для детей, пожилых людей, людей в стрессе или болезни.

- **Простой язык.** Почему: в стрессе понимание падает.
- **Без давления и срочности.** Почему: уязвимому человеку труднее сопротивляться давлению.
- **Помощь рядом.** Почему: человек должен знать, к кому обратиться, если не понимает.
- **Доверенное лицо продумано.** Почему: родитель, опекун или родственник действуют за другого, и их права должны быть ясны.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника открытыми вопросами для команды и юристов вынесены совершеннолетие ребёнка, доступ родителя и история при удалении профиля. Довод: «Команде дешевле обсудить их до разработки, чем переделывать после». Плашка «Нужна помощь?» на рабочем сайте будет открывать чат с оператором.

Как спорить о решении

Если метрика растёт от приёма, который вредит людям, полезно задать вопросы:

1. Поймёт ли человек, что произошло, если ему объяснить?
2. Согласился бы он, если бы понимал?

3. Захочет ли он вернуться после этого?
4. Готова ли команда показать этот приём публично?

Если хотя бы на один вопрос ответ «нет», приём стоит пересмотреть. Почему: доверие теряется быстро, а восстанавливается годами.

Главное в главе

- Тёмный паттерн заставляет сделать то, чего человек не сделал бы, понимая происходящее.
- Отказ так же доступен, как согласие, выход так же прост, как вход.
- Согласие отдельной отметкой, с документом и с понятным отзывом.
- Обходные пути ослабляют проверки.
- Интерфейс не придумывает норм, дат и точности.
- Для уязвимых людей простой язык, помощь и ясные права доверенных лиц.

Вопросы для самопроверки

1. Чем отмеченная заранее галочка отличается от согласия?
2. Почему подсветка просрочки без нормы нечестна?
3. Почему второй путь получения согласия опасен?
4. Какие четыре вопроса помогают проверить спорный приём?

Упражнение

Найдите в трёх сервисах, которыми пользуетесь, по одному приёму, похожему на тёмный паттерн. Для каждого опишите, кому он выгоден, чем вредит и как сделать честно.

15. Интерфейсы с ИИ внутри продукта

Всё больше продуктов используют языковые модели и другие системы ИИ: помощники в чате, подсказки при заполнении, автоматическая проверка, сводки документов. Такие функции ведут себя иначе, чем обычный код: они ошибаются уверенно, отвечают по-разному на один вопрос и не объясняют своих рассуждений. Интерфейс должен это учитывать.

Чем ИИ отличается от обычной функции

Обычная функция	Функция с ИИ	Что из этого следует для интерфейса
Один вход, один результат	Один вход, разные результаты	Нужна возможность повторить и выбрать
Ошибка видна как сбой	Ошибка выглядит как правильный ответ	Нужно показывать основания и давать проверить
Поведение описано заранее	Поведение зависит от данных	Нужны тесты на реальных примерах
Работает мгновенно	Иногда работает секунды	Нужно показывать ход работы
Не требует доверия	Требует откалиброванного доверия	Нужно честно говорить о пределах

Кто принимает решение

Главный вопрос любой функции с ИИ: кто отвечает за результат.

Роль ИИ	Пример	Когда подходит
Подсказывает	Предлагает формулировку, человек правит	Ошибка дешёвая, человек эксперт
Готовит черновик	Собирает сводку, человек проверяет	Работа объёмная, проверка быстрее создания
Проверяет за человеком	Отмечает спорное, человек решает	Ошибка дорогая, нужна вторая пара глаз
Решает сам	Сортирует обращения	Ошибка дешёвая и обратимая

Почему это важно: в дорогих решениях (медицина, деньги, право) ответственность остаётся у человека, и интерфейс должен это показывать.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора автоматическая проверка критериев построена на правилах протокола, без ИИ, но принцип тот же. Автопроверка свёрнута, пока всё сходится, и раскрывает спорное. Решение остаётся за врачом: есть «Всё равно одобрить» с обязательной причиной, а несоответствия уходят в историю. Довод из базы проекта: решение за врачом, журнал отвечает и «что», и «почему». По ICH E6(R3) исследователь отвечает за делегированные задачи.

Показывать основания

- **Рядом с выводом показывать, на чём он основан.** Почему: человек может проверить вывод и не верить ему на слово.
- **Ссылки на источники в тексте.** Почему: так видно, какое утверждение откуда взято, и легко проверить выборочно.
- **Факт и правило рядом.** Почему: коллега, который не помнит правило, всё равно проверит решение.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача критерий показывает и факт, и правило протокола: «до 20 лет по протоколу СД1». Довод: чужое исследование может проверить коллега, который протокол наизусть не помнит. Тот же приём работает для выводов ИИ: вывод без основания проверить нельзя.

Говорить о неуверенности

- **Отличать уверенный ответ от предположения.** Почему: одинаковая подача приучает доверять всему или ничему.
- **Не изображать точность процентами,** если модель их не даёт честно. Почему: «уверенность 87 %» без калибровки это ложная точность.
- **Говорить, чего модель не знает.** «По этим данным нельзя сказать». Почему: честный отказ полезнее уверенной выдумки.

Исправление и контроль

- **Любой результат можно отредактировать.** Почему: ИИ ошибается, и человек должен исправить без обходных путей.
- **Можно повторить или попросить иначе.** Почему: следующий ответ может быть лучше.
- **Можно отменить действие ИИ.** Почему: если ИИ что-то изменил, человек должен вернуть как было.
- **Обратная связь о качестве.** Почему: без сигнала о плохих ответах функцию не улучшить.

Ожидание и потоковый вывод

- **Показывать, что модель работает.** Почему: ответ может идти несколько секунд, и без индикатора человек решит, что всё сломалось.
- **Выводить ответ по мере готовности.** Почему: человек начинает читать раньше, и ожидание ощущается короче.

- **Давать остановить генерацию.** Почему: если ответ пошёл не туда, ждать конца бессмысленно.

Чат или встроенная функция

Чат кажется универсальным, но часто хуже встроенной функции.

Чат	Встроенная функция
Человек должен знать, что спросить	Функция появляется в нужном месте
Ответ нужно перенести в работу руками	Результат сразу в поле или документе
Гибко, но требует усилий	Узко, но быстро

Почему встроенная функция часто лучше: человеку не нужно формулировать задачу, продукт уже знает контекст.

Данные и приватность

- **Говорить, какие данные уходят модели.** Почему: человек вправе знать, что его данные обрабатывает внешняя система.
- **Не отправлять лишнего.** Почему: каждое лишнее поле увеличивает риск утечки.
- **Медицинские и личные данные только с явным согласием.** Почему: это требование закона и условие доверия.

Тестирование функций с ИИ

- **Набор реальных примеров с ожидаемыми ответами.** Почему: модель ведёт себя по-разному на разных данных, и проверить её можно только на примерах.
- **Трудные и крайние случаи.** Почему: на простых примерах всё работает, ломается на редких.
- **Проверка людьми, которые знают предмет.** Почему: только эксперт заметит уверенную ошибку.

- **Повторная проверка после смены модели.** Почему: новая версия модели может сломать то, что работало.

Опыт работы с ИИ-ассистентом

Многие уроки построения ИИ-функций видны на работе дизайнера с ассистентом.

> **ИЗ ПРАКТИКИ**
>
> Первая редакция аудита стекла кабинета участника, которую готовил ИИ-ассистент, по памяти требовала плотную поверхность у всего содержания. После сверки ориентиров с первоисточниками аудит переписали до правок. Урок для продукта: модель уверенно ошибается в фактах, которые «помнит», и источник нужно показывать рядом с выводом.

> **ИЗ ПРАКТИКИ**
>
> В прогоне сценариев кабинета участника три находки из 29 оказались ошибкой самой проверки. Урок: автоматический вывод нужно перепроверять вручную, когда он выглядит подозрительно.

Главное в главе

- ИИ ошибается уверенно, поэтому интерфейс показывает основания.
- В дорогих решениях ИИ подсказывает или проверяет, решает человек.
- Неуверенность называют честно, без ложных процентов.
- Любой результат можно исправить, повторить и отменить.
- Встроенная функция часто лучше чата.
- Функцию с ИИ проверяют на реальных и трудных примерах.

Вопросы для самопроверки

1. Почему вывод ИИ без основания опасен?
2. Когда ИИ может решать сам?
3. Чем встроенная функция лучше чата?

4. Почему после смены модели нужно проверять функцию заново?

Упражнение

Придумайте функцию с ИИ для любого продукта, которым пользуетесь. Опишите: кто принимает решение, какие основания показываются, как человек исправляет результат, какие данные уходят модели, на каких примерах вы её проверите.

16. Прототипирование и проверка

Прототип нужен, чтобы проверить решение дешевле, чем его построить. Чем выше риск ошибки, тем ближе к настоящему продукту должен быть прототип.

Уровни прототипа

Уровень	Что это	Что проверяет	Цена
Набросок	Бумага, доска	Структура, порядок шагов	Минуты
Каркас (wireframe)	Серые блоки без стиля	Состав экрана, навигация	Часы
Кликабельный макет	Экраны в Figma со связями	Путь, тексты, вид	Дни
Рабочий прототип	HTML или код с логикой и данными	Состояния, крайние случаи, ощущение	Дни и недели
Пилот	Настоящий продукт для части людей	Поведение и метрики	Недели

Выбор уровня зависит от вопроса. Порядок шагов проверяют наброском. Логику семьи с несколькими участниками и заявками кликабельный макет не проверит: в нём нет состояния.

Рабочий прототип

Рабочий прототип ведёт себя как продукт: хранит данные, меняет состояния, показывает ошибки. Его можно показывать заказчику, тестировать с людьми и прогонять скриптами.

Плюсы:

- видны все состояния, включая те, что никто не нарисовал;
- требования собираются быстрее, когда есть что показать;
- прототип становится источником значений для дизайн-системы;
- его можно прогонять автоматически.

> **ИЗ ПРАКТИКИ**
>
> Кабинет врача-модератора собирали прототипом во многом ради требований. Слово автора: «этот прототип нужен также для сбора требований, когда есть визуализация — требования собираются проще». Поэтому часть этапов в нём показана примерным решением и подписана так.

Прототипирование с ИИ-ассистентом

Нейросети ускоряют сборку прототипа, но не снимают ответственности за решения. Рабочая схема:

1. Дизайнер ставит задачу и описывает ограничения. Почему: без ограничений ассистент выберет решение, которое не подходит.
2. Ассистент собирает вариант.
3. Дизайнер проверяет результат глазами и проверками. Почему: ассистент ошибается уверенно.
4. Решение принимает дизайнер. Почему: ответственность за решение не делегируется.
5. Ошибки ассистента записываются правилами. Почему: записанное правило не даст повторить ошибку.

> **ИЗ ПРАКТИКИ**
>
> В кейсах автора участие ИИ описано одной формулой: автор ставила задачи, принимала решения и проверяла результат, а работа ассистента подписана «по моему запросу». Ошибки ассистента стали для него правилами. Например, после правки раскладки снимать экран не только на ширине 1440, но и окном 600.

Версии

Каждое изменение прототипа удобно выпускать новой версией:

- **мастер-версия** с постоянным именем, рабочая; Почему: команда всегда знает, какой файл рабочий.
- **архив** неизменяемых копий прежних версий; Почему: к прежней версии можно вернуться и сравнить.
- **журнал**: номер, дата, что изменено, по чьему слову. Почему: журнал отвечает, что и почему поменялось.

Рабочей версия становится по решению владельца продукта. Прежние версии не правятся: к ним можно вернуться и снять с них кадры «до и после».

> **ИЗ ПРАКТИКИ**
>
> После утверждения кабинета участника каждая правка шла отдельной версией, рабочей версия становилась только по слову автора. Это позволило вернуть два пункта первого аудита, когда на живой странице они читались хуже, и позже снять кадры для кейса адресом «версия:состояние».

Ветки и потерянные правки

Если новую версию собирают не из последней, правки теряются.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника пунктирные рамки заменили сплошными в одной версии, но следующую ветку собрали из более ранней, и пунктир вернулся. Правку пришлось повторять в двух поздних версиях. Вывод: перед сборкой новой версии проверять, из какой версии она собирается.

Проверки прототипа

Проверки бывают разными, и каждая ловит своё.

Проверка	Что ловит
Линтер прототипа	Нарушения правил разметки и стилей
Сверка с дизайн-системой	Значения, которые разошлись
Прогон логики без браузера	Ошибки в переходах состояний
Обход состояний в браузере	Поломки вёрстки в конкретных состояниях
Снимки на нескольких ширинах	Наезды, переносы, пустоты
Просмотр глазами	Иерархию, смысл, то, что не ловит автоматика
Тест с людьми	Понимание и поведение

Обход состояний

Экранов мало, состояний много. Проверка, которая открывает только экраны, пропускает большую часть поломок.

> **ИЗ ПРАКТИКИ**
>
> Для кабинета участника завели обход состояний: скрипт проходит все состояния прототипа на нескольких ширинах и снимает их. Разбор мобильной версии нашёл 16 поломок, все в состояниях мастера, куда прежние проверки не заходили.

Нарочная поломка

Если проверка всегда показывает ноль, это может значить, что она ничего не проверяет. Нужно сломать что-нибудь нарочно и убедиться, что проверка это поймала.

> **ИЗ ПРАКТИКИ**
>
> Раскладку логики кабинета участника на модули проверили так: 98 состояний, 37 288 вызовов обработчиков, ноль расхождений со старой версией. Нулю поверили только после того, как проверка поймала нарочную поломку.

Ошибки самой проверки

Проверка тоже ошибается. Если находка подозрительная, её проверяют вручную.

> **ИЗ ПРАКТИКИ**
> Из 29 расхождений прогона сценариев 3 оказались ошибкой проверки. Одна из них: кнопки «молчали», потому что регулярное выражение искало точку в конце сообщения. Другая: анкета «не доводила до первой ошибки», потому что плавная прокрутка не анимируется в скрытой вкладке браузера.

Аудит интерфейса

Аудит это экспертная проверка по ориентирам. Он не заменяет тест с людьми, зато быстрый и дешёвый.

Как проводить:

1. **Назвать ориентиры** до начала: эвристики Нильсена, WCAG, гайды Apple, Material, практики конкретных сервисов. Почему: если ориентиры выбирают после находок, их подгоняют под мнение.
2. **Сверить ориентиры с первоисточниками.** По памяти легко ошибиться. Почему: по памяти ориентиры легко исказить.
3. **Сравнивать одинаковые состояния.** Иначе находка может оказаться разницей в данных. Почему: разные состояния различаются данными, и разница в данных выглядит как ошибка.
4. **Значения брать из файла,** а не со снимка на глаз. Почему: снимок искажает размеры и цвета.
5. **Каждый пункт**: что не так, где, по какому ориентиру, что предлагается. Почему: пункт без ориентира и предложения нельзя ни принять, ни оспорить.
6. **Решение принимает владелец продукта.** Аудит предлагает. Почему: владелец держит контекст, которого у аудита нет.

Десять эвристик Нильсена

1. Видимость состояния системы.
2. Соответствие системы реальному миру.
3. Свобода и контроль пользователя.

4. Согласованность и стандарты.
5. Предотвращение ошибок.
6. Узнавание вместо припоминания.
7. Гибкость и эффективность.
8. Эстетичный и минималистичный дизайн.
9. Помощь в распознавании и исправлении ошибок.
10. Справка и документация.

> **ИЗ ПРАКТИКИ**
>
> Выводы со страницы аудитов кабинета участника: «Аудит предлагает, решаю я»; «Тихо и незаметно это разное»; «Значения сверять с файлом»; «Ориентиры сверять до правок»; «Сравнивать одинаковые состояния». Одна из неточностей аудита: «полоса на телефоне, на десктопе нет» оказалось сравнением разных состояний.

Сравнение вариантов

Когда выбирают между вариантами вида, их показывают рядом, в одинаковом состоянии, на настоящем фоне и рядом с главной кнопкой для сравнения веса.

Главное в главе

- Уровень прототипа выбирают по вопросу, который нужно проверить.
- Рабочий прототип показывает состояния и ускоряет сбор требований.
- С ИИ-ассистентом дизайнер ставит задачи, решает и проверяет.
- Каждая правка выходит новой версией, прежние не правятся.
- Обходят все состояния. Экранов для проверки мало.
- Проверку проверяют нарочной поломкой.
- Аудит предлагает, решает владелец продукта.

Вопросы для самопроверки

1. Какой уровень прототипа нужен, чтобы проверить логику семьи с несколькими участниками?
2. Зачем хранить прежние версии, если есть рабочая?
3. Почему проверка, которая всегда показывает ноль, подозрительна?
4. Что значит «сравнивать одинаковые состояния»?

Упражнение

Возьмите прототип любого своего проекта и составьте список всех его состояний. Отметьте, какие из них вы когда-либо открывали при проверке. Проверьте три, которые не открывали никогда.

17. Передача в разработку

Передача в разработку проверяет, насколько решение продумано. Всё, что не описано, разработчик решит сам, и не всегда так, как задумано.

Что передают

Что	Зачем
Спецификация	Значения, правила, поведение
Макеты или прототип	Как выглядит и как работает
Сценарии	Порядок шагов и развилки
Состояния	Пустое, загрузка, ошибка, успех, ожидание
Крайние случаи	«А что, если» с ответами
Тексты	Все строки интерфейса, включая ошибки
Токены и компоненты	Связь с кодом
Метрики и события	Что логировать для измерения
Открытые вопросы	Что ещё не решено и кто решает

Спецификация

Спецификация описывает правила и вид. Хорошая спецификация отвечает на вопросы разработчика до того, как он их задаст.

Типичные разделы:

1. Основы: сетка, отступы, шрифты, цвета токенами. Почему: основы нужны всем компонентам, их описывают первыми.
2. Компоненты: варианты, состояния, поведение.

3. Экраны: состав, порядок, адаптивность.
4. Логика: условия показа, расчёты, переходы состояний. Почему: логика не видна на картинке, её нужно описать словами.
5. Тексты и словарь.
6. Доступность.
7. Открытые решения. Почему: разработчик должен знать, что ещё может измениться.

> **ИЗ ПРАКТИКИ**
>
> Спецификация кабинета участника ведётся отдельно от прототипа. Новые правки сначала копятся в отдельном файле очереди и переносятся в спецификацию по слову автора. Открытые решения вынесены в отдельный раздел. Значения для спецификации берутся скриптом из прототипа, их не перепечатывают руками.

Логика словами

Логику описывают условиями. Картинки её не передают.

Пример:

> Кнопка «Записаться на визит» активна, если анкета принята врачом и запись открыта. Запись открывается за месяц до ориентировочной даты визита. До этого на месте кнопки подпись «Запись откроется 12 февраля».

Такую формулировку можно проверить и перевести в код без догадок.

Адаптивность

Указывают точки перелома и что меняется на каждой:

Ширина	Что меняется
1200 и шире	Полная раскладка с боковой колонкой

Ширина	Что меняется
768–1199	Боковая колонка скрывается в меню
до 767	Навигация в нижнем таб-баре, кнопки в столбик, таблицы списком

ИЗ ПРАКТИКИ

Точки перелома кабинета участника: 1100, 900, 620, 400 и 360. На телефоне вертикальный список шагов становится горизонтальным, навигация уходит в нижний таб-бар, таблица результатов становится списком. Зоны касания не меньше 44, поля с кеглем 16 до ширины 900.

Требования к интеграции

Если продукт зависит от внешних систем, требования к ним записывают заранее: что нужно получать, как часто, в каком формате.

ИЗ ПРАКТИКИ

В кейсе кабинета участника есть совет центру за пределами интерфейса: записать требования к интеграции до покупки лабораторной системы. Иначе после покупки окажется, что у системы нет программного интерфейса и стадии «в обработке» неоткуда взяться.

Что делать, если чего-то ещё нет

Продукт часто строят, когда часть систем ещё не выбрана. Тогда проектируют штатный ручной режим.

> **ИЗ ПРАКТИКИ**
>
> Лабораторная система для кабинета участника не выбрана, поэтому ручные статусы спроектированы как штатный режим. Временной заглушкой их не считают. В кабинете врача-модератора интеграций нет ни с кабинетом участника, ни с календарями, ни с лабораторией: всё смоделированное подписано, чтобы разработка не приняла его за готовое решение.

Работа с разработчиками

- **Подключать рано.** Разработчик на этапе идей экономит недели: он знает, что дорого.
- **Обсуждать.** Передача это разговор. Одной отправки файла мало.
- **Давать приоритеты.** Что обязательно, что можно упростить. Почему: разработка всегда упирается в сроки, и ей нужно знать, чем жертвовать.
- **Отвечать быстро.** Вопрос без ответа превращается в решение разработчика.
- **Принимать результат.** Сверять построенное со спецификацией по чек-листу. Почему: без приёмки расхождения уходят в продукт и остаются там.

Приёмка

Приёмка проверяет, что построено то, что задумано. Чек-лист приёмки:

- все экраны и состояния на месте;
- значения совпадают с токенами;
- тексты совпадают со словарём;
- работает с клавиатуры и диктором;
- адаптивность на всех точках перелома;
- крайние случаи из списка отрабатывают;
- события для метрик логируются.

Найденное оформляют задачами: где, что ожидалось, что получилось, снимок.

Дизайн-долг

Не всё делается сразу. Дизайн-долг записывают так же, как технический: что отложено, почему, чем грозит, когда вернуться.

Главное в главе

- Всё, что не описано, разработчик решит сам.
- Спецификация описывает правила и логику словами.
- Значения берут из источника, руками не перепечатывают.
- Требования к интеграции записывают до покупки системы.
- Если системы ещё нет, ручной режим проектируют как штатный.
- Передача это разговор, приёмка проверяет по чек-листу.

Вопросы для самопроверки

1. Почему логику лучше описывать условиями?
2. Что входит в чек-лист приёмки?
3. Как подписать в прототипе то, что смоделировано без интеграции?
4. Чем грозит вопрос разработчика, оставленный без ответа?

Упражнение

Опишите словами логику одной кнопки вашего продукта: когда она видна, когда активна, что происходит при нажатии, что видно вместо неё в остальных случаях.

18. Метрики и итерации

Метрика показывает, решила ли работа задачу. Без метрик продукт развивается по мнениям, и спор выигрывает тот, кто громче.

Типы метрик

Тип	Что показывает	Примеры
Главная (North Star)	Ценность, ради которой существует продукт	Доля участников, прошедших исследование до конца
Метрики задачи	Решена ли конкретная задача	Доля отправленных анкет, время заполнения
Контрольные	Что не должно ухудшиться	Потеря данных, число обращений в поддержку
Операционные	Нагрузка на сотрудников	Время проверки заявки врачом
Качество данных	Можно ли доверять данным	Доля отклонённых записей при импорте

Фреймворки

HEART

Фреймворк Google для пользовательского опыта:

- **Happiness**: удовлетворённость, оценки;
- **Engagement**: вовлечённость, частота использования;
- **Adoption**: освоение новых функций;
- **Retention**: удержание;
- **Task success**: успешность задач, время, ошибки.

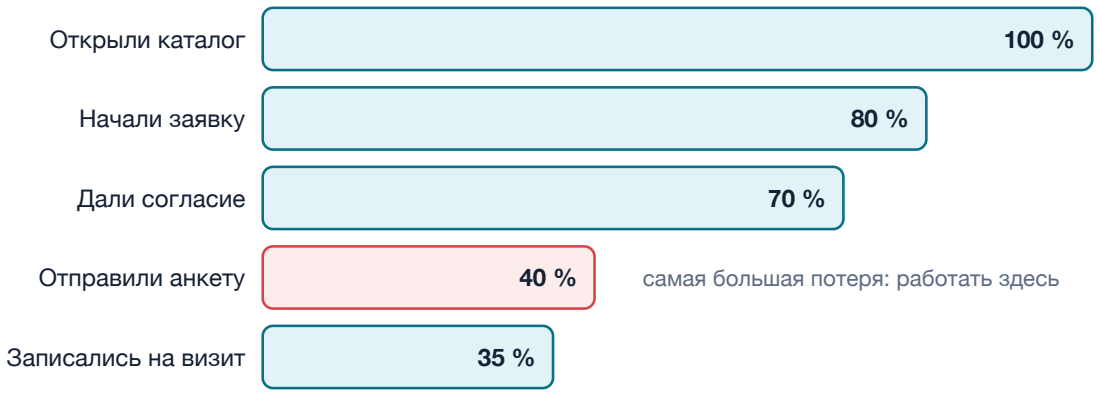
К каждой букве подбирают цель, сигнал и метрику.

AARRR

Воронка роста: привлечение, активация, удержание, доход, рекомендации. Подходит потребительским продуктам с маркетингом.

Воронка сценария

Для конкретного пути строят воронку шагов: сколько людей начали, сколько дошли до каждого шага. Место, где теряется больше всего, первое кандидатное место для работы.



Пример воронки сценария, числа условные

Связать метрику с решением

Хорошая метрика отвечает на вопрос, от которого зависит решение. Полезно держать в таблице колонку «На что отвечает».

Метрика	Как мерить	На что отвечает
Время проверки анкеты врачом	События открытия и решения	Окупается ли экран врача
Доля брошенных анкет по разделам	Последний заполненный раздел	Какой раздел сократить
Доля переставших отвечать	Нет входа за период	Менять ли напоминания и вход

> **ИЗ ПРАКТИКИ**
>
> В первом кейсе кабинета участника была таблица «Что предлагаю измерять» с колонкой «На что отвечает»: время проверки отвечает, окупается ли экран врача. Позже колонка выпала, и в разборе кейсов её рекомендовали вернуть: она связывает метрику с решением.

Правила действий по данным

Ещё сильнее, когда заранее записано, что делать при каждом исходе.

> **ИЗ ПРАКТИКИ**
>
> В кейсе кабинета участника записаны правила: «Если растёт доля переставших отвечать, менять нужно напоминания и вход. Если анкету бросают на одном разделе, его нужно сократить».

Разделить зону влияния

Метрика должна зависеть от того, что делает продукт. Если на неё влияет много внешних причин, её делят.

> **ИЗ ПРАКТИКИ**
>
> Выбывание участников в кабинете разделено на зону кабинета и всё остальное. На медицинские причины, забор крови и дорогу кабинет не влияет. Из кейса: «Если этого разделения не сделать, любая цифра удержания будет означать что угодно».

Базовый уровень

Изменение можно оценить, только если известен уровень до него. Если базового уровня нет:

- измерить его в первые недели после запуска; Почему: собственный уровень точнее любого чужого ориентира.

- взять ориентир из опубликованных исследований со ссылкой; Почему: чужой ориентир лучше пустоты, если источник проверяем.
- честно написать, что чисел нет. Почему: придуманная цифра опаснее честного пробела.

> **ИЗ ПРАКТИКИ**
>
> Данных о старом процессе кабинета участника нет: сколько анкет возвращали, сколько людей уходило. Кейс так и пишет. Цель удержания поставили сначала без ориентира, модель показала, что она выше верхнего сценария, и новую цель решили ставить от уровня, измеренного после запуска.

Метрики до запуска

Метрики настраивают до запуска. Если цикл продукта длинный, запуск без метрик теряет целый цикл.

> **ИЗ ПРАКТИКИ**
>
> Визит в исследовании бывает раз в полгода. Первые полгода без метрик означают потерянный цикл: следующая возможность сравнить будет только через шесть месяцев.

Модель эффекта

Модель эффекта оценивает, что даст решение, до того как его построят. Правила:

- **диапазон вместо точки**: худший, базовый и лучший сценарии; Почему: точка создаёт ложную уверенность.
- **источник рядом с цифрой**: ссылка на исследование в том же предложении; Почему: так цифру можно проверить, не разыскивая источник.
- **допущения названы допущениями**; Почему: читатель должен видеть, на чём держится расчёт.
- **чувствительность**: сколько стоит один пункт главной величины. Почему: так видно, какая величина сильнее всего влияет на итог и что важнее измерить.

> **ИЗ ПРАКТИКИ**
>
> Модель эффекта кабинета участника даёт диапазон от 17 до 156 человек, базовый сценарий около 70. Один пункт доли равен 40 людям. Две главные величины названы допущениями. Цена минуты врача посчитана диапазоном 210–380 часов за цикл, и разброс объяснён. О скорости работы врача кейс ничего не утверждает: время проверки не измеряли.

A/B-тесты

A/B-тест сравнивает два варианта на случайно разделённых группах. Он показывает причинный эффект, но требует условий:

- достаточно трафика, чтобы набрать нужный объём выборки; Почему: на маленькой выборке разница тонет в случайности.
- одна главная метрика, выбранная до теста; Почему: если выбирать метрику после, найдётся та, что «выросла» случайно.
- заранее рассчитанный срок и размер выборки; Почему: иначе тест останавливают, когда результат понравился.
- контрольные метрики; Почему: улучшение одного часто ломает другое.
- никаких остановок «как только стало значимо». Почему: ранняя значимость часто оказывается случайной.

В продуктах с маленькой аудиторией, например в клинических исследованиях на несколько тысяч человек, A/B-тесты часто невозможны. Тогда работают сравнение до и после, качественные методы и наблюдение за когортами.

Качество данных

Продукт, который собирает данные, отвечает и за их качество. Метрики качества:

- доля записей с ошибками;
- доля повторов;
- время от события до записи в систему;
- расхождение между системой и реальностью.

> **ИЗ ПРАКТИКИ**
>
> В станции биобанка главная ошибка была в том, что связь «пробирка — ячейка — запись» проверялась только при загрузке файла на следующий день, когда штатив уже стоял в морозильнике. Если связь потеряна, пробирка становится неопознанным биоматериалом, и обычно это выясняется через месяцы. Отсюда требование: дубликат находить в момент сканирования.

Итерации

После запуска работа продолжается циклом:

1. Посмотреть метрики и обратную связь.
2. Найти место, где теряется больше всего. Почему: работа там даст больше всего.
3. Сформулировать гипотезу.
4. Сделать изменение и измерить.
5. Записать, что узнали. Почему: иначе выученное забудут, и ошибку повторят.

Обратная связь сотрудников

В рабочих продуктах важнейший источник обратной связи это сотрудники. Удобная форма отчёта: таблица «Что сказали / Что поменяли».

> **ИЗ ПРАКТИКИ**
>
> В кейсе кабинета участника петля обратной связи показана таблицей «Что сказали сотрудники» рядом с «Что я поменяла». Так видно, что каждая реплика к чему-то привела.

Главное в главе

- Главная метрика, метрики задачи, контрольные, операционные и качество данных.
- У метрики колонка «На что отвечает» и правило действий.
- Метрику делят по зоне влияния продукта.
- Базовый уровень измеряют или честно пишут, что его нет.

- Метрики настраивают до запуска.
- Модель эффекта даёт диапазон, источники и допущения.
- A/B-тест нужен трафик, иначе другие методы.

Вопросы для самопроверки

1. Чем контрольная метрика отличается от главной?
2. Почему нельзя останавливать A/B-тест, как только результат стал значимым?
3. Как поставить цель без базового уровня?
4. Зачем делить метрику по зоне влияния?

Упражнение

Для любого сценария вашего продукта составьте таблицу из пяти метрик с колонками «Тип», «Как мерить», «С чем сравнивать», «На что отвечает», «Что делаем, если растёт или падает».

19. Процессы в команде

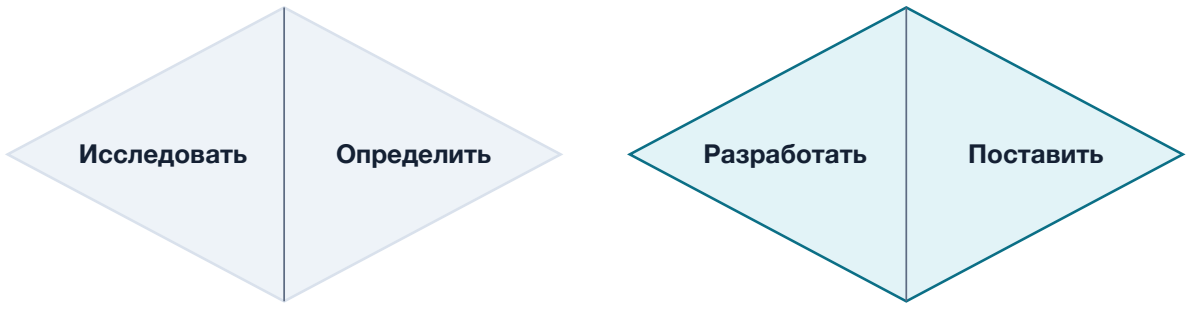
Дизайн делается не в одиночку. Даже если в команде один дизайнер, вокруг него продакт, разработка, заказчик, эксперты. Глава про то, как организовать работу, чтобы решения принимались вовремя и не терялись.

Двойной ромб

Модель «двойной ромб» (Британский совет по дизайну) делит работу на четыре фазы:

Фаза	Что делаем	Почему так
Исследовать	Расширяем понимание проблемы	Без этого решаем не ту задачу
Определить	Сужаем до одной сформулированной задачи	Без фокуса работа расползается
Разработать	Расширяем варианты решения	Первое решение редко лучшее
Поставить	Сужаем до одного решения и выпускаем	Варианты без выбора не дают продукта

Почему модель полезна: она напоминает, что расширение и сужение чередуются. Ошибка новичков в том, чтобы сразу сужать, не рассмотрев вариантов.



Двойной ромб: сначала проблема, потом решение

Исследование и поставка параллельно

В продуктовых командах часто идут два потока сразу: исследование следующей задачи (discovery) и поставка текущей (delivery). Почему: если исследовать только после выпуска, команда разработки простаивает, а если не исследовать вовсе, она строит не то.

Вопросы до работы

Перед началом задачи дизайнер собирает вопросы одним списком и задаёт их сразу.

- **Сначала прочитать всё, что уже есть.** Почему: вопросы, ответ на которые лежит в документах, тратят время заказчика.
- **Спрашивать только то, чего нет в материалах.** Почему: так вопросов меньше, и на них отвечают.
- **Задавать вопросы одним списком.** Почему: вопросы по одному растягивают начало работы на дни.
- **Не начинать, пока нет ответов на главное.** Почему: работа на догадках часто уходит в корзину.

> **ИЗ ПРАКТИКИ**
>
> Бриф кабинета участника составляли так: вместо вводных от продакт-менеджера взяли документы проекта и спрашивали только то, чего в них нет. Было два раунда вопросов, и автор сократила углубление словами «давай по найденному».

Решения записывают

- **Решение записывают с датой и тем, кто решил.** Почему: через месяц никто не вспомнит, почему сделано так.
- **Дословная формулировка лучше пересказа.** Почему: пересказ незаметно меняет смысл.
- **Отклонённые варианты тоже записывают.** Почему: иначе их предложат снова, и спор повторится.

- **Открытые вопросы отдельным списком.** Почему: так видно, что ещё не решено и кто должен решить.

> **ИЗ ПРАКТИКИ**
>
> В проектах автора решения записываются с датой и дословной репликой. Например, правило «Далее» записано словами автора: «нет, не надо блокировать кнопку далее, она должна быть секондари как и назад, а когда все поля подшага заполнены становиться праймери». Новые правки для спецификации сначала копятся в отдельном файле очереди и переносятся в спецификацию по слову автора.

Версии и журнал

- **Каждая правка новой версией.** Почему: можно вернуться и сравнить.
- **Журнал: номер, дата, что изменено, по чьему слову.** Почему: журнал отвечает на вопрос «когда это поменялось и почему».
- **Рабочая версия назначается явно.** Почему: команда должна знать, на какую версию опираться.
- **Прежние версии не правятся.** Почему: иначе история перестаёт быть историей.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника каждая правка шла отдельной версией, рабочей версия становилась только по слову автора. Это позволило вернуть два пункта первого аудита, когда на живой странице они читались хуже, и сравнивать решения кадрами «до и после».

Дизайн-ревью и критика

Ревью помогает найти проблемы до разработки. Правила:

- **Автор говорит, какую задачу решает экран и какая обратная связь нужна.** Почему: без этого ревью уходит во вкусовщину.
- **Критикуют решение, не человека.** Почему: защищаться от нападок труднее, чем обсуждать задачу.

- **Каждое замечание с причиной.** Почему: замечание без причины нельзя ни принять, ни оспорить.
- **Различать ошибку и предпочтение.** Почему: если правило не нарушено, это вкус, и решает автор.
- **Решение остаётся за автором экрана.** Почему: он держит контекст задачи целиком.

> **ИЗ ПРАКТИКИ**
> Вывод автора по аудитам кабинета участника: «Аудит предлагает, решаю я». Часть пунктов первого аудита автор вернула: на живой странице они читались хуже.

Сверка с соседними продуктами и командами

Если продукт связан с другими, процесс сверяют до рисования.

> **ИЗ ПРАКТИКИ**
> Главная находка кабинета врача-модератора: самые дорогие ошибки шли от несверенного процесса. Шаги визита придумали без кабинета участника, и запись выпала из работы модератора. Согласие сделали жёстким критерием, хотя кабинет участника и так не отправит заявку без согласий. Почему это дорого: каждую такую ошибку находили уже в готовых экранах.

> **ИЗ ПРАКТИКИ**
> Аудит кабинета врача вынес пять вопросов к процессу, и автор ответила на них в тот же день. Пятым ответом пришло задание заказчика на три маршрута записи. Запись перестроили по маршрутам: выезд водителя, офисы лабораторной сети и запись по согласованию.

Воркшопы

Воркшоп собирает людей, чтобы за короткое время прийти к общему решению.

Типичные форматы:

Формат	Для чего	Почему работает
Карта пути вместе с экспертами	Увидеть процесс целиком	Каждый знает свой кусок, вместе видна картина
Сбор идей (как можно больше вариантов)	Найти варианты решения	Количество рождает качество
Голосование точками	Быстро выбрать из многих	Видно, где согласие, без долгого спора
Приоритизация на матрице	Договориться о порядке	Видно, что ценно и что дорого
Дизайн-спринт	Проверить идею за неделю	Решение и проверка в одном цикле

Правила фасилитации:

- **Цель и результат названы в начале.** Почему: участники понимают, зачем они здесь.
- **Время на каждый этап ограничено.** Почему: без рамок обсуждение растягивается.
- **Сначала все пишут сами, потом обсуждают.** Почему: так громкие не заглушают тихих.
- **Итоги записаны и разосланы в тот же день.** Почему: на следующий день договорённости уже расходятся в памяти.

Прототип как инструмент разговора

Прототип помогает собрать требования быстрее, чем документ.

> **ИЗ ПРАКТИКИ**
>
> Слово автора о кабинете врача-модератора: «этот прототип нужен также для сбора требований, когда есть визуализация — требования собираются проще». Поэтому часть этапов показана примерным решением и подписана так.

Оценка сроков

- **Разбить работу на части,** которые можно оценить. Почему: большую задачу оценить нельзя, маленькую можно.
- **Оценивать диапазоном.** Почему: точная цифра создаёт ложную уверенность.
- **Закладывать время на проверку и правки.** Почему: проверка почти всегда находит то, что нужно переделать.
- **Сверять оценку с фактом после.** Почему: так оценки со временем становятся точнее.

Работа с ИИ-ассистентом как с участником команды

- **Ставить задачу с ограничениями и критериями готовности.** Почему: ассистент делает то, что попросили, и не знает того, что не сказано.
- **Задавать порядок работы.** Например, сначала вопросы, потом сборка. Почему: иначе ассистент начнёт строить на догадках.
- **Проверять результат.** Почему: ассистент ошибается уверенно.
- **Превращать ошибки в правила.** Почему: записанное правило не даст повторить ошибку.
- **Записывать, что сделал ассистент.** Почему: в отчёте и кейсе нужно честно разделять своё и сделанное по запросу.

Главное в главе

- Расширение и сужение чередуются: исследовать, определить, разработать, поставить.

- Вопросы собирают одним списком после чтения материалов.
- Решения записывают с датой, автором и отклонёнными вариантами.
- Каждая правка выходит версией с журналом.
- Ревью критикует решение и различает ошибку и вкус.
- Процесс сверяют с соседними продуктами до рисования.
- Прототип ускоряет сбор требований.

Вопросы для самопроверки

1. Почему отклонённые варианты нужно записывать?
2. Чем ошибка отличается от предпочтения на ревью?
3. Зачем на воркшопе сначала писать самому, а потом обсуждать?
4. Почему сроки лучше оценивать диапазоном?

Упражнение

Проведите ревью любого экрана с коллегой по правилам главы. Запишите все замечания с причинами, разделите их на ошибки и предпочтения и решите, что примете.

20. Сложные домены

Медицина, наука, финансы, право и промышленность отличаются от потребительских сервисов. Ошибка здесь стоит здоровья, денег или данных, процессы заданы регламентами, а пользователи часто специалисты. Глава собрана на примере медицинских и научных продуктов.

Чем отличается сложный домен

Особенность	Что из неё следует
Процесс задан регламентом или протоколом	Дизайнер не меняет шаги, он делает их выполнимыми
Решения принимают специалисты	Интерфейс поддерживает решение и не принимает его за человека
Ошибка дорогая и отложенная	Проверки ставят в момент действия
Несколько ролей с разными правами	Модель ролей проектируют отдельно
Длинные циклы	Человек возвращается редко и должен сразу понять, где он
Чувствительные данные	Согласия, хранение, доступ, удаление продумывают заранее
Мало пользователей	A/B-тесты невозможны, нужны качественные методы

Погружение в предметную область

Без понимания домена дизайнер рисует красиво и неправильно. Как погружаться:

- читать сами регламенты и протоколы, пересказов мало; Почему: пересказ теряет детали, на которых держатся решения.

- составить глоссарий терминов с определениями; Почему: без словаря дизайнер и эксперт говорят разными словами об одном.
- нарисовать процесс целиком со всеми ролями; Почему: каждый участник знает свой кусок, и разрывы видны только на целой картине.
- найти эксперта, который ответит на вопросы, и записывать его ответы с датой; Почему: слово эксперта с датой можно потом предъявить, когда решение оспорят.
- сверять процесс с соседними продуктами до рисования. Почему: соседний продукт часто уже решает часть задачи, и дублировать опасно.

> **ИЗ ПРАКТИКИ**
>
> Главная находка кабинета врача-модератора: самые дорогие ошибки шли от несверенного процесса. Шаги визита придумали без кабинета участника, и запись выпала из работы модератора. Согласие сделали жёстким критерием проверки, хотя кабинет участника и так не отправит заявку без согласий. Вывод: процесс сверять с соседним продуктом до рисования.

Роли

В сложных доменах роли и права важнее экранов. Вопросы:

- кто что видит;
- кто что решает и кто за это отвечает; Почему: в медицине решение и ответственность закреплены за ролью.
- как роли меняются со временем; Почему: состав команды меняется, и модель ролей должна это пережить.
- что видит человек, о котором принимают решение. Почему: человеку важно понимать, что с ним происходит, даже если он не видит внутреннюю работу.

> **ИЗ ПРАКТИКИ**
>
> В кабинете врача-модератора переключатель ролей прожил две версии. Клинические системы задают роль на пересечении человека и исследования. Набор ролей меняется: слово заказчика, «у исследования сахарного диабета может быть координатор и врач, у моногенных — только врач. потом это может поменяться после ротации кадров». Решение о приёме остаётся за врачом: по ICH E6(R3) исследователь отвечает за делегированные задачи, поэтому решение координатора должно оставаться видимым врачу.

Третий исход

В бизнес-процессах часто два исхода: да или нет. В медицине бывает третий.

> **ИЗ ПРАКТИКИ**
>
> Из записей по кабинету врача: «В продажах сделка либо движется вперёд, либо проиграна. У нас есть третий исход: брак пробы — не отказ и не движение вперёд, а возврат на шаг назад с новой записью и без вины участника». В кабинете участника пересдача показана так: вся плашка визита красная, кнопка «Пересдать — выбрать новую дату» сразу открывает запись, номер визита не растёт.

Семья и доверенные лица

Медицинские продукты часто используют за другого человека: родитель за ребёнка, взрослый за пожилого родственника. Это меняет модель данных.

Вопросы:

- один аккаунт на семью или у каждого свой; От ответа зависит вся модель данных.
- кто подписывает согласие и с какого возраста; Это задаёт, кто ставит отметку и когда её нужно поставить заново.
- что происходит при совершеннолетии; Здесь сталкиваются права родителя и права взрослого участника.
- кто видит результаты;

- что происходит с данными при удалении профиля. Часть данных может храниться по закону, и человек должен знать это заранее.

> **ИЗ ПРАКТИКИ**
>
> В кабинете участника все заявки семьи показаны на одном экране, по людям. Окно «Кого вы записываете?» позволяет добавить участника прямо при подаче заявки, без ухода в настройки. Перевод ребёнка в собственный кабинет в 18 лет отклонён: второй номер пациента ломает автоматическую рассылку отчётов. Вопросы совершеннолетия, доступа родителя и удаления профиля вынесены команде и юристам.

Согласия и персональные данные

- Согласие явное: отдельная отметка, ссылка на документ, без «продолжая, вы соглашаетесь». Почему: согласие должно быть осознанным действием.
- Отзыв согласия так же доступен, как его дача, и последствия написаны прямым текстом. Почему: право, которым трудно воспользоваться, остаётся формальным.
- Удаление данных объясняет, что удалится и что останется по закону.
- Опасные действия с аккаунтом убраны глубже и требуют подтверждения. Почему: случайное нажатие стоит дороже лишней секунды поиска.
- Обходные пути (скан бумаги, загрузка по почте) создают второй, менее контролируемый путь.

> **ИЗ ПРАКТИКИ**
>
> Загрузку скана бумажного согласия в кабинете участника предлагал заказчик. От неё отказались: это второй, менее контролируемый путь в обход основного. Отзыв согласия и удаление аккаунта убраны в закрытый аккордеон «Дополнительные настройки».

Рабочие места специалистов

Инструменты для лаборантов, врачей, операторов проектируют под их среду:

- руки заняты или в перчатках;

- работа сменой, много повторяющихся операций;
- сканеры и другие устройства ввода;
- небольшие экраны или, наоборот, несколько мониторов;
- прерывания: работа урывками между другими делами.

> **ИЗ ПРАКТИКИ**
>
> Станция биобанка работает со сканером, который ведёт себя как клавиатура. Окно, которое перехватывает фокус, проглатывает коды, поэтому в требованиях запрещены модальные окна и системные `alert` и `confirm`. Лаборант работает в перчатках и снимает их как можно реже. В разных лабораториях два способа раскладки, и станция поддерживает оба, чтобы переучиваться никому не пришлось.

> **ИЗ ПРАКТИКИ**
>
> Модель «сорок заявок подряд за смену» в симуляции не подтвердилась: врач проверяет урывками между приёмами. Это гипотеза для первых интервью, и она меняет требования к сохранению прогресса и к очереди.

Не придумывать за домен

Если в процессе нет правила, интерфейс его не изобретает.

> **ИЗ ПРАКТИКИ**
>
> Норматива срока проверки заявки ещё нет, поэтому в кабинете врача-модератора «ждёт 2 ч» показано фактом, без красного и слова «просрочено». Плана набора нет, поэтому нигде нет «X из Y». Онлайн-правок анкеты врачом нет, решение принимается только по присланному.

Ложная точность

В медицине неточная информация опаснее отсутствия информации. Даты, которые никто не назначал, сроки без гарантии, интерпретации результатов без врача вредят доверию и могут навредить человеку.

Главное в главе

- В сложном домене дизайнер делает регламент выполнимым и не меняет его.
- Процесс сверяют с соседними продуктами и экспертами до рисования.
- Роль задаётся на пересечении человека и процесса.
- Бывает третий исход, и его проектируют отдельно.
- Семья и доверенные лица меняют модель данных.
- Согласие явное, отзыв так же доступен, обходные пути опасны.
- Интерфейс не изобретает правил, которых нет в процессе.

Вопросы для самопроверки

1. Почему в сложных доменах редко работают A/B-тесты?
2. Как спроектировать удаление профиля ребёнка, у которого есть заявки?
3. Что делать, если норматива ещё нет, а интерфейсу нужен срок?
4. Чем рабочее место лаборанта отличается от потребительского приложения?

Упражнение

Выберите домен, с которым вы не работали (страхование, логистика, банк). Найдите один регламентирующий документ и выпишите из него пять ограничений, которые повлияют на интерфейс.

21. Профессия

Сделанную работу нужно уметь показать и защитить. Глава про кейсы в портфолио, аргументацию решений и работу с ИИ.

Кейс в портфолио

Кейс рассказывает, какую задачу решал автор, как и с каким результатом. Читатель оценивает автора: как он думает, что решал сам, как работает без полных данных.

Кто читает кейс

Один кейс читают разные люди, и каждый ищет своё.

Читатель	Что ищет	Что ему важно в кейсе
HR	Роль, уровень, масштаб, предметную область, ключевые слова вакансии	Лид из трёх предложений, роль перечнем работ, масштаб одной строкой, «Коротко»
Нанимающий менеджер	Проблему, ограничения, решение, подтверждение	Задача через последствие для бизнеса, находка, которая поменяла план, приоритизация, честность
Дизайнер	Решения взаимодействия, состояния, развилки, пары «до и после»	Развилки с причинами, правила системы, итерации
Аналитик	Метрики, источники, допущения, план измерения	Диапазоны, источники рядом с цифрами, проверка проверки

ИЗ ПРАКТИКИ

Разбор всех версий кейса кабинета участника глазами этих четырёх читателей показал, что всем не хватает одного и того же: находок интервью и юзабилити-тестов с участниками. Без них фраза «проверяла на интервью и юзабилити-тестах» читается как заявление.

Структура кейса

1. **Коротко.** Четыре пункта: что было, что сделано, главная находка, результат. Это кейс на две минуты. Почему: многие читатели дальше не пойдут.
2. **Роль и масштаб.** Кто автор в проекте, что делал, сколько пользователей, какой срок. Почему: HR сверяет это с вакансией.
3. **Задача.** Через последствие для бизнеса, с метриками. Почему: менеджер оценивает, понимает ли автор бизнес.
4. **Ограничения.** Что задано извне и какие решения из этого выросли. Почему: ограничения объясняют решения лучше любых слов о вкусе.
5. **Исследование.** Метод, объём, кто проводил, что нашли. Почему: без метода и объёма вывод не проверить.
6. **Главная находка.** Что поменяла в плане. Почему: находка показывает, как автор думает.
7. **Решения.** Развилки «Отказалась / Выбрала» с ценой. Почему: цена показывает, что автор взвешивал варианты.
8. **Проверка.** Как проверяли и что нашли.
9. **Результат.** Что измерено, что пока нет. Почему: честный пробел вызывает больше доверия, чем красивое обещание.
10. **Что не сделано и что бы сделал иначе.** Почему: это показывает зрелость и способность учиться.

Практики подачи

- **Развилка с ценой.** «Отказалась», «Выбрала», затем что теряет пользователь или команда.
- **Своё, общее и чужое раздельно.** «Идею предложили врачи-модераторы», «ассистент по моему запросу», «мы с командой пересмотрели решение».
- **Не утверждать неизмеренное.** «Поэтому о скорости работы врача я ничего не утверждаю».
- **Пробел с последствием.** Если чего-то нет, написать, чем это грозит.
- **Цифра рядом с источником** в том же предложении.
- **Таблица вместо хронологии.** «В первых версиях / Перед утверждением», «Что сказали сотрудники / Что я поменяла».
- **Выводы как правила.** Правило жирным, затем факт.

- **Интерфейс по рабочей версии.** Кейс описывает экраны той версии, с которой сняты кадры.
- **Разрезы под позицию.** Один проект можно подать полным, кратким и продуктовым текстом.

Чего не писать

- Оценок вместо фактов: «удобный», «интуитивный», «эффективный». Почему: оценка ничего не доказывает.
- Чисел без источника и спорных чисел. Почему: одно спорное число ставит под сомнение все остальные.
- Склеенных или неточных цитат в кавычках. Почему: неточная цитата это подлог, даже если смысл похож.
- Заголовков-афоризмов: заголовок называет тему. Почему: читатель ищет тему по заголовку.
- Безличных конструкций, которые стирают авторство. Почему: читатель должен понять, что сделал именно автор.

> **ИЗ ПРАКТИКИ**
>
> Сильный смысл можно сохранить, поменяв форму. Фразу «Продукт здесь отвечает за выполнимость исследования, а не за приятные впечатления» в разборе кейсов предложено сказать так: «Кабинет отвечает за то, чтобы исследование можно было провести до конца». Смысл тот же, конструкция проще.

Материал для кейса копят по ходу

Кейс пишется легче, если по ходу работы записывать: постановку, причины решений, дословные реплики с датами, отвергнутые варианты, где лежат артефакты. Кадры «до и после» можно снимать позже из сохранённых версий прототипа.

Аргументация решений

Решение защищают фактами и ценой. Вкус аргументом не считается.

Схема аргумента:

1. **Что выбрали.**
2. **Что отклонили** и почему, со ссылкой на источник: исследование, ограничение, правило.
3. **Что это стоит**: что теряем.
4. **Как проверим**, что выбор верный.

> **ИЗ ПРАКТИКИ**
>
> Для кнопки «Подать заявку», когда заявка уже есть, отклонили «умную» кнопку, которая сама решает, куда вести. Выбрали окно, которое всегда открывается одинаково и показывает статус рядом с каждым именем. Окно предсказуемо и отвечает, кто из семьи уже участвует. Цена: решение скучнее.

Работа с заказчиком

- Записывать решения заказчика дословно с датой. Почему: пересказ меняет смысл, и потом не разобраться, кто что решил.
- Предлагать несколько вариантов с ценой каждого. Почему: один вариант нельзя сравнить, и выбор превращается в да или нет.
- Различать решение заказчика и его предложение. Почему: предложение можно обсуждать, решение нужно выполнять.
- Возражать, если предложение создаёт риск, и объяснять какой. Почему: молчание о риске делает дизайнера соучастником ошибки.

> **ИЗ ПРАКТИКИ**
>
> Заказчик предложил загрузку скана бумажного согласия. Автор отказалась и объяснила: это второй, менее контролируемый путь в обход основного.

Работа с аудитом и чужим мнением

Аудит, эксперт и ассистент предлагают. Решает владелец продукта. Если пункт аудита на живой странице работает хуже, его можно вернуть, и это нормальная часть работы.

Работа с ИИ

ИИ-ассистенты меняют скорость работы дизайнера. Что они делают хорошо:

- собирают прототипы по описанию;
- проводят первичный аудит по ориентирам;
- сводят значения, считают контраст, ищут расхождения;
- черновики текстов и документов;
- прогоняют сценарии и состояния.

Что остаётся за дизайнером:

- постановка задачи и ограничения;
- решения и их цена;
- проверка результата глазами;
- отношения с заказчиком и командой;
- ответственность.

Правила работы:

1. **Проверять всё.** Ассистент уверенно ошибается, особенно в ориентирах по памяти.
2. **Ошибки ассистента превращать в правила** для следующих задач. Почему: записанное правило не даст повторить ошибку.
3. **Хранить версии**, чтобы откатить неудачную правку.
4. **Честно описывать участие ИИ** в кейсах. Почему: читатель должен понять, что сделал автор, а что ассистент.

> **ИЗ ПРАКТИКИ**
>
> Аудит стекла кабинета участника в первой редакции по памяти требовал плотную поверхность у всего содержания. После сверки ориентиров с первоисточниками он был переписан до правок. Отсюда правило: ориентиры сверять до правок.

> **ИЗ ПРАКТИКИ**
>
> Формулировка участия ИИ в кейсах автора: «Прототип, разборы и аудиты я делала вместе с ИИ-ассистентом Claude», автор «ставила задачи, принимала решения и проверяла результат». Работы, которые делал ассистент, подписаны «ассистент по моему запросу».

Рост в профессии

Навыки, которые отличают сильного продуктового дизайнера:

- ставит задачу через проблему и метрику;
- проверяет модель данных раньше экранов;
- держит в голове все состояния;
- пишет требования, которые можно проверить;
- работает без полных данных и честно называет допущения;
- отделяет своё от чужого;
- признаёт ошибки и пишет, что сделал бы иначе.

Главное в главе

- Кейс читают HR, менеджер, дизайнер и аналитик, и каждый ищет своё.
- «Коротко» в четыре пункта, развилки с ценой, своё и чужое раздельно.
- Неизмеренное не утверждают, цифра стоит рядом с источником.
- Решение защищают фактами и ценой.
- ИИ ускоряет работу, решения и ответственность остаются за дизайнером.

Вопросы для самопроверки

1. Что HR ищет в кейсе в первую минуту?
2. Как переписать фразу «удобный и интуитивный интерфейс» в фактах?
3. Из каких частей состоит аргумент за решение?
4. Как описать участие ИИ в кейсе честно?

Упражнение

Возьмите свой последний проект и напишите раздел «Коротко» в четыре пункта. Затем выпишите три развилки «Отказалась / Выбрала / Цена» и проверьте, что у каждой есть источник.

Приложение А. Сквозной разбор: кабинет участника

В главах кабинет участника клинических исследований встречается кусками. Здесь он разобран целиком, в порядке работы над продуктом: от задачи до метрик. У каждого решения указано, почему оно сделано так. Если причина записана в материалах проекта, она дана как довод. Если не записана, сказано, что это общий принцип.

Продукт

Кабинет нужен участникам исследований медицинского научного центра. Человек подаёт заявку за себя или за ребёнка, подписывает согласие, заполняет анкету, записывается на визит для сдачи крови и получает результаты. Участников около 4000, половина из них дети. Исследований три, скоро будет пять, у части из них повторные визиты в течение нескольких лет.

Роль автора. Продуктовый дизайнер полного цикла и дизайн-аналитик: анализ рынка, пути, логика, интерфейс, прототип, дизайн-система, сопровождение разработки и приёмка. Прототип, разборы и аудиты автор делала вместе с ИИ-ассистентом Claude: ставила задачи, принимала решения и проверяла результат.

Статус. Прототип утверждён. Запуска ещё не было, интеграции с лабораторной системой нет.

Как было до кабинета

Данные собирали через Яндекс Формы: одна анкета на сто с лишним полей на одной странице. Перед первым вопросом стояли четыре абзаца о том, к какой группе исследования себя отнести. После отправки человек не получал ни подтверждения, ни статуса, ни ответа врача. Вопросы решали перепиской по почте. Врач читал ответы сплошным списком в таблице выгрузки.

Задача

Задачу сформулировали через последствие для центра: данные участника, который ушёл на середине, почти бесполезны. У задачи три части, у каждой своя метрика.

Почему так (общий принцип). Если формулировать задачу как перенос анкеты в кабинет, успехом станет сам факт переноса. Формулировка через последствие делает успехом удержание участника, а это и есть ценность для исследования.

Ограничения

Ограничение	Что из него выросло	Почему так
Единый номер пациента	Ребёнок не переходит в свой кабинет в 18 лет	Второй номер ломает автоматическую рассылку отчётов
Лабораторная система не выбрана	Ручные статусы как штатный режим	Если ждать интеграцию, кабинет не заработает
Протокол задаёт число визитов и интервал	В интерфейсе это параметры протокола	Исследования разные, и константы пришлось бы переписывать
Половина лабораторных систем под Windows	Экран проверки анкеты у врача самодостаточный	У врача будет два рабочих места

Причины первой и четвёртой строк записаны в материалах, четвёртая в разборе 11 лабораторных систем. Во второй и третьей строке это общий принцип.

Исследование рынка

Разобраны три группы сервисов: кабинеты лабораторий, государственные сервисы и платформы клинических исследований. Отдельно разобраны 11 лабораторных систем, которые рассматривает центр.

Главный вывод. Семью и процесс исследования в одном аккаунте не совмещает никто. Поэтому семейная модель оказалась самой тяжёлой частью проекта: её не у

кого было списать.

Почему это важно (общий принцип). Где есть готовый образец, можно опереться на привычки людей. Где образца нет, решение нужно проверять особенно тщательно, и это определило, куда ушло больше всего проверок.

Главная находка: модель данных

Прогоны сценариев раз за разом находили ошибки одного рода. Возрастная блокировка прилипала к следующему участнику. Своя заявка называлась «другой». Номер у всех заявок был один. Вывод автора: «Все системные находки в проекте… это одна и та же ошибка в разных костюмах: логику писали для одного участника, а применяли к нескольким».

Решение. Первой в плане стала модель данных. Заявки хранятся по паре «участник и исследование». Решение закреплено контрольной метрикой: потеря данных при переключении равна нулю.

Почему так. Неверную модель данных приходится чинить в каждом экране и каждом сценарии. Автор записала это как то, что сделала бы иначе: «Проверять модель данных в первую неделю».

Путь заявки

Четыре шага: согласие, анкета, запись на визит, результаты. У повторного визита три шага, без согласия. Слева всегда виден список шагов, у шага цвет состояния.

Почему так. Это общий принцип мастера: человек в любой момент видит, где он и что дальше. Повторный визит без согласия, потому что согласие уже дано.

Каталог

В карточке исследования статус набора, длительность и число визитов. Под описанием строка «Заявки в семье».

Почему так. Довод из кейса: человек оценивает нагрузку до входа в поток.

Семья

Все заявки семьи на одном экране, по людям. Кнопки «Кто проходит» появляются только при двух участниках. При подаче заявки открывается окно «Кого вы записываете?».

Почему так.

- Отказались от переключателя «Все / Себе / Ребёнок»: первый вопрос человека, который заходит раз в полгода, «что у меня сейчас открыто», а переключатель заявок поверх переключателя участников давал два уровня скрытого состояния.
- Отказались от «умной» кнопки, которая сама решает, куда вести: окно всегда открывается одинаково и показывает статус рядом с каждым именем. Цена: решение скучнее.

Согласие

Две отметки со ссылками на документы. Кнопка не пускает дальше, пока не отмечены обе. Формулы «продолжая, вы соглашаетесь» нет. Скан бумажного согласия загрузить нельзя.

Почему так. От загрузки скана отказались: это второй, менее контролируемый путь в обход основного. Отдельные отметки следуют общему принципу: согласие должно быть осознанным действием.

Анкета

Девять разделов по смыслу, от 65 до 100 полей по возрасту. Вложенные вопросы раскрываются по ответу. У взрослых и детей разные ветки. Черновик сохраняется при каждом изменении с меткой времени. Индекс массы тела считается сам.

Проверка полей. «Далее» ничего не проверяет, обязательные поля проверяются один раз при отправке. Кнопка «Далее» второстепенная, пока есть пустые поля, и главная, когда раздел заполнен.

Почему так. Многие листают анкету, чтобы посмотреть, что их ждёт, и красный наказывал бы за просмотр. Слово автора: «Красный — это ошибка в анкете, а не незаполнено». Цена: об ошибках человек узнаёт позже.

Прогресс. Считает разделы, заполненные по факту. Ответ «нет» засчитывается, пропуск нет.

Почему так. Прогресс по позиции показывал бы готовность, которой нет. Подпись «шаг N из M» считается одной функцией для всех мест: раньше на одном экране стояло «шаг 0 из 3» рядом с «Текущий шаг 1 из 3».

Ожидание врача

Экран пишет, что заявка у врача, что проверка обычно до одного рабочего дня и что придёт уведомление.

Почему так. Общий принцип ожидания: сказать, кто действует, сколько это занимает и как человек узнает о результате. В кейсе это названо самой дешёвой функцией в проекте и, кажется, самой полезной.

Замечания врача

Раздел с замечаниями краснеет и показывает их число. Кнопка «К полю» открывает раздел, прокручивает, ставит фокус и подсвечивает поле. Исправил, и замечание снялось само. Чип заполненного раздела с замечанием красный.

Почему так. То, что надо доделать, важнее сделанного. Идею замечания в строке ответа предложили врачи-модераторы, и кейс так и пишет: «Сама я до неё не дошла».

Запись на визит и визиты

Сначала выбирается место: клиника центра или лаборатория. Подтверждение записи держится плашкой «Запись оформлена» до следующей стадии. Будущие визиты идут без дат с пометкой «Ещё впереди», ориентировочная дата только у следующего визита.

Почему так.

- Плашка вместо тоста: запись на визит слишком конкретное обязательство, чтобы подтверждать его тостом на три секунды.

- Без дат будущих визитов: дату, которую никто не назначал, человек читает как обещание. Побочно ушла хрупкость: арифметика дат на неверных данных роняла приложение.

Образец и пересдача

Пять стадий образца в левой панели тем же компонентом, что разделы анкеты. Пересдача: вся плашка визита красная, кнопка «Пересдать — выбрать новую дату» сразу открывает запись, номер визита не растёт.

Почему так. Второй способ показать то же самое заставил бы разбираться заново. Пересдача единственная стадия, которая требует действия, поэтому она самая заметная.

Статусы

Два правила выросли из прогона сценариев. Административный статус заявки не подменяет клинический статус результата. У статуса одна точка правды.

Почему так. Прогон нашёл, что визит получал статус «образец хорош» в момент подтверждения записи врачом, а тост и карточка показывали разное.

Визуальный язык

Стекло на фоне с молекулами, шкала цвета «морская волна», тоновый градиент только у главной кнопки, форма Material 3.

Почему так.

- Стекло: автор объясняла, что оно облегчает длинные формы и делает медицинский интерфейс спокойным.
- Изумрудный основной цвет отклонили: он сливался со статусом «успех».
- Красный только для ошибки и опасного действия: правило выросло из случая с возрастом, где человек указал настоящую дату и не ошибся.

Проверка

- Прогон 22 сценариев: 29 расхождений, 26 исправлено, 3 оказались ошибкой проверки.
- Аудиты вида: фокус, шрифт, стекло.
- Разбор мобильной версии: 16 поломок в состояниях мастера, куда прежние проверки не заходили.
- Сверка дизайн-системы с прототипом скриптом, сверку проверяют нарочной поломкой.

Почему так. Прогон ловит сбои на стыке состояний, тест с людьми показывает понимание. Урок разбора мобильной версии: «Проверка, которая не заходит в состояния, ничего не проверяет».

Приоритизация

Первая версия разобрана по MoSCoW: 57 функций, из них 28 обязательных, 14 желательных, 10 возможных и 5 отложенных. Серьёзные проблемы UX-аудита отнесены к обязательным.

Почему так. Довод из кейса: запуск с ними потерял бы людей первого цикла.

Метрики и модель эффекта

- Выбывание разделено на зону кабинета и остальное.
- Модель эффекта даёт диапазон от 17 до 156 человек, базовый сценарий около 70. Один пункт доли равен 40 людям.
- Цель удержания поставили сначала без ориентира, модель показала, что она выше верхнего сценария. Новую цель решили ставить от уровня, измеренного после запуска.

Почему так. Довод из кейса: «Если этого разделения не сделать, любая цифра удержания будет означать что угодно». Две главные величины модели названы допущениями, поэтому результат дан диапазоном.

Что не сделано

- Находок тестов с участниками в материалах проекта нет. Всем читателям кейса их не хватает.
- Открытые вопросы для команды и юристов: совершеннолетие ребёнка, доступ родителя, история при удалении профиля, общий научный отчёт семьи.
- Данных о старом процессе нет: сколько анкет возвращали, сколько людей уходило.

Чему учит разбор

1. Формулировать задачу через последствие для бизнеса.
2. Проверять модель данных в первую неделю.
3. Делать ограничения источником решений.
4. Объяснять каждое решение причиной и ценой.
5. Проверять все состояния, включая редкие.
6. Называть допущения допущениями и не утверждать неизмеренное.

Приложение Б. Глоссарий

Термин	Что значит
A/B-тест	Сравнение двух вариантов на случайно разделённых группах пользователей
AARRR	Воронка роста: привлечение, активация, удержание, доход, рекомендации
Аффинити-карта	Группировка наблюдений исследования по смыслу
Атом	Простой компонент дизайн-системы: кнопка, поле, чип
Аудит интерфейса	Экспертная проверка по заранее названным ориентирам
Базовый уровень	Значение метрики до изменения
Бриф	Короткий документ с договорённостью о задаче
Вариант компонента	Другой вид при той же роли: размер, тип
Воронка	Последовательность шагов с долей людей, дошедших до каждого
Гаттер	Расстояние между колонками сетки
Гипотеза	Утверждение «если изменить X, метрика Y изменится, потому что Z», с методом проверки
Дизайн-долг	Отложенная работа по дизайну с записанной причиной и риском
Дизайн-система	Набор токенов, компонентов, паттернов и правил с документацией и связью с кодом
Допущение	Предположение, на котором стоит расчёт или решение, без подтверждения
Интерлиньяж	Межстрочный интервал

Термин	Что значит
Каркас (wireframe)	Схема экрана без стиля
Карта пути (CJM)	Описание опыта человека по этапам: действия, мысли, каналы, боли
Композит	Компонент, собранный из атомов
Контрольная метрика	Метрика, которая не должна ухудшиться при изменении
Модель Кано	Классификация функций по влиянию на удовлетворённость
Модель эффекта	Расчёт ожидаемого эффекта решения с диапазоном и допущениями
MoSCoW	Приоритизация: Must, Should, Could, Won't
North Star	Главная метрика ценности продукта
Очередь	Список дел, ждущих действия конкретного человека
Паттерн	Известное решение повторяющейся задачи интерфейса
Персона	Описание типичного пользователя, основанное на данных
PRD	Документ с требованиями к продукту
Примитив (токен)	Токен, который называет значение: `blue/600`
Прогон сценариев	Проход всех путей и состояний прототипа руками или скриптом
Реестр	Полный список объектов, независимо от того, ждут ли они действия
Реестр сопоставления	Таблица связи фрагментов кода и компонентов дизайн-системы
RICE	Приоритизация по охвату, влиянию, уверенности и усилиям

Термин	Что значит
Рабочий прототип	Прототип с логикой, данными и состояниями
Scope	Объём работы версии: что входит и что нет
Семантический токен	Токен, который называет роль: `color/text/muted`
Сценарий (user flow)	Путь человека к цели с развилками
Состояние	Вариант экрана или компонента в зависимости от данных и действий
Спецификация	Описание правил, значений и логики для разработки
Статус	Название состояния объекта, видимое человеку
Токен	Именованное значение дизайн-системы
HEART	Фреймворк метрик опыта: удовлетворённость, вовлечённость, освоение, удержание, успешность задач
JTBD	Jobs to be Done: описание задачи, ради которой человек использует продукт
WCAG	Международные правила доступности веб-содержимого
Юзабилити-тест	Наблюдение за тем, как человек выполняет задачи в интерфейсе

Приложение В. Чек-листы

Постановка задачи

- [ ] Проблема описана через последствие для бизнеса и человека
- [ ] Названы пользователи и роли
- [ ] Есть главная метрика и контрольные метрики
- [ ] Известен базовый уровень или способ его получить
- [ ] Ограничения записаны с источником
- [ ] Есть раздел «Что не делаем»
- [ ] Открытые вопросы собраны и у каждого есть кто решает

Сценарий и состояния

- [ ] Сценарий начинается с цели человека
- [ ] Показаны все развилки и возвраты
- [ ] У каждого экрана есть пустое, частичное, ошибочное состояние и ожидание
- [ ] Статусы не обгоняют реальность, у статуса одна точка правды
- [ ] Нет дат и сроков, которых никто не назначал
- [ ] Проверены вопросы «а что, если участников двое», «а что, если человек вышел посреди», «а что, если удалить»
- [ ] Модель данных проверена на нескольких пользователях и объектах

Экран

- [ ] Одна главная задача и одна главная кнопка

- [ ] Фокус совпадает с главным действием (проверка размытием)
- [ ] Отступы кратны шагу сетки, близкое сгруппировано
- [ ] Шрифт и цвет по ролям, у роли один вид
- [ ] Красный только для ошибки и опасного действия
- [ ] «Отмена» нейтральная, опасное действие тише главного
- [ ] Цвет проверен на том фоне, где будет жить
- [ ] Экран просмотрен целиком на десктопе и телефоне

Доступность

- [ ] Контраст текста 4,5 : 1, крупного текста и элементов 3 : 1
- [ ] Всё доступно с клавиатуры, фокус виден, порядок логичный
- [ ] Окна закрываются по Esc, фокус возвращается
- [ ] Кликабельное сделано кнопкой или ссылкой
- [ ] У каждого поля видимая подпись, ошибки связаны с полями
- [ ] Цвет дублирован текстом или иконкой
- [ ] Цели касания не меньше 44
- [ ] Уважаются «Уменьшить движение» и «Уменьшить прозрачность»

Тексты

- [ ] Важное в начале, одна мысль в предложении
- [ ] Кнопки называют действие глаголом
- [ ] Ошибка говорит, что случилось и как исправить
- [ ] Нет слова «успешно» и восклицательных знаков

- [ ] Статус называет состояние с точки зрения человека
- [ ] Одно слово на одну вещь, совпадает со словарём
- [ ] Кавычки «», даты в нужном формате, «вы» с маленькой буквы

Прототип и проверка

- [ ] Правка выпущена новой версией, есть строка в журнале
- [ ] Новая версия собрана из последней рабочей
- [ ] Пройдены все состояния, включая редкие
- [ ] Снимки на нескольких ширинах
- [ ] Проверка проверена нарочной поломкой
- [ ] Подозрительные находки проверены вручную
- [ ] Результат просмотрен глазами

Готовность к передаче в разработку

- [ ] Спецификация описывает значения токенами и логику условиями
- [ ] Все состояния и крайние случаи описаны
- [ ] Тексты всех строк, включая ошибки
- [ ] Точки перелома и что меняется на каждой
- [ ] События для метрик
- [ ] Смоделированное без интеграции подписано
- [ ] Открытые вопросы перечислены

Кейс в портфолио

- [ ] «Коротко» в четыре пункта
- [ ] Роль и масштаб в одной строке
- [ ] Задача через последствие, у частей задачи метрики
- [ ] Развилки «Отказалась / Выбрала» с ценой и источником
- [ ] Своё, общее и чужое раздельно, участие ИИ описано
- [ ] Цифры с источником, спорных чисел нет
- [ ] Заголовки называют тему
- [ ] Есть «Что не сделано» и «Что сделала бы иначе»

Приложение Г. Ответы к вопросам для самопроверки

Ответы короткие. Если вопрос про ваш собственный проект, дан ориентир, как на него ответить.

1. Что такое продуктовый дизайн

1. UI-дизайнер отвечает за то, как устроен экран. Продуктовый дизайнер отвечает за то, решает ли продукт задачу человека и бизнеса, и доводит решение до измеренного эффекта.
2. Например, мгновенный результат анализа в кабинете, когда лаборатория выдаёт его через неделю. Такое решение откладывают или ищут выполнимую часть: честно показать стадию и срок.
3. Ориентир: закон, протокол, техника, сроки, деньги. Для каждого ограничения назовите документ или человека, от которого оно пришло.
4. Записать цель как допущение, указать, как и когда будет измерен базовый уровень, и поставить окончательную цель от измеренного уровня.

2. Исследования

1. «Почему бросают» узнают интервью и юзабилити-тестом. «Сколько бросают» показывает аналитика: доля людей, у которых последний заполненный раздел третий.
2. «Расскажите, как вы узнаёте, что пора записаться на следующий визит» или «Что было в последний раз, когда вы пропустили запись?».
3. Наблюдение фиксирует, что произошло. Находка объясняет, почему, и подтверждается несколькими наблюдениями.
4. Симуляция повторяет допущения того, кто её построил. О реальных людях она нового не говорит, и подача её как исследования вводит в заблуждение.

3. Постановка задачи

1. Например: «Человек находит заявку по фамилии участника или номеру за один запрос, первый результат подходит в 9 случаях из 10».
2. У гипотезы есть изменение, метрика, причина и способ проверки, назначенные заранее. Мнение ничем этим не обладает.
3. Ориентир: функция, отсутствие которой вызовет жалобы, но наличие которой никто не похвалит. Проверка: опрос по модели Кано или анализ обращений в поддержку.
4. Когда готовые системы уже решают нужную операцию, а их внедрение и доработки дешевле собственной разработки. Сравнение делают явно, по параметрам.

4. Информационная архитектура и сценарии

1. Пациент, врач, клиника, услуга, слот расписания, запись, напоминание. У врача много слотов, у пациента много записей, запись связывает пациента, врача и слот.
2. Пустое состояние, ожидание чужого действия, частичное заполнение. Также часто забывают ошибку загрузки и экран без доступа.
3. Человек читает дату как обещание. Если она не сбудется, доверие к продукту падает. К тому же человек может перестроить планы впустую.
4. В очереди только то, что ждёт действия конкретного человека. Реестр хранит всё. Очередь отвечает «что мне делать», реестр отвечает «что вообще есть».

5. Формы и анкеты

1. Чекбокс не различает «нет» и «не ответил». Два переключателя требуют явного выбора.
2. Сразу проверяют формат в коротких формах. При отправке проверяют в длинных анкетах, которые люди листают, чтобы посмотреть, что их ждёт.
3. Без метки человек не верит автосохранению и боится закрыть страницу.
4. Человек не должен отдельно отмечать исправленное. Врач при этом видит актуальное состояние без лишних замечаний.

6. Интерфейс

1. Размыть снимок или прищуриться и посмотреть, что видно первым. Это должно быть главное действие экрана.
2. Человек путает главную кнопку со статусом «успех» и не понимает, что уже сделано и что ему предлагают сделать.
3. Когда действие важное и обязывающее: запись, оплата, отправка документа. Тост исчезнет, а человеку нужно подтверждение, которое останется.
4. Поток по одному шагу вместо панели, нижний таб-бар вместо меню, лист снизу вместо окна, список вместо таблицы, крупные зоны касания.

7. Движение и микровзаимодействия

1. Иначе человек не понимает, что запрос идёт, и нажимает повторно. Система получает два запроса.
2. Окно крупное, и быстрое движение крупного объекта выглядит рывком. Подсветка мелкая и должна ощущаться мгновенной.
3. Отмена лучше для частых обратимых действий. Окно нужно, когда действие необратимо или есть риск: несоответствие, удаление.
4. Заменить перемещения и масштабирование на появление без сдвига или убрать анимацию.

8. Рабочие места и интерфейсы с данными

1. Человек работает каждый день и делает сотни повторов. Ему важнее видеть больше сразу, чем красиво понять с первого раза.
2. Кто, что ждёт, сколько ждёт и кнопка с названием задачи.
3. Пакетом легко пропустить несоответствие, которое система сама считает. Решение за специалистом должно приниматься по каждому объекту.
4. Отсканированная пробирка может остаться на столе. Если считать её размещённой сразу, база разойдётся с тем, что стоит в штативе.

9. Дизайн-системы

1. Примитив называет значение (`blue/600`), семантический токен называет роль (`color/action/primary`). Компоненты ссылаются на роль.
2. Вариант нужен, когда роль та же, а вид другой. Отдельный компонент нужен, когда роль или поведение другие.
3. Они понадобятся в следующей задаче, а восстанавливать дороже, чем хранить.
4. Иерархию: что главное на экране и не уехало ли главное действие вниз.

10. Работа в Figma

1. В списке остаются только подходящие значения, и ошибиться труднее.
2. Композит стоит на многих экранах, и его ошибка повторяется на всех.
3. Тексты могут лежать не в своих элементах: подпись в кружке, счётчик в подписи.
4. Когда важна логика и состояния, а прототип уже работает и его значения проверены.

11. Доступность

1. Он исчезает при вводе, часто слишком бледный и не всегда читается экранным диктором как подпись.
2. На самом светлом краю градиента под текстом.
3. Фокус возвращается на элемент, который открыл окно.
4. Понятность текста, логичность порядка, смысл подписей, удобство с диктором. Автоматика проверяет наличие. Смысл она не оценивает.

12. Тексты в интерфейсе

1. «Введите номер телефона из 10 цифр, например 9161234567».
2. «Обрабатывается» не говорит, кто действует и сколько ждать. «На проверке у врача» отвечает на оба вопроса.
3. «Здесь появятся уведомления о заявках и визитах. Пока новых нет».
4. «Соглашаюсь» не зависит от рода и звучит как действие самого человека.

13. Возвращение и удержание

1. За полгода пароль почти наверняка забудут, и вход станет барьером на каждом визите.
2. Это корреляция: напоминания чаще ставят там, где люди и так уходят. Следствие: сначала убирать барьеры, потом напоминать.
3. Кто сейчас действует, сколько это обычно занимает и как человек узнает о результате.
4. Оставить путь живым без вины: предложить новую запись, связаться, узнать причину.

14. Этика и тёмные паттерны

1. Галочка, поставленная за человека, означает отсутствие возражения. Согласие это действие самого человека.
2. Интерфейс придумывает правило, которого нет в процессе, и наказывает за его нарушение.
3. На одном из путей проверки слабее, и человек может оказаться в процессе без настоящего согласия.
4. Поймёт ли человек, что произошло? Согласился бы, если бы понимал? Захочет ли вернуться? Готова ли команда показать приём публично?

15. Интерфейсы с ИИ внутри продукта

1. ИИ ошибается уверенно, и без основания ошибку невозможно заметить.
2. Когда ошибка дешёвая и обратимая: сортировка, черновик, подсказка.
3. Человеку не нужно формулировать задачу, продукт знает контекст, результат сразу встаёт в работу.
4. Новая модель может по-другому отвечать на те же данные и сломать то, что работало.

16. Прототипирование и проверка

1. Рабочий прототип с данными и состояниями. Кликабельный макет логику не покажет.

2. Чтобы вернуться, сравнить, снять кадры «до и после» и понять, когда что поменялось.
3. Ноль от сломанной проверки выглядит так же, как ноль от исправного продукта.
4. Сравнивать экраны с одинаковыми данными и в одном режиме. Иначе разница в данных выглядит как ошибка интерфейса.

17. Передача в разработку

1. Условие можно проверить и перевести в код без догадок. Картинка показывает один случай.
2. Экраны и состояния, значения токенов, тексты, клавиатура и диктор, адаптивность, крайние случаи, события метрик.
3. Пометкой «примерное решение» или подписью, что этап смоделирован без интеграции.
4. Разработчик решит сам, и не всегда так, как задумано.

18. Метрики и итерации

1. Главная показывает ценность, ради которой делается работа. Контрольная следит, чтобы улучшение не сломало другое.
2. Ранняя значимость часто случайна. Если остановиться в удачный момент, вывод окажется ложным.
3. Записать цель как допущение, измерить базовый уровень в первые недели и поставить цель от него. Можно опереться на опубликованный ориентир со ссылкой.
4. Иначе в цифре смешаются причины, на которые продукт влияет, и те, на которые не влияет, и она перестанет что-либо означать.

19. Процессы в команде

1. Иначе их предложат снова, и спор повторится без новых доводов.
2. Ошибка нарушает правило или задачу. Предпочтение это вкус, и решение по нему за автором экрана.
3. Так громкие участники не заглушают тихих, и идей больше.

4. Точная цифра создаёт ложную уверенность, диапазон честно показывает неопределённость.

20. Сложные домены

1. Пользователей мало, ошибки дорогие, и экспериментировать на людях с медицинскими решениями нельзя.
2. Предупредить, что будет с заявками и результатами, дать выбор, подтверждать в два шага, сохранить данные, которые нужно хранить по закону, и не переносить заявки на другого ребёнка.
3. Показать факт без оценки («ждёт 2 ч»), заложить место под норму и не подсвечивать просрочку.
4. Руки заняты, перчатки, сканер как клавиатура, смена из сотен повторов, сверка с физическими объектами.

21. Профессия

1. Роль, уровень, масштаб, предметную область, работу с ИИ и ключевые слова вакансии.
2. Назвать измеримое, например (числа условные): «время заполнения сократилось с 25 до 12 минут на тесте с 8 людьми» или «8 из 8 участников записались без подсказки». Если чисел нет, описать, что проверяли и как.
3. Что выбрали, что отклонили и почему со ссылкой на источник, что это стоит, как проверим.
4. Разделить, что делал автор и что ассистент по запросу. Автор ставит задачи, принимает решения и проверяет результат.

Приложение Д. Литература и источники

Список собран как отправная точка. Перед цитированием сверяйте издание и формулировку по первоисточнику.

Книги

Основы и мышление

- Дон Норман. «Дизайн привычных вещей» (The Design of Everyday Things).
- Стив Круг. «Не заставляйте меня думать» (Don't Make Me Think).
- Алан Купер. «Об интерфейсе» (About Face).
- Джефф Готельф, Джош Сейден. «Lean UX».

Продукт и постановка задачи

- Марти Каган. «Вдохновлённые» (Inspired).
- Тереза Торрес. «Continuous Discovery Habits».
- Клейтон Кристенсен и соавторы. «Конкурируя с удачей» (Competing Against Luck), про Jobs to be Done.
- Алан Клемент. «When Coffee and Kale Compete», про Jobs to be Done.

Исследования

- Роб Фицпатрик. «Спроси маму» (The Mom Test).
- Стив Портигал. «Interviewing Users».
- Эрика Холл. «Just Enough Research».
- Инди Янг. «Mental Models».

Интерфейс и визуальный язык

- Адам Уотан, Стив Шогер. «Refactoring UI».
- Ян Чихольд. «Новая типографика».
- Йозеф Мюллер-Брокманн. «Модульные системы в графическом дизайне».

- Джефф Джонсон. «Designing with the Mind in Mind», про психологию восприятия.

Дизайн-системы

- Алла Холматова. «Design Systems» (Smashing Magazine).
- Брэд Фрост. «Atomic Design».

Тексты

- Максим Ильяхов, Людмила Сарычева. «Пиши, сокращай».
- Торри Подмаджерски. «Strategic Writing for UX».

Метрики

- Алистер Кролл, Бенджамин Йосковиц. «Lean Analytics».
- Рон Кохави, Диана Тан, Я Сюй. «Доверительное A/B-тестирование» (Trustworthy Online Controlled Experiments).

Гайды и стандарты

- WCAG 2.2: w3.org/TR/WCAG22.
- Apple Human Interface Guidelines: developer.apple.com/design/human-interface-guidelines.
- Material Design 3: m3.material.io.
- Microsoft Fluent 2: fluent2.microsoft.design.
- IBM Carbon Design System: carbondesignsystem.com.
- GOV.UK Design System: design-system.service.gov.uk, сильные образцы форм и текстов.

Исследовательские центры и статьи

- Nielsen Norman Group: nngroup.com, статьи по эвристикам, юзабилити-тестам, формам.
- Baymard Institute: baymard.com, исследования форм и оформления заказа.
- Google HEART: статья «Measuring the User Experience on a Large Scale» (Kerry Rodden и соавторы, 2010).

Инструменты проверки

- Контраст: WebAIM Contrast Checker.
- Доступность: axe DevTools, Lighthouse, WAVE.
- Тексты: «Главред» (glvrd.ru), «Грамота.ру» (gramota.ru).